

SAPIENZA
UNIVERSITÀ DI ROMA

Short Propositional Proofs are Hard to Find: a Meta-Complexity Approach

Faculty of Information Engineering, Computer Science and Statistics
Master's Degree in Computer Science

Simone Bianco
ID number 1986936

Advisor
Prof. Nicola Galesi
Prof. Massimo Lauria

Academic Year 2025/2026

Short Propositional Proofs are Hard to Find: a Meta-Complexity Approach
Master's Thesis. Sapienza University of Rome

© 2026 Simone Bianco. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: bianco.simone@outlook.it

«It is by logic that we prove, but by intuition that we discover.»
— Henri Poincaré

Abstract

Modern SAT solvers decide formulas with millions of variables and, whenever they report that a formula is unsatisfiable, they implicitly construct refutations of them in some proof system. Whether such proofs can always be found efficiently is the question of automatability. A proof system Q is said to be automatable if a proof of any tautology can be found in time polynomial with respect to the size of its shortest Q-proof. Under standard cryptographic assumptions, strong systems such as Frege are not automatable. In recent years, a line of work started by Atserias and Müller extended this by showing that Resolution and many weak systems are not automatable unless $\mathsf{P} = \mathsf{NP}$. Their key tool comes from meta-complexity: the formula $\mathrm{Ref}_s(\varphi)$, stating that φ has a Resolution refutation of size at most s, is easy to refute when φ is satisfiable and exponentially hard to refute otherwise. Arteche, Atserias, de Rezende and Khaniki recently turned this argument around and introduced the Proof Analysis Problem: given a proof of a Resolution lower bound for φ, decide whether φ is satisfiable. A proof system for which this is possible in polynomial time is called analyzable.

This thesis presents these results in a self-contained way to then extend them by investigating analyzability of proof systems above Resolution. In particular, we prove that Polynomial Calculus with Resolution (PCR) is analyzable, hence so are Nullstellensatz and Polynomial Calculus, and obtain a new proof that automating PCR is NP-hard. This is the first analyzability result for a system not below Resolution. We also compare analyzability with feasible interpolation, another property closely related with automatability. We show that proving that feasible interpolation implies analyzability would show that Resolution is not weakly automatable unless $\mathsf{P} = \mathsf{NP}$. Conversely, for every disjoint NP-pair we construct an analyzable proof system that has feasible interpolation only if the pair is polynomial-time separable. This last result shows that, under standard assumptions, analyzability does not imply feasible interpolation.

Contents

Introduction **1**

History of the problem 1

Contributions 3

Thesis outline 4

1 Preliminaries **5**

1.1 Propositional logic 5

1.2 Encodings of combinatorial principles 7

1.3 Computability 8

1.4 Computational complexity 10

1.5 Decision trees 13

2 Proof complexity **15**

2.1 Propositional proof systems 15

2.2 Common proof systems 16

2.2.1 Resolution and Frege systems 16

2.2.2 Algebraic and geometric systems 19

2.2.3 Tree-like systems 21

2.3 Comparing the strengths of different proof systems 23

2.3.1 Complexity measures for proof systems 23

2.3.2 Simulations between proof systems 25

2.4 Game characterizations of Resolution measures 27

3 Automatability **32**

3.1 The quest for finding proofs efficiently 32

3.2 Quasi-automatability of Tree-like Resolution 34

3.3 Non-automatability of strong proof systems 36

3.4 Non-automatability of weak proof systems 39

3.4.1 The Ref formula 39

3.4.2 An upper bound on Ref for satisfiable formulas 42
3.4.3 Decision tree reductions 44
3.4.4 The retraction pigeonhole principle 47
3.4.5 Reduction from rPHP to Ref for unsatisfiable formulas 48
3.4.6 A lower bound on Ref for unsatisfiable formulas 51

4 Analyzability **54**
4.1 Do proofs of lower bounds leak information? 54
4.2 Search-analyzability of Resolution 56
4.3 Extending analyzability to algebraic systems 63
4.4 Is analyzability linked to feasible interpolation? 67

Conclusions **72**

Acknowledgements **74**

Bibliography **77**

Introduction

History of the problem

It's 1956. The genius mathematician John von Neumann is hospitalized at the Walter Reed Army Medical Hospital in Washington, USA. One year earlier, a mass was found near his collarbone, which turned out to be cancer, probably caused by exposure to radiation at the Los Alamos National Laboratory during World War II [Mac92; Pou92]. After his condition improved thanks to a successful treatment, his colleague and friend Kurt Gödel sent a letter to express solidarity for his diagnosis, hoping for a full recovery. To keep von Neumann busy during his hospital stay, Gödel proposed to his friend a seminal problem of fundamental interest both for logic and computer science.

> «*One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n, allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\psi(F, n)$ be the number of steps the machine requires for this and let $\varphi(n) = \max_F \psi(F, n)$. The question is how fast $\varphi(n)$ grows for an optimal machine. One can show that $\varphi(n) \geq kn$.*
> *If there really were a machine with $\varphi(n) \sim kn$ (or even $\sim kn^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem.*»
>
> Gödel to von Neumann, Princeton, 20 March 1956 [Har89; Sip92]

In a less formal way, Gödel's question asks if there is an efficient algorithm capable of deciding if a given mathematical statement has a proof of some given length. In modern times, the above problem became known as the *Proof Size Problem* (PSP) [Iwa97; ABM+01; AAR+25]. Sadly, no reply from von Neumann is known. During his long hospital stay, von Neumann's mental health rapidly declined due to his extreme thanatophobia, the fear of impending death [Mac92]. Almost a year later after being hospitalized, von Neumann died [Pou92; Mac92].

When restricted from first-order predicate logic to propositional logic, Gödel's question becomes a historical precursor of the P *vs.* NP question, which asks whether

problems whose solutions are easy to double-check, i.e. problems that lie in the class NP, are also easy to solve from scratch, i.e. if they also lie in the class P [Coo71; Lev73]. The most interesting problems in NP are the so-called NP-complete problems, that is, problems in NP whose efficient solution would imply an efficient solution for all NP problems, i.e. P = NP. The current state of the art conjectures that P ≠ NP, meaning that these so-called complete problems cannot be efficiently solved in their general case (see Section 1.4 for more detailed information).

If this were the case, no algorithm could solve them in polynomial time and the best known algorithms are not much better than brute force. For instance, if a problem asks to determine if there is a sub-structure with a specific property, the brute force approach tests every possible sub-structure. In his letter, Gödel himself states that he sees no better approach than that of reducing the search space through clever techniques. Over the years, researchers began to explore the problem for *propositional proof systems* [CR79; Kra19; GT26], i.e. logical systems that use restricted rules to find proofs of propositional statements, showing that deciding whether short proofs exist is NP-hard already for Resolution [Iwa97; ABM+01].

Although P ≠ NP would rule out efficient proof-search algorithms, the practical demand for Automated Theorem Proving (ATP) [BHM+21] in industrial applications kept growing. This branch of computer science is applied in all areas where absolute reliability is required (hardware design, software verification, modern mathematics, ...). To address this issue, computer scientists and logicians started investigating a closely related problem, referred to as automatability [BPR97]. *Automatability* asks if for a fixed proof system one can find an algorithm that returns a proof of a given statement with an at most polynomial blow-up with respect to the size of the shortest proof.

A central discovery of modern proof complexity is that natural propositional proof systems are not automatable under some widely believed complexity assumption, such as standard cryptographic assumptions [KP98; BPR00; BDG+04] or even P ≠ NP [AM20; RGN+21; GKM+20; Gar24; Pap24; IR22; Rez21]. Previously, similar results were known under stronger hypotheses [AR08; MPW19; GL10a]. These results highlight a certain paradox in the world of proof search. One would expect that a more expressive proof system would be capable of achieving remarkably short proofs, making it easier to generate proofs. However, the very same increase in expressive power implies that the search space of those proofs may be so unstructured that no algorithm can find them efficiently. In other words, it seems that if an algorithm fails to discover a short proof that is guaranteed to exist then the fault lies not in the algorithm, but in the proof space itself.

These findings culminate in the recently introduced concept of analyzability [AAR+25]. While automatability evaluates whether an algorithm can efficiently output a proof bounded by the size of the shortest existing proof in a fixed system, *analyzability* inspects the structural complexity of its search space. In particular, we are interested in understanding if proofs of lower bounds can leak information about the formula. For instance, we may be able to learn some information about the satisfiability of a formula by investigating what allows a proof system to achieve short proofs of the formula's lower bounds.

Contributions

A self-contained exposition of the line of work ranging from automatability to analyzability is presented up to Section 4.2. In particular, we give a single uniform presentation of the meta-complexity approach to proof search, adapting and reproving all main results to a single encoding of $\mathrm{Ref}_s(\varphi)$. All previous works [AM20; RGN+21; AAR+25; ARK+26] use different encodings. In the remaining sections of Chapter 4, the following original results are proven:

1. *Polynomial Calculus with Resolution is analyzable.* Given a PCR refutation of the fact that a CNF formula φ on n variables has no Resolution refutation of size $s \geq n^3 \log^2 n$, one can decide in polynomial time whether φ is satisfiable or not. The proof lifts the block-width lower bound for $\mathrm{Ref}_s(\varphi)$ to a block-degree lower bound through algebraic decision-tree reductions. Since analyzability is preserved downwards under p-simulation, Nullstellensatz and Polynomial Calculus are analyzable too. Combined with Pudlák's upper bound, this gives a new proof that automating PCR is NP-hard. This is the first analyzability result for systems above Resolution.

2. *Analyzability does not imply feasible interpolation, unless* NP*-pairs are separable.* For every disjoint NP-pair (U, V) we construct a proof system that simulates Resolution, is closed under restrictions and is analyzable, but has feasible interpolation only if (U, V) itself is polynomial-time separable. The idea is to add to Resolution a rule on a reserved alphabet that encodes the pair, so that the ordinary alphabet, which is all that the analysis of $\mathrm{Ref}_s(\varphi)$ sees, behaves exactly like Resolution.

3. *An obstacle to the converse implication.* Both analyzability and feasible interpolation extract information from proofs and every analyzable system known so far has feasible interpolation. We show that proving that feasible interpolation implies analyzability would imply that Resolution is not weakly automatable unless $\mathsf{P} = \mathsf{NP}$. Since the weak automatability of Resolution is a long-standing open problem, a proof of this implication would require new ideas.

Further contributions include:

1. *Generalization of the reduction from* rPHP *to* $\mathrm{Ref}_s(\varphi)$. We give a version of the reduction from the retraction pigeonhole principle to $\mathrm{Ref}_s(\varphi)$ that holds for every size parameter s. In previous results, these factors were usually hidden inside asymptotic notation.

2. *Obstacles for search-analyzability of PCR.* We discuss why the extraction algorithm for Resolution does not extend directly to PCR, and hence why search-analyzability of PCR remains open.

Thesis outline

In Chapter 1 we recall the background used in the rest of the thesis. In particular, we fix notions from propositional logic, encodings of combinatorial principles such as the pigeonhole principle. We then recall topics regarding computational models, focusing on computability, complexity classes and decision trees.

In Chapter 2 we introduce proof complexity, defining Cook-Reckhow propositional proof systems and presenting the proof systems discussed in later sections. We then define complexity measures and simulations between proof systems as tools to compare them. We give game characterizations for Resolution measures, focusing on the Prover-Delayer and Spoiler-Duplicator games, characterizing tree-like size and width. These games are the main tool for the lower bounds of the following chapters.

In Chapter 3 automatability is extensively discussed. After stating the basic definitions, we present the quasi-polynomial proof search algorithm for tree-like Resolution and the non-automatability of strong systems, which follows from failure of feasible interpolation under cryptographic assumptions. The rest of the chapter proves that automating Resolution is NP-hard through a meta-complexity argument. We introduce the formula $\mathrm{Ref}_s(\varphi)$, which states that φ has a Resolution refutation of size s, and prove Pudlák's polynomial upper bound on the minimum size of Resolution refutations of $\mathrm{Ref}_s(\varphi)$ in the case of satisfiable formulas. When φ is unsatisfiable, we prove an exponential lower bound through a decision-tree reduction from the retraction pigeonhole principle and a random restriction argument, following the original argument of Atserias and Müller revisited by de Rezende, Göös, Nordström, et al. All proofs are given in full for the unary relativized encoding of $\mathrm{Ref}_s(\varphi)$.

In Chapter 4 we discuss analyzability. After introducing the Proof Analysis Problem and its basic facts, such as the analyzability of Resolution, we present an algorithm capable of extracting satisfying assignments of φ from short Resolution refutations of $\mathrm{Ref}_s(\varphi)$. This algorithm shows that Resolution is also search-analyzable. Finally, we extend known results by proving that PCR is analyzable, by lifting the block-width lower bound for $\mathrm{Ref}_s(\varphi)$ to a block-degree lower bound. This result is followed by a brief discussion on why the extraction algorithm of Resolution does not trivially extend to PCR. In the last section, we compare analyzability with feasible interpolation, showing that neither implication between the two properties holds in general or is easy to prove.

Chapter 1

Preliminaries

1.1 Propositional logic

Propositional logic serves as the foundational mathematical framework for reasoning over declarative statements using Boolean truth values and formal logical operators. For our purposes, we will be interested in a fragment of propositional logic that works with finite amount of elements. For a more in depth explanation of propositional logic covering topics necessary to understand this work, the reader is referred to [End01; Kra19; GT26].

Let $\mathrm{Var} = \{x_1, \ldots, x_n\}$ denote a non-empty, finite set of atomic propositional symbols, called *variables*, and let $\mathscr{O} = \{\wedge, \vee, \neg, \rightarrow, \leftrightarrow\}$ be the set of Boolean logical connectives. Each of these connectives represents, respectively, the concept of conjunction, disjunction, negation, implication and logical equivalence. By recursively combining these connectives, starting from variables, we can build formulas that fully express sentences and properties. For instance, the sentence "I'll take my umbrella and my boots if it rains" can be modeled by the formula $R \rightarrow (U \wedge B)$.

Definition 1.1. The set of formulas Fml is defined as the free algebra generated by Var and $\mathscr{O}$. That is, Fml is the smallest set satisfying the following three conditions:

- $\mathrm{Var} \subset \mathrm{Fml}$;
- If $\varphi \in \mathrm{Fml}$ then $\neg\varphi \in \mathrm{Fml}$;
- If $\varphi, \psi \in \mathrm{Fml}$ and $\circ \in \mathscr{O} \setminus \{\neg\}$ then $(\varphi \circ \psi) \in \mathrm{Fml}$.

The set Fml defines propositional logic from a syntactical point of view, i.e. how correct formulas are written. To give meaning to the formulas, we have to also define it from a semantic perspective using *assignments*.

Definition 1.2. An assignment is a mapping $\alpha : \mathrm{Var} \rightarrow \{0, 1\}$ that sets a truth value to each variable.

By extension, each assignment α over variables induces an assignment $\widehat{\alpha} : \mathrm{Fml} \rightarrow \{0, 1\}$ through the semantical meaning of the logical connectives:

φ	ψ	$\neg\varphi$	$\varphi \wedge \psi$	$\varphi \vee \psi$	$\varphi \to \psi$	$\varphi \leftrightarrow \psi$
1	1	0	1	1	1	1
1	0	0	0	1	0	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

Table 1.1. Summary of truth assignments for logical connectives in $\mathscr{O}$.

- $\widehat{\alpha}(\neg\varphi) = 1$ if and only if $\widehat{\alpha}(\varphi) = 0$
- $\widehat{\alpha}(\varphi \wedge \psi) = 1$ if and only if $\widehat{\alpha}(\varphi) = \widehat{\alpha}(\psi) = 1$
- $\widehat{\alpha}(\varphi \vee \psi) = 0$ if and only if $\widehat{\alpha}(\varphi) = \widehat{\alpha}(\psi) = 0$
- $\widehat{\alpha}(\varphi \to \psi) = 1$ if and only if $\widehat{\alpha}(\psi) = 1$ whenever $\widehat{\alpha}(\varphi) = 1$ holds
- $\widehat{\alpha}(\varphi \leftrightarrow \psi) = 1$ if and only if $\widehat{\alpha}(\varphi \to \psi) = \widehat{\alpha}(\psi \to \varphi) = 1$

We will abuse notation throughout this whole work by using α instead of $\widehat{\alpha}$. Assignments can be partial or total. A *partial assignment* is an assignment defined only on a subset of variables, i.e. it is a partial function. Variables x_i for which the assignment is not defined can be viewed as if they were mapped to a special symbol, $\alpha(x_i) = *$. A *total assignment*, instead, assigns a value to all variables of Var.

Next, we list a series of standard definitions that will be extensively utilized in the next chapters. A formula $\varphi \in \mathrm{Fml}$ is said to be *satisfiable* if there is at least one assignment α such that $\alpha(\varphi) = 1$. When all possible assignments satisfy a formula, we refer to that formula as a *tautology* (or valid formula). Clearly, φ is a tautology if and only if $\neg\varphi$ is unsatisfiable.

Two formulas φ and ψ are said to be *logically equivalent*, written $\varphi \equiv \psi$, when for each assignment α it holds that $\alpha(\varphi) = \alpha(\psi)$. Logical equivalences often allow us to rewrite formulas in a way that is more convenient for the current context. For instance, *De Morgan's rule* states that $\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi$ and, similarly, that $\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi$.

Given a Boolean variable x, a *literal* ℓ of x is either the variable x itself or its negation $\neg x$. A *clause* is a disjunction of literals, $C = \ell_1 \vee \ldots \vee \ell_k$, while a *term* is a conjunction of them, $T = \ell_1 \wedge \ldots \wedge \ell_t$. A *conjunctive normal form* (CNF) formula φ is a conjunction of m clauses $C_1, \ldots, C_m$. Similarly, a *disjunctive normal form* (DNF) formula is a disjunction of s terms $T_1, \ldots, T_s$. Any formula can be written as a logically equivalent CNF or a logically equivalent DNF. For instance, the formula $\varphi = ((x_1 \vee \neg x_2) \wedge x_3) \vee x_4$ can be converted into the CNF and DNF formulas written below:

$$(x_1 \vee \neg x_2 \vee x_4) \wedge (x_3 \vee x_4) \tag{1.1}$$

$$(x_1 \wedge x_3) \vee (\neg x_2 \wedge x_3) \vee x_4 \tag{1.2}$$

A *restriction* of a CNF formula $\varphi \in \text{Fml}$ over an assignment α is a formula $\varphi_{\restriction_\alpha}$ obtained from φ through the following procedure. For each literal ℓ with $\alpha(\ell) = b$, where $b \in \{0, 1\}$, we modify φ in the following way:

- If $b = 0$, we remove the literal ℓ from each clause of φ;
- If $b = 1$, we remove each clause of φ that contains ℓ.

For instance, consider again the CNF formula in Equation (1.1). When restricted under the assignment $\alpha = \{x_2 \mapsto 1, x_3 \mapsto 1\}$, the formula becomes $x_1 \vee x_4$.

1.2 Encodings of combinatorial principles

Instead of working with CNF formulas that do not follow a structure, throughout this work we will focus on CNF formulas that encode combinatorial principles [Kra19; BHM+21; GT26]. Intuitively, these formulas make proving computational results easier thanks to their underlying principles [Kra19; GT26]. For instance, the CNF formula PHP_n^m describing the *Pigeonhole Principle* — the combinatorial principle stating that m pigeons cannot be mapped to n holes when $m > n$ — allows us to imagine a partial assignment for this formula as a partial mapping between pigeons and holes.

To encode combinatorial principles we usually rely on mathematical relations. Let X be a finite set. For each pair $i, j \in X$ we define a *Boolean indicator variable* $x_{i,j}$. Through these variables, any total assignment on the variables $\{x_{i,j}\}_{i,j \in X}$ corresponds to a relation $R_\alpha \subseteq X \times X$ with the intended meaning of $\alpha(x_{i,j}) = 1$ if and only if $R(i, j)$ holds. Any formula satisfied by an assignment describes a property satisfied by the relation described by such assignment.

As an example, consider the formula $x_{i,j} \wedge x_{j,k}$. This formula may be interpreted as the statement "j stands between i and k" under some order relation $\preceq$. Another possible interpretation is the statement "there are two edges (i, j) and (j, k)" under some edge relation describing a graph.

As long as X is finite, it is always possible to define a finite set of variables that express the relation R over X in propositional form. More complex properties of relations are described through the *existential* and *universal quantifiers*, $\exists$ and $\forall$. Indeed, such statements can also be encoded as propositional formulas — again, assuming X is finite — by translating them, respectively, into a chain of logical ORs or logical ANDs, $\vee$ and $\wedge$. For instance, the statement "there is an element i in X such that for all elements j in X it holds that $R(i, j)$" can be expressed as the Boolean disjunction ranging over all the elements i of X with a nested Boolean conjunction of $x_{i,j}$, ranging over all elements j of X:

$$\bigvee_{i \in X} \bigwedge_{j \in X} x_{i,j} \tag{1.3}$$

Going back to combinatorial principles, consider again the Pigeonhole Principle. This simple counting principle captures one of the most basic primitives of mathematics. Since the pigeonhole principle entails a counting argument, it has been considered as a prominent example of formula that is hard to prove for reasoning

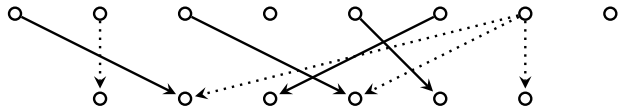


Figure 1. Graphical representation of a partial assignment on PHP_6^8. Each edge is a variable $p_{i,j}$. Solid edges are assigned as true, missing edges are assigned as false and dotted edges are unassigned.

systems that have no rules that allow them to count [PBI93; Ajt94; Bus87; Kra19; GT26].

Formally, the principle says that if f is a total mapping from $[m] = \{1, \ldots, m\}$ to $[n] = \{1, \ldots, n\}$ with $m > n$ then f has a *collision*, that is, there are two pigeons $i, i' \in [m]$ with $i \neq i'$ mapped to the same hole, meaning $f(i) = j$ and $f(i') = j$. In first-order logic, this can be expressed as:

$$(\forall i \in [m]\, \exists j \in [n]\, f(i) = j) \to \exists i, i' \in [m]\, \exists j \in [n]\, i \neq i' \wedge f(i) = j \wedge f(i') = j \quad (1.4)$$

By introducing the Boolean variables $p_{i,j}$ with the intended meaning of $\alpha(p_{i,j}) = 1$ if and only if pigeon i goes to hole j, i.e. $f(i) = j$, the above formula can be translated into a propositional formula as follows:

$$\left(\bigwedge_{i \in [m]} \bigvee_{j \in [n]} p_{i,j} \right) \to \bigvee_{\substack{i,i' \in [m]\colon j \in [n] \\ i \neq i'}} \bigvee p_{i,j} \wedge p_{i',j} \quad (1.5)$$

Since the underlying combinatorial principle is always true, we know that the formula must be a tautology. For many applications, its negated formula is often needed. Using the logical equivalence $\varphi \to \psi \equiv \neg\varphi \vee \psi$, this negated formula can be described by the Boolean conjunction of the following $m + n\binom{m}{2}$ clauses:

$$\begin{array}{lll} p_{i,1} \vee \ldots \vee p_{i,n} & \forall i \in [m] & \text{(Pigeon axioms)} \\ \neg p_{i,j} \vee \neg p_{i',j} & \forall i, i' \in [m] \text{ with } i \neq i' \text{ and } \forall j \in [n] & \text{(Hole axioms)} \end{array}$$

This last propositional formula is referred to as PHP_n^m. Proving that PHP_n^m is unsatisfiable is equivalent to proving that $\neg\text{PHP}_n^m$ is a tautology. Many variants of this unsatisfiable formula are used in many applications, adding or removing some properties, such as functionality, surjectivity and bijectivity [Raz02]. In Section 3.4.4 we will work with one such variant.

1.3 Computability

Depending on the objective, a problem can ask for a binary answer, a specific solution structure or the best possible solution out of many. We divide these problems into three categories: *decision*, *search* and *optimization* [Sip96; Pap03]. A decision problem is a problem for which we are asked to determine if a chosen property holds for a given a instance of the problem. A search problem, instead, asks to find some

structure that certifies the validity of the chosen property. Similarly, an optimization problem asks us to find the optimal structure among all the available ones. It is easy to see that these three ideas form some sort of hierarchy between types of problems. In general, optimization problems are at least as hard as search problems, which in turn is at least as hard as decision problems. For NP-complete problems (see Section 1.4), search and decision are generally polynomially equivalent. This property is known as *self-reducibility* [BG94; Pap03].

We face these types of problems every day in our life. For instance, suppose that we want to reach a destination B from a starting point A. If we are interested in finding the *shortest* route between point A and B, this becomes an optimization problem. If we want to find a route between the two sites, without caring about its length, this becomes a search problem. If, instead, we want to just establish the existence of at least one route, this becomes a decision problem.

Let Σ be a finite set of symbols (e.g. $\Sigma = \{a, b, c, 0, \$, 3, x, \ldots\}$), referred to as *alphabet*, and let Σ^* be the infinite set of all finite strings formed by characters taken from that alphabet. Any subset $L \subseteq \Sigma^*$ is referred to as a *language*. Typically, we are interested in the case where $\Sigma = \{0, 1\}$.

Consider a binary predicate $P : \Sigma^* \to \{0, 1\}$ representing some property, meaning that $P(x) = 1$ means that the property holds for the input instance x and $P(x) = 0$ means that it does not. Let $R_P \subseteq \Sigma^* \times \Sigma^*$ be the binary relation such that $R_P(x, y)$ holds if and only if y certifies the fact that $P(x) = 1$. We refer to $\mathrm{Sol}_P(x) = \{y \in \Sigma^* \mid R_P(x, y)\}$ as the set of *solutions* (or *witnesses*) for the instance x over the property P.

Definition 1.3. The decision problem of P is defined as the language $\Pi^P_{\mathrm{dec}} = \{x \in \Sigma^* \mid \mathrm{Sol}_P(x) \neq \varnothing\}$. The problem Π^P_{dec} is said to be decidable (or computable) if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) = 1$ when $\mathrm{Sol}_P(x) \neq \varnothing$ and $\mathcal{A}(x) = 0$ otherwise.

We observe that the above definition of computability for decision problem can also be stated as the existence of an always-halting algorithm $L(\mathcal{A}) = \Pi^P_{\mathrm{dec}}$. Moreover, we also observe that the above definition could've been stated without the use of the binary relation R_P since $\Pi^P_{\mathrm{dec}} = \{x \in \Sigma^* \mid P(x) = 1\}$ is an equivalent definition. In this setting, the definition is given using R_P only for consistency with search and optimization problems. In fact, for the other two types of problems binary relations are necessary due to the fact that one instance may have more than one possible witness. For instance, there may be more than one (optimal) route from point A to point B.

Definition 1.4. The search problem of P is defined as the binary relation $\Pi^P_{\mathrm{search}} = R_P$. The problem Π^P_{search} is said to be solvable (or computable) if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \mathrm{Sol}_P(x)$ when $\mathrm{Sol}_P(x) \neq \varnothing$ and $\mathcal{A}(x) = \perp$ otherwise.

Here, the symbol $\perp$ is used to represent the fact that no solution exists for the given input instance. The definition allows any algorithm computing the problem to return a different solution on each execution. Optimization problems require an additional component: a *cost function* $c : \Sigma^* \times \Sigma^* \to \mathbb{R}$ expressing the optimality of the solution for the given instance. We denote the set of optimal solutions

as $\mathrm{Opt}_P(x) = \{y \in \mathrm{Sol}_P(x) \mid c(x,y) = \max_{z \in \mathrm{Sol}_P(x)} c(x,z)\}$. Notice that each maximization problem can be rewritten as an equivalent minimization problem.

Definition 1.5. The optimization problem of P over c is defined as the binary relation $\Pi^P_{\mathrm{opt}} = \{(x,y) \in \Sigma^* \times \Sigma^* \mid y \in \mathrm{Opt}_P(x)\}$. The problem Π^P_{opt} is said to be solvable (or computable) if there is an algorithm $\mathcal{A}$ such that $\mathcal{A}(x) \in \mathrm{Opt}_P(x)$ when $\mathrm{Sol}_P(x) \neq \varnothing$ and $\mathcal{A}(x) = \perp$ otherwise.

As previously stated, these three categories form a hierarchy of problems. This is clear from their own definitions: any algorithm computing an optimization problem also computes a search problem and any algorithm computing a search problem can be adapted into one that computes a decision problem.

1.4 Computational complexity

In his seminal work, Turing [Tur37] proved that some problems are *undecidable*, meaning that there is no algorithm that can compute them exactly. Following his work, the pioneers of computer science started noticing that even though many problems can be computed, the algorithms computing them would take too many resources, making them infeasible to process for large inputs. In particular, they focused on two main measures: *time* and *space* [HS65; Sip96; AB09].

Definition 1.6. Let $\mathcal{A}$ be an algorithm and let $t, s \colon \mathbb{N} \to \mathbb{R}^+$ be two functions. We say that $\mathcal{A}$ *runs in time* t if on every input x the algorithm halts after at most $t(|x|)$ steps and that $\mathcal{A}$ *runs in space* s if on every input x it uses at most $s(|x|)$ memory cells.

These two complexity measures are intrinsically related one to another and they are, usually, negatively correlated. Generally, if we allow our algorithm to use more space then the computation can be sped up by saving partial computations, while if we want to lower the amount of space needed then these partial computations have to be calculated each time they are required. Larger inputs require larger amounts of computational resources, making the values $t(n)$ and $s(n)$ proportional to the size n of the input.

Usually, we work with these measures as the input length grows asymptotically. In particular, borrowing notation from standard calculus, we are interested in bounding costs from above and below, up to some constant factor. Let $f, g : \mathbb{N} \to \mathbb{R}^+$ be two functions. We say that:

- $f(x)$ is in $O(g(x))$ when there are two constants $c, n_0 > 0$ such that $\forall n \geq n_0$ we have $f(n) \leq c \cdot g(n)$;
- $f(x)$ is in $\Omega(g(x))$ when there are two constants $c, n_0 > 0$ such that $\forall n \geq n_0$ we have $f(n) \geq c \cdot g(n)$;
- $f(x)$ is in $\Theta(g(x))$ when $f(x)$ is both in $O(g(x))$ and $\Omega(g(x))$;
- $f(x)$ is in $o(g(x))$ when for each constant $c > 0$ there is a constant $n_0 > 0$ such that $\forall n \geq n_0$ we have $f(n) < c \cdot g(n)$.

The main goal of computational complexity is establishing what can and cannot be solved under some complexity measure constraints. In real world situations, any

algorithm with runtime larger than $O(n^3)$ is considered impractical since it requires too much time for large inputs. Nonetheless, complexity theorists use this to their advantage: if a problem cannot be solved even when allowing this constraint to be any form of *polynomial time*, i.e. time $O(n^c)$ for some $c > 0$, the problem is ruled out as *intractable*. Therefore, in this context efficient and polynomial-time are considered synonyms of each other. The class of all efficiently solvable decision problems is denoted with P and the same holds for FP, the class of efficiently solvable search problems [Cob65; AB09].

Definition 1.7. We define P as the class of decision problems solvable in polynomial time by an algorithm, i.e. in time $O(n^c)$ for some $c > 0$. Similarly, FP is the class of search problems solvable in polynomial time.

The aforementioned P vs. NP question (or FP vs. FNP for search problems) asks to settle if the class of efficiently solvable problems corresponds to the class of efficiently verifiable problems. Here, a *verifier* for a problem L is an algorithm that takes an additional input w, referred to as the *witness*, and uses it to check if $x \in L$ holds or not. Usually, this witness is a string that describes a real solution of the problem for the given input. The length of the witness must be bounded by at most a polynomial with respect to the input length.

Definition 1.8. We define NP as the class of decision problems verifiable in polynomial time by a verifier. In particular, for an NP language with verifier V, it holds that $x \in L$ if and only if $\exists w \in \Sigma^*$ for which $V(x, w) = 1$ and $|w| \leq O(|x|^c)$ for some $c > 0$.

By definition, we have that $\mathsf{P} \subseteq \mathsf{NP}$ since if a problem is efficiently solvable then we can just ignore the given witness and solve the problem. Hence, the P vs. NP question asks to settle if this inclusion also holds in the converse direction, i.e. $\mathsf{NP} \subseteq \mathsf{P}$. After more than 50 years of scientific literature studying this problem and its ramifications, researchers conjecture that the answer to this problem is negative [Sip92; AB09; Aar17].

Conjecture 1.9. $\mathsf{P} \neq \mathsf{NP}$

For our purposes, we will assume this conjecture to be true. Another class of our interest is the class coNP, defined as the set of all problems whose complement is verifiable in polynomial time.

Definition 1.10. $\mathsf{coNP} = \{L \subseteq \Sigma^* \mid \overline{L} \in \mathsf{NP}\}$.

it is easy to see that problems that lie in coNP admit a verifier that uses witnesses to discard invalid input instances: if w is a witness for the membership of x in $\overline{L}$ then w is also a witness for the non-membership of x in L. In other words, a problem is in NP if its correct instances can be efficiently verified, while a problem is in coNP if the same holds for incorrect instances.

Again, it is easy to see that $\mathsf{P} \subseteq \mathsf{coNP}$, but the converse is not known and it is actually a question equivalent to establishing that $\mathsf{P} = \mathsf{NP}$. This is due to the

fact that if a problem is efficiently solvable then its complement is also efficiently solvable (we can just negate the output of the same algorithm). This same reasoning implies that if $\mathsf{P} = \mathsf{NP}$ then $\mathsf{NP} = \mathsf{coNP}$. Therefore, proving that $\mathsf{NP} \neq \mathsf{coNP}$ could be a way of establishing that $\mathsf{P} \neq \mathsf{NP}$. It is also conjectured that $\mathsf{NP} \neq \mathsf{coNP}$: there are evident asymmetries between the hardness of witnessing that something holds and that of witnessing something does not hold both in mathematics and computer science [CR79].

Conjecture 1.11. $\mathsf{NP} \neq \mathsf{coNP}$

Going upwards with time complexity, we will also be interested in the quasi-polynomial and sub-exponential analogues of P, referred to as the classes QP and SUBEXP [AB09], respectively. Here, *quasi-polynomial* means that the runtime is a polynomial with at most a poly-logarithmic exponent, while *sub-exponential* means any runtime that is strictly inferior to exponential time.

Definition 1.12. We define QP as the class of decision problems solvable in quasi-polynomial time by an algorithm, i.e. in time $O(n^{\log^c n})$ for some $c > 0$. Similarly, SUBEXP is the class of decision problems solvable in less than exponential time, i.e. in time $O(2^{n^\varepsilon})$ for all $\varepsilon > 0$.

By definition, we have that $\mathsf{P} \subseteq \mathsf{QP} \subseteq \mathsf{SUBEXP}$. No relationship between NP and QP or SUBEXP is known. However, both $\mathsf{NP} \subseteq \mathsf{QP}$ and $\mathsf{NP} \subseteq \mathsf{SUBEXP}$ are conjectured to be false. Since the decision problems underlying most cryptographic assumptions lie in $\mathsf{NP} \cap \mathsf{coNP}$, if $\mathsf{NP} \subseteq \mathsf{QP}$ were to be true then all such assumptions would be breakable in quasi-polynomial time, which is believed to be false. $\mathsf{NP} \subseteq \mathsf{SUBEXP}$ is also conjectured to be false, mainly due to the *Exponential Time Hypothesis* (see Section 3.2).

Conjecture 1.13. $\mathsf{NP} \not\subseteq \mathsf{QP}$ and $\mathsf{NP} \not\subseteq \mathsf{SUBEXP}$.

Finally, we define one last class that introduces randomized computation [Gil74]. Intuitively, a randomized algorithm is an algorithm that is allowed to generate random bits according to a probability distribution, typically set to the uniform distribution over 0 and 1. In this work we will be interested in only one particular type of randomized computation, referred to as *zero-sided error computation*. A randomized algorithm is said to have zero-sided error when for each generable sequence of randomized bits the algorithm never makes mistakes, but it is allowed to answer with "I do not know" with probability at most $1/2$. The class of problems efficiently solvable by these algorithms is referred to as ZPP. We observe that $\mathsf{P} \subseteq \mathsf{ZPP} \subseteq \mathsf{NP} \cap \mathsf{coNP}$.

Definition 1.14. We define ZPP as the class of decision problems solvable in polynomial time by a zero-sided error algorithm, i.e. in time $O(n^c)$ for some $c > 0$.

To interact with these classes we often use the concept of reductions between problems [Kar72; Sip96]. The idea of problem reduction is simple: if there is a reducing function between an initial problem A and a target problem B and B can be solved then A can also be solved by using the reduction as a tunnel. This is due

to the fact that if an algorithm for the original problem can simply compute the reduction and then execute the algorithm that solves the new problem.

Definition 1.15 (Many-to-one reduction)**.** Let A and B be two decision problems. We say that A many-to-one reduces to B, written as $A \leq_m B$ when there is a computable function $f : \Sigma^* \to \Sigma^*$ such that $x \in A$ if and only if $f(x) \in B$.

When $A \leq_m B$ holds, we say that B is "at least as hard as" A since solving B implies solving A. If the reduction is computable in polynomial time, we say that A *p-reduces* to B, written as $A \leq_p B$. In two seminal papers, Cook and Levin [Coo71; Lev73] independently showed that the *satisfiability problem* $\text{SAT} = \{\langle\varphi\rangle \mid \varphi \text{ a satisfiable CNF}\}$, where $\langle\cdot\rangle$ denotes binary encoding, is NP-hard, meaning that every problem that lies in NP is many-to-one reducible to it. Under the assumption that $\mathsf{P} \neq \mathsf{NP}$, this makes this problem unsolvable in polynomial time.

1.5 Decision trees

Decision trees are a simple computational model used in many areas of computer science [Juk12; AB09]. In our case, we will consider decision trees as a way to compute Boolean functions $f : \{0,1\}^n \to \{0,1\}$ or, more generally, for search problems based on such functions. Informally, decision trees are graphical representations of algorithms for computing a function of an unknown input. Each node of the tree is labeled by a variable x_i, $i \in [n]$, and the branches from that node are labeled by the possible values of the variable (0 or 1 in our case). The leaves are labeled by the output of the function. The process starts at the root, knowing nothing, works down the tree, choosing to learn the values of some of the variables based on those already known and eventually reaches a decision.

Definition 1.16. A decision tree is a rooted directed binary tree whose nodes are associated with either an output value or an input Boolean variable. Each internal node is labeled by a variable and the two outgoing edges are labeled by the two possible values. Each leaf is labeled with an output $o \in O$, for a set of outputs O.

A Boolean function f is said to be computed by a decision tree $\mathcal{T}$ when for each input x it holds that $f(x) = \mathcal{T}(x)$. For this computation model we are interested in two main complexity measures: depth and size [Juk12; Bd02]. The *depth* of a decision tree is the number of edges in a longest path from the root to a leaf, or equivalently, the maximum number of bits tested on such a path. The *size* of a decision tree, instead, is the number of nodes in the tree. Generally, for a Boolean function f we denote with $D(f)$ the minimum depth across all decision trees computing f.

An interesting property of decision trees is their possibility to be represented as propositional formulas. In particular, a depth-d decision tree $\mathcal{T}$ computing f gives rise to both a d-DNF and a d-CNF representation for f.

Let $\text{Leaves}(\mathcal{T})$ be the set of all leaves in $\mathcal{T}$ and let $\text{out}(\ell)$ be the output value associated with a leaf $\ell \in \mathcal{T}$. Then, the associated d-DNF $\text{DNF}(\mathcal{T})$ is given by

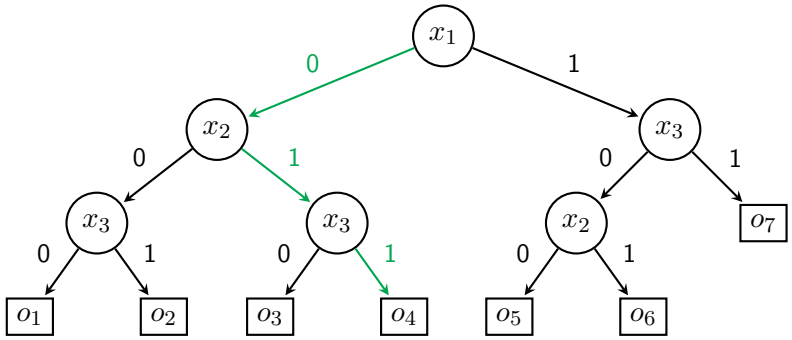


Figure 2. An example of a decision tree of size 13 and depth 3. The green path shows the computation made for the input $x = 011$.

$$\mathrm{DNF}(\mathcal{T}) := \bigvee_{\substack{\ell \in \mathrm{Leaves}(\mathcal{T}): \\ \mathrm{out}(\ell)=1}} D_\ell \tag{1.6}$$

where D_ℓ is the conjunction of all the literals corresponding to the query outcomes on the path from the root of $\mathcal{T}$ to the leaf ℓ. If the query x_i has outcome 1, the corresponding literal is x_i, otherwise it is $\neg x_i$. The associated d-CNF, instead, is obtained by negating the d-DNF formula associated with the negated decision tree $\neg\mathcal{T}$, corresponding to the tree $\mathcal{T}$ with inverted output values, computing $\neg f$. For instance, the decision tree shown in Figure 2 can be represented as the DNF formula

$$(\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge \neg x_3 \wedge x_2) \tag{1.7}$$

or as the CNF formula obtained by negating the DNF of the inverted tree

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_2) \wedge (\neg x_1 \vee \neg x_3) \tag{1.8}$$

Chapter 2

Proof complexity

2.1 Propositional proof systems

Proof complexity is the branch of computational complexity and logic that studies the resources that are needed in order to *certify* mathematical statements, rather than of the resources needed to decide them [Kra19; AB09]. At its core, proof complexity asks if there is a way of writing down proofs of propositional tautologies so that every tautology has short proofs. Such ways of proof writing can be viewed as sets of rules capable of manipulating propositional formulas, deriving new formulas from them. These sets of rules are referred to as *propositional proof systems*. When a formula ψ can be derived from a formula φ through the rules of a proof system Q, we write $\varphi \overset{Q}{\vdash} \psi$ or simply $\varphi \vdash \psi$ when the context is clear. The easiest example of propositional proof system is *Resolution*, based on two simple rules:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \quad \text{(resolution)} \qquad \frac{C}{C \vee B} \quad \text{(weakening)}$$

For a more detailed discussion on examples of proof systems, see Section 2.2. In their seminal paper, Cook and Reckhow [CR79] defined proof systems as surjective functions $Q : \Sigma^* \to \text{TAUT}$, where the latter is the set of all tautologies, $\text{TAUT} = \{\langle\varphi\rangle \mid \varphi \text{ a DNF tautology}\}$. The idea behind this definition is that of an algorithm that is able to determine which tautology is proven by a given proof. A modern and more descriptive definition can be achieved through the concept of verifier.

Definition 2.1. A propositional proof system (p.p.s.) is a verifier Q such that for all formulas φ it holds that $\varphi \in \text{TAUT}$ if and only if there is a proof $\pi \in \Sigma^*$ such that $Q(\varphi, \pi) = 1$ and Q runs in time $O((|\varphi| + |\pi|)^c)$ for some $c > 0$.

This if and only if definition can be split in two implications called completeness and soundness. A proof system Q is *complete* if whenever a formula is a tautology it also has a Q-proof. Vice versa, Q is *sound* if whenever a formula has a Q-proof it also is a tautology. When π is a Q-proof of φ, we often write $\pi : Q \vdash \varphi$.

When for each tautology there is a short proof, i.e. $|\pi| = O(|\varphi|^c)$ for some $c > 0$, we say that the proof system is *polynomially bounded*. Proof systems were originally

introduced by Cook and Reckhow as a possible way of attacking the NP vs. coNP problem. In fact, through the modern definition, finding a polynomially bounded p.p.s. is trivially equivalent to proving that NP = coNP [CR79].

Proposition 2.2. There is a polynomially bounded p.p.s. if and only if NP = coNP.

In their program [CR79], Cook and Reckhow outlined a possible roadmap to proving that NP ≠ coNP through p.p.s. Their idea involved proving that strong proof systems require super-polynomial size proofs for some formulas, to then generalize the various results to any possible proof system, i.e. showing that every proof system has at least one formula that is hard to prove. However, researchers quickly realized that this task was becoming very hard even for weaker systems. In fact, after almost 50 years of research in the field, even weak systems have yet to be completely outlined (e.g. Resolution).

The program has also a second, entirely practical, motivation. Every complete algorithm that establishes the satisfiability of a propositional formula, referred to as a *SAT-solver*, implicitly produces a proof of that formula in some proof system. For instance, the DPLL algorithm (Davis-Putnam-Logemann-Loveland) [DP60; DLL62] and CDCL solvers (Conflict-driven Clause Learning) [BKS04; PD11] produce different types of Resolution proofs, while Gröbner basis solvers produce algebraic proofs [CEI96]. A lower bound on proof size in a system is therefore an unconditional lower bound on the running time of *every* algorithm of the corresponding kind, regardless of how cleverly it is implemented. Proof complexity is, from this point of view, the theory of the limits of automated reasoning [BHM+21].

2.2 Common proof systems

2.2.1 Resolution and Frege systems

The Resolution proof system (Res), introduced by Blake [Bla38] and popularized by Robinson [Rob65] and Davis and Putnam [DP60], is the most extensively studied proof system. It is extensively applied in areas such as automated theorem proving, SAT-solving and artificial intelligence. Like many weak proof systems, Resolution's proofs are actually *refutations*, i.e. "proofs" of unsatisfiability of a formula. This is a feature, not an issue: since φ is a tautology if and only if $\neg\varphi$ is unsatisfiable, any refutation of $\neg\varphi$ is actually a proof of φ. In many works, the word refutation is often used as a synonym of proof. To help the reader, this won't be the case of this work.

Resolution refutations are based on repeated applications of two simple rules, starting from the axioms of the initial formula:

$$\frac{C \vee x \quad D \vee \neg x}{C \vee D} \text{ (resolution)} \qquad \frac{C}{C \vee B} \text{ (weakening)}$$

Given a CNF formula $\varphi = C_1 \wedge \ldots \wedge C_m$, let $\mathrm{Ax}(\varphi) = \{C_1, \ldots, C_m\}$ be the set of axioms of φ. Then, a Resolution refutation of φ is a sequence of clauses $\pi = (D_1, D_2, \ldots, D_t)$ such that for each $i \in [t]$ exactly one of the following holds:

D_1 :	$\neg x_3$	Axiom
D_2 :	$\neg x_1 \vee \neg x_2 \vee x_3$	Axiom
D_3 :	$x_1 \vee \neg x_2 \vee x_3$	Axiom
D_4 :	$\neg x_1 \vee \neg x_2$	$\mathrm{Res}(D_1, D_2)$
D_5 :	$x_1 \vee \neg x_2$	$\mathrm{Res}(D_1, D_3)$
D_6 :	$x_2 \vee x_3$	Axiom
D_7 :	$x_2 \vee \neg x_3$	Axiom
D_8 :	x_2	$\mathrm{Res}(D_6, D_7)$
D_9 :	$\neg x_1$	$\mathrm{Res}(D_4, D_8)$
D_{10} :	x_1	$\mathrm{Res}(D_5, D_8)$
D_{11} :	$\bot$	$\mathrm{Res}(D_9, D_{10})$

Figure 3. Example of a Resolution refutation for the formula $(x_2 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge \neg x_3$

- *Axiom*: $D_i \in \mathrm{Ax}(\varphi)$;
- *Resolution*: $D_i = \mathrm{Res}(D_j, D_k)$ with $j, k \in [i-1]$;
- *Weakening*: $D_i = \mathrm{Weak}(D_j)$ with $j \in [i-1]$.

For the last clause D_t, it must hold that $D_t = \bot$, where $\bot$ represents the *empty clause*, i.e. the clause that does not contain any literal.

The idea behind the Resolution proof system comes from the propagation of satisfiability underlying the resolution rule and the weakening rule.

Proposition 2.3. Let α be an assignment. If $\alpha(C \vee x) = 1$ and $\alpha(D \vee \neg x) = 1$ then $\alpha(C \vee D) = 1$. Similarly, if $\alpha(C) = 1$ then $\alpha(C \vee B) = 1$.

One can prove by induction that if a clause Z is derived through repeated applications of resolution and weakening starting from the axioms then $\alpha(\varphi) = 1$ implies $\alpha(Z) = 1$. When $Z = \bot$, we reach a contradiction since, by definition, the empty clause is unsatisfiable. Therefore, if we are able to derive $\bot$ starting from $\mathrm{Ax}(\varphi)$, we obtain a valid proof of the fact that φ must be unsatisfiable. This proves the soundness of Resolution. Completeness can be proven similarly by induction on the number of variables.

For standard Resolution, it can be proven that the weakening rule is actually not necessary to achieve refutations. In particular, for each Resolution refutation π there is an equivalent refutation π' for the same formula φ such that π' never uses the weakening rule and such that π' uses at most the same number of steps of π. This property will be useful to manipulate refutations by making assumptions on how and when the weakening steps are applied inside them. Notice that this holds only when deriving $\bot$ and not when deriving a generic clause.

For any fixed natural number $k \in \mathbb{N}$, we define an extension of the Resolution proof system called *Resolution over k-DNFs*, denoted $\mathrm{Res}(k)$, first introduced by Krajíček [Kra01]. This proof system works over sets of k-DNF formulas, expressed

as a disjunction of conjunctions of at most k literals. Its rules are inspired by those of Resolution, adapting them to manage k-DNFs:

$$\frac{D_1 \vee \left(\bigvee_{j\in J} \ell_j\right) \quad D_2 \vee \left(\bigwedge_{j\in J} \neg\ell_j\right)}{D_1 \vee D_2} \qquad \text{(cut)}$$

$$\frac{D_1 \vee \left(\bigwedge_{j\in J} \ell_j\right) \quad D_2 \vee \left(\bigwedge_{j'\in J'} \ell_{j'}\right)}{D_1 \vee D_2 \vee \left(\bigwedge_{j\in J\cup J'} \ell_j\right)} \qquad (\wedge\text{-introduction})$$

$$\frac{D_1}{D_1 \vee \left(\bigwedge_{j\in J} \ell_j\right)} \qquad \frac{D_1 \vee \left(\bigwedge_{j\in J\cup J'} \ell_j\right)}{D_1 \vee \left(\bigwedge_{j\in J} \ell_j\right)} \qquad \text{(weakenings)}$$

where D_1 and D_2 are k-DNF formulas.

Frege systems are another classical type of propositional proof system, whose proofs are intended to infer tautologies. These proof systems are well known in the classical logic literature and, furthermore, they work not only with CNF formulas, but also with general Boolean formulas [Kra19; GT26].

Formally, a Frege system is a propositional proof system built from a finite set of axiom schemes and a finite set of rules of inference that can be applied to sets of formulas. An *axiom scheme* is a meta-formula with variables over formulas that can be substituted by any formula. For instance, both $x_1 \wedge x_2 \to x_1$ and $(x_1 \vee x_2) \wedge x_2 \to (x_1 \vee x_2)$ are formulas that can be obtained from the axiom scheme $(P \wedge Q) \to P$, where P and Q are variables over Fml. A *rule scheme* is a typical rule of logical derivation. A prominent example of a scheme rule is the *Modus Ponens* (MP), defined as:

$$\frac{P \qquad P \to Q}{Q} \tag{2.1}$$

Modus Ponens allows one to cut implications from formulas of the form $P \to Q$ once the formula P alone is derived. We can define different Frege systems introducing different axiom schemes. Nonetheless, Cook and Reckhow [CR74] proved that all Frege systems defined over the same connectives are polynomially equivalent to each other, meaning that the axiom scheme and rule scheme chosen is not relevant, up to some polynomial factor (see Section 2.3.2 for an explanation of polynomial equivalence). Therefore, we do not actually care about which rules and axioms are used in our Frege system. This allows us to restrict our interest to a standardized Frege system, known in the literature as *Hilbert-style Frege System*, denoted with F or Frege. The rule scheme of this proof system allows only the use of Modus Ponens over the following standard axiom scheme $\mathrm{Ax}(\mathsf{F})$ of Hilbert-style systems for propositional logic:

(1)	$A \to B \to (A \wedge B)$	
(2)	$(A \wedge B) \to A$	($\wedge$ rules)
(3)	$(A \wedge B) \to B$	
(4)	$A \to (A \vee B)$	
(5)	$B \to (A \vee B)$	($\vee$ rules)
(6)	$(A \to C) \to (B \to C) \to ((A \vee B) \to C)$	
(7)	$(\bot \to A)$	
(8)	$(A \to \bot) \to \neg A$	($\neg$ rules)
(9)	$A \to (\neg A \to \bot)$	
(10)	$(\neg A \to \bot) \to A$	
(11)	$A \to A$	
(12)	$A \to (B \to A)$	($\to$ rules)
(13)	$(A \to (B \to C)) \to ((A \to B) \to (A \to C))$	

Given a formula A, an F-proof of A is a sequence of formulas $\pi = (A_1, \ldots, A_t)$ such that for each $i \in [t]$ exactly one of the following holds:

- *Axiom*: $A_i \in \mathrm{Ax}(\mathsf{F})$;
- *Modus Ponens*: $A_i = \mathrm{MP}(A_j, A_k)$ with $j, k \in [i-1]$.

For the last formula A_t, it must hold that $A = A_t$. The soundness of F comes immediately from the validity of the axiom scheme and the preservation of validity induced by Modus Ponens. Completeness follows similarly.

Extended Frege systems [CR79; CR74], denoted with EF, are Frege systems augmented with an *extension rule* that allows the introduction of new variables that abbreviate more complex formulas.

- *Extension*: $A_i = q \leftrightarrow B$ where q is a new variable that never appears in $A, A_1, \ldots, A_{i-1}, B$ and B is any propositional formula.

Intuitively, this rule allows us to write shorter proofs by encoding partial statements as single variables and use it in the proof, instead of carrying over the initial statement across the whole proof. In some sense, Frege proofs can be viewed as proofs that reason about polynomial size Boolean formulas, while Extended Frege ones reason about polynomial size Boolean circuits [Kra19; Juk12].

2.2.2 Algebraic and geometric systems

Algebraic proof systems are based on algebraic polynomial rings. These systems can be used to witness the fact that a system of polynomial equations over the polynomial ring $\mathbb{F}[x_1, \ldots, x_n]$ is unsolvable; equivalently, that the polynomials do not have a common root.

Consider a CNF formula φ defined over $x_1, \ldots, x_n$. The formula can be encoded as a set of polynomials such that the formula is unsatisfiable if and only if such set of polynomials has no solution. We map each clause of φ to an equation in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \prod_{i \in I} x_i \prod_{j \in J} (1 - x_j) = 0 \tag{2.2}$$

where I and J are, respectively, the set of positive and negative literals of the clause. We observe that truth values are *inversely represented* in this context: the value 0 represents truth, while 1 represents falsity. This comes as a byproduct of the value 0 acting as a way of "deleting monomials" under restrictions, while assigning the value 1 simply removes the literal from the monomial. Some textbooks and research articles fix this issue by mapping $x_i \mapsto 1 - x_i$ and $\neg x_i \mapsto x_i$ in order to preserve the same truth values [Kra19; RGN+21]. In this work, we will be using the flipped truth values.

For each clause C, its corresponding polynomial is denoted with $P(C)$. The formula φ is unsatisfiable if and only if the system $\Phi = \{P(C)\}_{C \in \varphi} \cup \{x_i^2 - x_i = 0\}_{i \in [n]}$ has no solutions. In particular, the latter set of Boolean axioms is used to restrict all possible solutions to those that give Boolean values to $x_1, \ldots, x_n$. After this transformation is made, the question of whether a formula is unsatisfiable is reduced to the question of whether a set of polynomial equations is solvable or not. Algebraic tools can then be used in order to decide unsatisfiability of the original formulas.

Two particular algebraic proof systems are extensively studied in proof complexity: Nullstellensatz and Polynomial Calculus [BIK+94; CEI96]. *Nullstellensatz* (NS) is a static algebraic proof system, i.e. one that does not use rules of inference, based on Hilbert's Nullstellensatz, which states that given m polynomials $p_1, \ldots, p_m \in \mathbb{F}[x_1, \ldots, x_n]$, where $\mathbb{F}$ is an algebraically closed field, the system $p_1(x) = p_2(x) = \ldots = p_m(x) = 0$ is unsolvable if and only if 1 lies in their generated ideal. This is equivalent to the existence of m polynomials $g_1, \ldots, g_m$ such that $\sum_{j=1}^{m} g_j p_j = 1$. For any CNF φ, an NS-proof of φ is the set of polynomials $\pi = \{g_1, \ldots, g_m, h_1, \ldots, h_n\}$ such that:

$$\sum_{j=1}^{m} g_j p_j + \sum_{i=1}^{n} h_i (x_i^2 - x_i) = 1 \tag{2.3}$$

where $p_j = P(C_j)$ for each $j \in [m]$.

Polynomial Calculus (PC) is a proof system extending Nullstellensatz, based on two inference rules:

$$\frac{p \qquad q}{\alpha p + \beta q} \quad \text{(linear combination)} \qquad\qquad \frac{p}{xp} \quad \text{(multiplication)}$$

where $\alpha, \beta \in \mathbb{F}$ and $x \in \{x_1, \ldots, x_n\}$. A PC-proof of the CNF φ is a sequence of polynomials $\pi = (f_1, \ldots, f_t)$ such that each $i \in [t]$ exactly one of the following holds:

- *Axiom*: f_i is the polynomial of some equation in Φ
- *Linear combination*: $f_i = \mathrm{Sum}(f_j, f_k)$ with $j, k \in [i-1]$
- *Multiplication*: $f_i = \mathrm{Mul}(f_j)$ with $j \in [i-1]$

For the last polynomial f_t, it must hold that $f_t = 1$. The soundness of PC can be proven in way similar to Resolution: if we assume that the system of equations Φ has a solution, the equality with 0 can be propagated down to the last polynomial $f_t = 1$, raising a contradiction. Completeness follows from the completeness of Nullstellensatz.

Geometric proof systems, instead, are based on linear programming and optimization routines [Chv73; Gom58; Juk12; Kra19]. All such proof systems are variants of the *Cutting Planes* (CP) proof system, a refutational method derived from 0-1 integer programming. In CP, any CNF formula φ is converted into an equivalent system of linear inequalities (similar to how algebraic systems convert them into equations) whose integer binary solutions correspond directly to the satisfying assignments of φ. Each clause of a formula φ is mapped to an inequality in the following way:

$$\bigvee_{i \in I} x_i \vee \bigvee_{j \in J} \neg x_j \mapsto \sum_{i \in I} x_i + \sum_{j \in J} (1 - x_j) \geq 1 \tag{2.4}$$

For each clause C, its corresponding inequality is denoted with $E(C)$. Interpreting truthness as 1 and falseness as 0, the formula φ is unsatisfiable if and only if the system $\Phi = \{E(C)\}_{C \in \varphi} \cup \{x_i \geq 0; x_i \leq 1\}_{i \in [n]}$ has no integer solution. The set of all feasible solutions for this system yield a polytope P. While any Boolean point in P yields a satisfying assignment, an unsatisfiable formula φ may still leave non-integer (fractional) points within P. CP eliminates these fractional solutions using the *Chvátal-Gomory cut rule* [Chv73; Gom58]. In particular, CP is based on the two following rules:

$$\frac{\sum_{i \in [n]} \alpha_i x_i \geq \gamma \qquad \sum_{i \in [n]} \beta_i x_i \geq \delta}{\sum_{i \in [n]} (\alpha_i + \beta_i) x_i \geq \gamma + \delta} \quad \text{(linear combination)}$$

$$\frac{\sum_{i \in [n]} \eta \alpha_i x_i \geq \gamma}{\sum_{i \in [n]} \alpha_i x_i \geq \left\lceil \frac{\gamma}{\eta} \right\rceil} \quad \text{(cut)}$$

where η is a positive integer coefficient. Formally, a CP-proof of the CNF φ is a sequence of inequalities $\pi = (f_1, \ldots, f_t)$ such that each $i \in [t]$ exactly one of the following holds:

- *Axiom*: $f_i \in \Phi$
- *Linear combination*: $f_i = \mathrm{Sum}(f_j, f_k)$ with $j, k \in [i-1]$
- *Cut*: $f_i = \mathrm{Cut}(f_j)$ with $j \in [i-1]$

For the last inequality f_t, it must hold that $f_t : 0 \geq 1$ to ensure a contradiction.

2.2.3 Tree-like systems

Consider again the refutation in Figure 3. To make things easier to read, we can represent this refutation as a Directed Acyclic Graph (DAG) by connecting clauses with lines. These refutations are called *DAG-like refutations*.

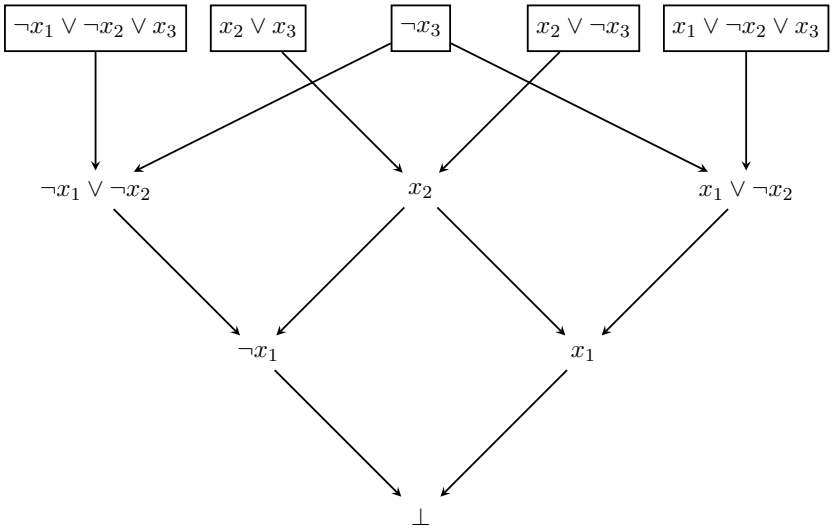


Figure 4. DAG-like representation of the refutation in Figure 3.

We observe that the definition of Resolution refutation allows rules to be applied more than once onto each clause of the proof. There may also be multiple copies of a single clause, each obtained by applying different rules. When each copy of a clause is used only once as premise in the whole proof, we say that the refutation is *tree-like* (named after the fact that the graph becomes a tree). This restriction implies that if we want to use a clause multiple times then it must be derived again, usually starting from the same axioms.

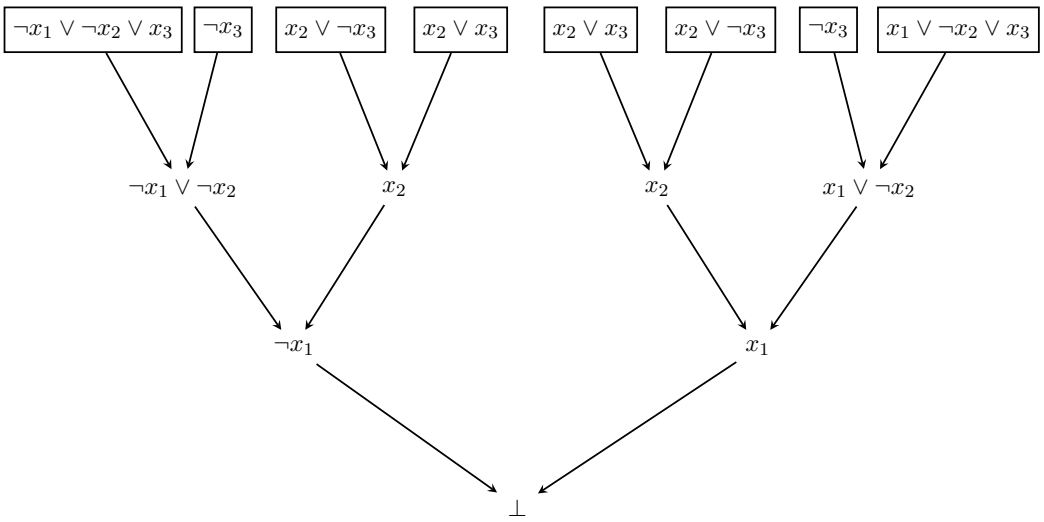


Figure 5. Tree-like adaptation of the refutation in Figure 3.

This sub-system of Resolution is called *tree-like Resolution*, denoted with Res*. This particular system is extensively studied due to its one-to-one correspondence to the DPLL algorithm. The same idea can also be applied to other dynamic systems, obtaining sub-systems like Res$^*(k)$, F^* and PC*.

2.3 Comparing the strengths of different proof systems

2.3.1 Complexity measures for proof systems

Complexity measures lie at the heart of proof complexity. These measures provide a rigorous language to measure if a minimal proof must be short or long, allowing us to evaluate how different systems perform on proving a common statement [Kra19; AB09; GT26]. Different measures highlight different facets of a proof, often interlinking them. By analyzing these measures, we gain a deeper understanding of the inherent difficulty of logical reasoning, even from an algorithmic perspective.

Resolution is the quintessential proof system when it comes to complexity measures. Consider a Res-refutation π. For any clause C, let $|C|$ denote the number of literals in C. Our main complexity measure of interest is the *size* of π, written as $|\pi|$, corresponding to the total number of literals inside it, i.e. $|\pi| = \sum_{D_i \in \pi} |D_i|$. For instance, the refutation in Figure 3 has size 18. In a natural way, the size of the proof corresponds to the number of symbols used to describe the witness of the verifier-based definition of the proof system. For any formula φ with m clauses over n variables, the Resolution size of φ, written as $\text{size}_{\text{Res}}(\varphi)$, is the minimum size across all Res-refutations of φ (equivalently, across all proofs of $\neg\varphi$).

$$\text{size}_{\text{Res}}(\varphi) = \min_{\pi : \text{Res} \vdash \neg\varphi} |\pi| \tag{2.5}$$

Finding the Resolution size of a formula tells us the exact minimum number of symbols in a witnessing string for the given formula. Proving the existence of at least one formula φ, with m clauses over n variables, whose Resolution size is super-polynomial with respect to $|\varphi| = \Theta(n+m)$ is sufficient to conclude Resolution is not polynomially bounded. Haken [Hak85] was the first to show that one such formula is the Pigeonhole Principle formula PHP_n^{n+1}, proving that it requires proofs of size at least $2^{\Omega(n)}$. This result not only implies that Resolution is not polynomially bounded, but it also implies Resolution is weak against a wide number of formulas due to this combinatorial principle being at the foundation of mathematics.

A natural complexity measure related with size is the *length* of a proof, which measures the total number of clauses inside π, i.e. the number of steps in the proof. The Resolution length of a formula φ, written as $\text{length}_{\text{Res}}(\varphi)$, is the minimum length across all Res-refutations of φ. It is straightforward to prove that $\text{length}_{\text{Res}}(\varphi) \leq \text{size}_{\text{Res}}(\varphi) \leq n \cdot \text{length}_{\text{Res}}(\varphi)$ holds for any φ. For this reason, the concepts of size and length are often treated as synonyms in the case of Resolution. In this work, we will extensively use size as a synonym of proof length.

Another complexity measure inspired by Haken's proof is the *width* of a proof π, defined as the maximum number of literals across all clauses of π, i.e. $\max_{D_i \in \pi} |D_i|$. The Resolution width of φ, written as $\text{width}_{\text{Res}}(\varphi)$, is defined as the minimum across all proofs. Ben-Sasson and Wigderson [BW01] proved that Resolution size and width are related to one another through the trade-off.

Theorem 2.4 (Resolution size-width tradeoff)**.** For any unsatisfiable CNF φ it holds that $\text{size}_{\text{Res}}(\varphi) = \exp\left(\Omega\left(\frac{(\text{width}_{\text{Res}}(\varphi) - \text{width}(\varphi))^2}{n}\right)\right)$ where $\text{width}(\varphi)$ is the maximum width of any axiom of φ.

Proof. Omitted. The reader is referred to [BW01]. □

We observe that in the exponent of the right hand side of the equation we have the difference between the Resolution width and the width of the axioms in φ. Because of this, for the case of families of formulas with large initial clauses, like the Pigeonhole Principle formula PHP_n^{n+1}, this method can prove only trivial size lower bounds. Moreover, Bonet and Galesi [BG01] that this tradeoff is optimal, meaning that no better general tradeoff can be achieved.

All Resolution measures carry over to $\text{Res}^*, \text{Res}(k)$ and CP, with a small exception: in $\text{Res}(k)$ and CP, there is no real concept of width, implying that the above trade-off theorem does not have an analogue in both of these worlds.

Moving to other proof systems, size and length measures are inherited by Frege systems in a natural way, but the relation between $\text{size}_{\text{Frege}}(\varphi)$ and $\text{length}_{\text{Frege}}(\varphi)$ is not so straightforward anymore since the formulas inside the proof are not restricted to being clauses. Nonetheless, Cook and Reckhow [CR74] proved that they are still polynomially related to each other.

Other than this, no general trade-offs are known for Frege systems. In addition, there is no concept of width (again, formulas are not restricted to being clauses). Nonetheless, there is another complexity measure of our interest called *depth*, that being the maximum nesting level of logical connectives across all formulas of the proof. For general Frege system, there are no known formulas that require super-polynomial size proofs [Bus87]. However, when restricted to $\Theta(1)$ depth, some formulas are known. This sub-system of Frege is known as AC^0-Frege. For this particular sub-system, exponential lower bounds for PHP_n^m are known [Hås23; PBI93; Ajt94].

Both in Nullstellensatz and Polynomial Calculus, size is defined differently. The size of a polynomial $||p||$ is defined as the number of non-zero monomials obtained by rewriting it as a linear combination of monomials, i.e. after "expanding it". The size of a proof $||\pi||_Q$, with $Q \in \{\text{NS}, \mathsf{PC}\}$, is the sum of the sizes of all the polynomials inside it. The values of $\text{size}_Q(\varphi)$, $\deg_Q(\varphi)$ are defined accordingly.

Regarding polynomials, we have another complexity measure: *degree*. The degree of a polynomial $\deg(p)$ is the maximum degree of all monomials inside it. The degree of a proof $\deg_Q(\pi)$ is the maximum degree across all polynomials inside it. In particular, for an NS-proof $\pi = \{g_1, \ldots, g_m, h_1, \ldots, h_n\}$ over the system $\{p_1, \ldots, p_m\} \cup \{x_i^2 - x_i\}_{i \in [n]}$ the size and degree of π are defined by:

$$||\pi||_{\text{NS}} = \sum_{j=1}^{m} ||g_j||\,||p_j|| + \sum_{i=1}^{n} 2\,||h_i|| \tag{2.6}$$

$$\deg_{\text{NS}}(\pi) = \max_{j \in [m], i \in [n]} (\deg(g_j) + \deg(p_j), 2 + \deg(h_i)) \tag{2.7}$$

For a PC-proof $\pi' = (f_1, \ldots, f_t)$, instead, the size of π' is defined as $||\pi'||_{\text{PC}} = \sum_{i=1}^{t} ||f_i||$ and the degree is $\max_{i \in [t]}(\deg(f_i))$. By definition, it is easy to see that degree acts as an exponential size upper bound for Nullstellensatz proofs: when expanded out, a polynomial of degree d has at most n^d monomials each with at

Size	Total number of literals when expanded
Length	Total number of steps
Width/Degree	Number of literals in a clause/monomial
Depth	Number of nested Boolean operators

Table 2.1. Summary of the main complexity measures.

most n variables. For Polynomial Calculus, instead, an analogue of the Resolution trade-off can be proven [IPS99], which was also proven to be optimal [GL10b].

Theorem 2.5 (PC size-degree tradeoff)**.** For any unsatisfiable CNF φ it holds that $\text{size}_{\text{PC}}(\varphi) = \exp\left(\Omega\left(\frac{(\deg_{\text{PC}}(\varphi)-\deg(\varphi))^2}{n}\right)\right)$, where $\deg(\varphi)$ is the maximum deg of any axiom of φ.

Proof. Omitted. The reader is referred to [IPS99]. □

2.3.2 Simulations between proof systems

To compare the relative power across different proof systems, we define the concept of polynomial simulation (or p-simulation) [CR79]. This notion adapts the concept of polynomial time reduction from decision problems to proof derivations.

Definition 2.6 (p-simulation)**.** Let P and Q be two proof systems. We say that Q p-simulates P, written $P \leq_p Q$, when there is a polynomial time computable function $f : \Sigma^* \to \Sigma^*$ such that for all tautologies φ if π is a P-proof of φ then $f(\pi)$ is a Q-proof of φ.

Intuitively, this means that there is an algorithm that transforms P-proofs of size s for the formula φ into Q-proofs of size $O(s^c)$ for the same formula, with $c > 0$. It is easy to see that for any dynamic proof system Q and its tree-like version Q^* it holds that $Q^* \leq_p Q$ since any Q^*-proof is also a Q-proof. Furthermore, it also typically holds that $Q \not\leq_p Q^*$ since many formulas can be proven more efficiently when not restricted to tree-like proofs. If a proof system Q p-simulates another proof system P and the converse does not hold, we say that Q is *stronger* than P.

A classic benchmark illustrating this gap is the Ordering Principle formula OP_n, which asserts that every finite strict partial order on n elements has a minimal element. While standard Resolution can easily refute OP_n in $O(n^3)$ steps by reusing intermediate sub-refutations [Stå96], tree-like Resolution requires derivations of size at least $2^{\Omega(n)}$ [BEG+98]. This shows that Resolution is strictly stronger than its tree-like variant.

When two systems p-simulate one another, they are said to be *polynomially equivalent*. Equivalence between proof systems allows us in many situations to choose the best one to work with based on the context. For example, the DPLL algorithm is polynomially equivalent to tree-like Resolution, hence why lower bounds on Res^* give lower bounds on the runtime of the algorithm [DLL62].

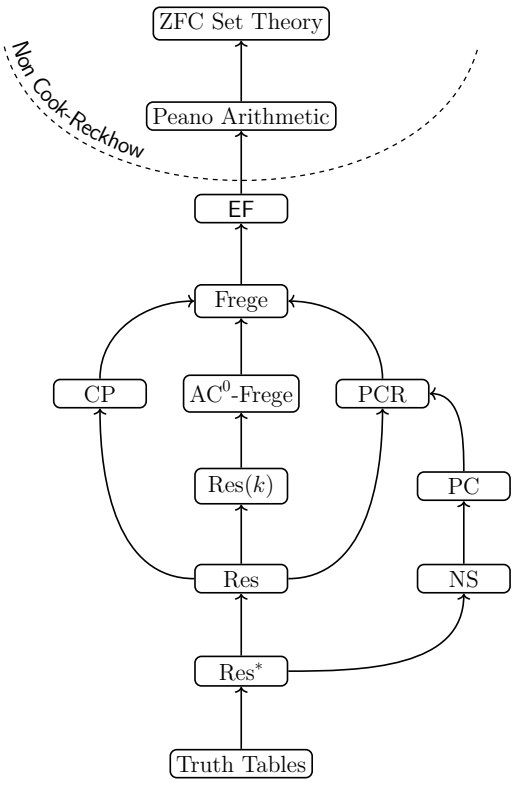


Figure 6. Hierarchy of common proof systems. Omitted arrows imply non p-simulation.

If both p-simulations are impossible, the two systems are said to be *incomparable*. Resolution is known to be incomparable with both Polynomial Calculus and Nullstellensatz, implying that changing the context from clauses to linear algebra neither gives nor takes away power to the proof system. The first non-simulation comes intuitively: Resolution lacks native linear algebra operations, making formulas that rely on it hard for Resolution and easy for Polynomial Calculus. A typical example of such difference is given by *Tseitin formulas* on expander graphs [Urq87; Gri98].

The second non-simulation, instead, comes from the encoding of negative literals in the algebraic context. Since negative literals $\neg x$ get mapped to $(1 - x)$, if a formula contains a clause with n negative literals $\bigvee_{i\in[n]} \neg x_i$, the corresponding polynomial will be $\prod_{i\in[n]}(1 - x_i)$. When expanded out, this polynomial is formed by 2^n monomials, making the size of any of its proofs blow-up to exponential [BGI+01].

To solve this representation issue, it suffices to add *twin variables*, i.e. an additional set of n variables $\overline{x_1}, \ldots, \overline{x_n}$ representing the negated literal. This new proof system is referred to as *Polynomial Calculus with Resolution* [ABR+02]. Here, each Boolean literal x_i is mapped to the algebraic variable x_i, while the literal $\neg x_i$ is mapped to $\overline{x_i}$. The rules of the proof system remain unchanged, with the addition of new axioms $1 - x_i - \overline{x_i}$ for all $i \in [n]$, used to force x_i and $\overline{x_i}$ to occupy opposite values to achieve satisfiability. Since any PC-refutation is also a PCR-refutation, it immediately follows that PC $\leq_p$ PCR. The simulation of Resolution, instead, requires more attention.

Proposition 2.7. Res $\leq_p$ PCR

Proof. Fix a formula φ and let π be a Resolution refutation of φ. Without loss of generality, assume that π never uses the weakening rule. We can construct a PCR-refutation π' for φ by simulating each resolution step in $O(n)$ applications of PCR rules.

Let m and m' be two monomials and suppose that we have derived $m \cdot x_i$ and $m \cdot \overline{x_i}$ in PCR. By applying $\deg(m)$ and $\deg(m')$ multiplication steps, we can derive the monomials $m \cdot m' \cdot x_i$ and $m \cdot m' \cdot \overline{x_i}$. By summing them, we obtain the polynomial $m \cdot m' \cdot (x_i + \overline{x_i})$. Similarly, we can multiply the axiom $1 - x_i - \overline{x_i}$ by m and m' to get $m \cdot m' \cdot (1 - x_i - \overline{x_i})$. Finally, by subtracting the two polynomials we obtain $m \cdot m'$ with a total of $O(n)$ steps.

It is easy to see that the above procedure can be used to simulate Resolution steps. Starting from the axioms, we inductively map each clause C of π to a monomial m_C. For the axioms, the standard conversion suffices. Now, consider any inner clause $C = A \vee B$ derived through a resolution step over $A \vee x$ and $B \vee \neg x$. Since these two clauses have already been mapped, respectively, to the monomials $m_{A \vee x} = m_C \cdot x$ and $m_{B \vee \neg x} = m_C \vee \overline{x}$, the above procedure allows us to derive m_C in $O(n)$ steps. By applying the procedure on the whole proof π, we obtain a PCR-proof π' of size $O(n\,|\pi|)$. □

2.4 Game characterizations of Resolution measures

To simplify working with Resolution measures, we are often interested in characterizing them through *games*. Besides being useful for proving width bounds, these characterizations depend only on the structure of the formula and can be defined independently of the concept of Resolution. This approach allows us to not only simplify proofs, but it also allows us to get insight not only on *how* hard the formula is, but also *why* it is hard.

The first example that we are going to see is the so-called *Prover-Delayer game* [PI00; Pud00]. The game is played by two entities, a Prover and a Delayer, over a fixed CNF formula φ. The *Prover* wants to prove that φ is unsatisfiable and wants to do so as quickly as possible, while the *Delayer* wants to slow down the Prover as much as possible.

Definition 2.8. The Prover-Delayer game is a two-player game played as follows. On each round, the Prover queries a variable x of φ (which may or may not be a variable never queried before) and the Delayer has to answer with one of three options: "it is true", "it is false" or "you choose". These options are represented by the values $1, 0$ and $*$. Whenever the Delayer answers with the third option, the Prover chooses an answer $b \in \{0, 1\}$ and, if they are not yet able to prove that the formula is unsatisfiable, the Delayer scores a *point*. When the union of all values assigned by both players falsifies φ or when the Delayer gives an answer inconsistent with previous ones, the Prover wins the game.

The objective of the Delayer is to define a general strategy that allows them

to score as many points as possible for any possible strategy of the Prover. We denote by $\mathrm{Del}(\varphi)$ the maximum number of points that Delayer can score while playing the game on φ against an optimal Prover strategy. We notice that, since the formulas used in the game are unsatisfiable, the Prover can always win by querying each variable of φ, thus $\mathrm{Del}(\varphi) \leq |\mathrm{Vars}(\varphi)|$ always holds. Any Delayer strategy corresponds to a lower bound on $\mathrm{Del}(\varphi)$, while any Prover strategy corresponds to an upper bound. In their strategies, both parties usually keep track of the game through some data structure, typically a partial assignment.

To show how the game works, we give an optimal Delayer strategy and an optimal Prover strategy for the Pigeonhole Principle formula, PHP^m_n.

Proposition 2.9. $\mathrm{Del}(\mathrm{PHP}^m_n) = n$ when $m > n$.

Proof. We start by giving the lower bound, corresponding to an optimal strategy for the Delayer. On each round, the Delayer keeps track of the game through a partial assignment α. When the Prover queries a variable $p_{i,j}$, the Delayer checks if $\alpha(p_{i,j}) \neq *$. If this holds, the Delayer answers with $\alpha(p_{i,j})$ in order to be consistent with their previous answer. Otherwise, the Delayer searches for a pigeon $i' \in [m] \setminus \{i\}$ such that $\alpha(p_{i,j}) = 1$. If such pigeon exists, the Delayer answers with 0, otherwise they let the Prover decide the value $b \in \{0, 1\}$, to then update $\alpha \leftarrow \alpha \cup \{p_{i,j} \mapsto b\}$.

it is easy to see that this Delayer strategy guarantees that no Hole Axiom $\neg p_{i,j} \vee \neg p_{i',j}$ is violated, thus the Prover can win only by violating a Pigeon Axiom $p_{i,1} \vee \ldots \vee p_{i,n}$. In order for this to happen, there must be a pigeon i such that $\alpha(p_{i,j}) = 0$ for all holes $j \in [n]$. Each of these values must have been assigned either by the Prover or by the Delayer.

Let $P = \{j_1, \ldots, j_t\}$ be the holes for which the Prover answered with 0 and let $D = \{k_1, \ldots, k_s\}$ be those for which the Delayer answered with 0, where $P \cup D = [n]$. All the holes of the first type imply that the Delayer scored t points. For each hole $j \in D$ of the second type, instead, the Delayer answered with 0 only because there is another pigeon $i^{(j)} \neq i$ such that $\alpha(p_{i^j,j}) = 1$, but this is possible only if the Prover chose this value since the Delayer never actively assigns the value 1. Therefore, these holes give the Delayer s additional points, for a total of $t + s = n$ points.

For the upper bound, the following Prover strategy suffices. The Prover keeps track of the game through a partial assignment β and a counter variable z. We denote with $\mathrm{Free}(\beta)$ the set of holes that are still unmapped by β. The Prover's strategy is based on sets of rounds, played as follows. On each set, the Prover subsequently queries the variables $p_{i,j}$ in $[n] \setminus \mathrm{Free}(\beta)$ until the Delayer answers with 1. Whenever the Delayer answers with $b \in \{0, 1\}$, the Prover updates β with $\{p_{i,j} \mapsto b\}$. Whenever the Delayer lets the Prover decide, instead, the latter always chooses the value 1 and updates β with $\{p_{i,j} \mapsto 1\}$. If $\beta(p_{i,j}) = 1$ holds after value is assigned (independently of which answer was given by the Delayer), the Prover increases z by one and then starts a new set of rounds.

In order to not violate any Pigeon Axiom, the Delayer has to eventually either answer with 1 or let the Prover decide, who also answers with 1. After n queried variables are answered with 1 by one of the parties, β describes a perfect matching between n pigeons and n holes. Thus, the next time one of them answers with 1 on

a new variable, a Hole Axiom is guaranteed to be violated. Therefore, the Delayer can let the Prover choose for at most n times, scoring at most n points. □

The Prover-Delayer game can be used to prove lower bounds on the size in tree-like Resolution [PI00; AB09]. In particular, a lower bound on the number of points the Delayer scores in the game played on a formula φ corresponds to a lower bound on the tree-like Resolution size of φ.

Theorem 2.10. Let φ be an unsatisfiable CNF formula. If $\text{size}_{\text{Res}^*}(\varphi) \leq s$ then $\text{Del}(\varphi) \leq \lceil \log s \rceil$.

Proof. Let π be a minimum size tree-like Resolution refutation of φ, where $\text{size}_{\text{Res}^*}(\varphi) \leq s$. Without loss of generality, we assume that the weakening rule is never applied and that each variable of φ is resolved only once on each path in π from the root to an axiom. The idea is for the Prover to use π as a template for the strategy. In particular, the strategy can be seen as a walk in the Resolution tree, going from the empty clause to some axiom. On each round, we keep track of the game state through a partial assignment β.

Starting from the empty clause, at each round the Prover checks a clause C in π. If C was obtained by resolving two clauses $C_0 := C \vee x_i$ and $C_1 := C \vee \neg x_i$, the Prover queries x_i. If the Delayer answers with $b \in \{0, 1\}$, the Prover updates β with $\{x_i \mapsto b\}$ and moves to the clause C_b. When the Delayer lets the Prover answer, instead, they will do so with the value $b' \in \{0, 1\}$ such that $|\pi_{b'}| \leq |\pi_{1-b'}|$, where π_0 and π_1 are the subtrees of π with root C_0 and C_1, respectively, to then move to the clause $C_{b'}$.

Assuming that the Delayer never contradicts themselves, the Prover will eventually reach an axiom D of φ. When this happens, the partial assignment β will falsify D since we walked down the tree following the values opposite to those contained in D. Moreover, we notice that, each time the Delayer lets the Prover answer, at least half of the remaining subtree of π gets discarded. Therefore, the Delayer can let the Prover answer for at most $\lceil \log s \rceil$ times before they win. □

Corollary 2.11. $\text{size}_{\text{Res}^*}(\text{PHP}_n^m) \geq 2^{n-1}$ when $m > n$.

We now move to another game characterization of Resolution measures, this time focusing on width [AD08]. The *Spoiler-Duplicator game* is also played by two entities, the Spoiler and the Duplicator, who take roles similar to the Prover and the Delayer, respectively. This game is also played by answering queries on each round, but the setup differs. Both players construct a common partial assignment α for a fixed CNF formula φ. The goal of the Spoiler is to falsify one of the axioms of φ with the constructed assignment, while keeping a partial assignment small. The Duplicator, instead, tries to keep this from happening by deciding the values of the variables.

Definition 2.12. The Spoiler-Duplicator game is a two-player game played as follows. Starting from the empty assignment $\alpha = \varnothing$, each round of the game is made of a query phase and a deletion phase. In the query phase, the Spoiler chooses a

variable x that is unassigned in α and queries its value to the Duplicator, who has to answer with either 0 or 1. After extending the assignment α with the answer of the Duplicator, the deletion phase is played. Here, the Spoiler chooses a subset of variables $y_1, \ldots, y_k$ that is assigned in α and "forgets them" by removing them from α. The Spoiler wins the game when α falsifies φ.

We observe two main differences between this game and Prover-Delayer game. First, the Duplicator is not allowed to let the Spoiler answer. Second, when a variable is queried again after it has been forgotten by the Spoiler, the Duplicator is not forced to be consistent with the previous value. This last is a rule of the game that allows the Duplicator to keep fooling the Spoiler while they forget values. In particular, we say that the Spoiler wins the *w-game* if the number of variables assigned in α at the end of each round is always at most w and, at some point of the game, some axiom in φ is falsified by α. If there is a strategy for Duplicator that prevents this from happening, they win the w-game.

A practical example of how the game can be played is delayed to Section 3.4.4, where the game will actually be needed to prove some important results. For now, we focus on showing that the characterization holds. To do this conveniently, we introduce the concept of *extensible family* of partial assignments [AD08]. A w-extensible family for φ is a downwards closed set of partial assignments which do not falsify φ and can be extended to any variable as long as they assign at most $w - 1$ variables.

Definition 2.13 (w-extensible family)**.** Let φ be an unsatisfiable CNF formula and let $w \in \mathbb{N}$ be some parameter. A set of partial assignments $\mathcal{H}$ is a w-extensible family for φ if all of the following properties hold:

- $\mathcal{H} \neq \varnothing$
- if $\alpha \in \mathcal{H}$ then α does not falsify the axioms of φ
- if $\alpha \in \mathcal{H}$ and $\beta \subseteq \alpha$ then $\beta \in \mathcal{H}$
- if $\alpha \in \mathcal{H}$ and $|\mathrm{Dom}(\alpha)| < w$ then for all $x \notin \mathrm{Dom}(\alpha)$ it holds that $\alpha \cup \{x \mapsto 0\} \in \mathcal{H}$ or $\alpha \cup \{x \mapsto 1\} \in \mathcal{H}$.

Theorem 2.14. Let φ be an unsatisfiable CNF formula and let $w_F > 0$ be the maximum width of the axioms of φ. Then, for any $w > w_F$ the following are equivalent:

(i) $\mathrm{width}_{\mathrm{Res}}(\varphi) \geq w$;
(ii) There is a w-extensible family for φ;
(iii) Duplicator wins the $(w-1)$-game on φ.

Proof. (i) $\Rightarrow$ (ii) Let $\mathcal{F}_{w-1}$ be the conjunction of all the clauses that can be derived from φ by Resolution using only clauses of width at most $w - 1$. We observe that $\mathcal{F}_{w-1}$ includes all the axioms of φ due to them having width smaller than w by assumption, making $\mathcal{F}_{w-1}$ unsatisfiable, but it does not contain the empty clause $\bot$.

Let $\mathcal{H}$ be the set of all partial assignments over φ that have at most w variables assigned and that do not falsify any clause of $\mathcal{F}_{w-1}$. We show that $\mathcal{H}$ is a w-extensible family for φ. By construction, the first three properties are already satisfied. By way of contradiction, suppose that there is a partial assignment $\alpha \in \mathcal{H}$

with $|\mathrm{Dom}(\alpha)| < w$ such that $\alpha_0 := \alpha \cup \{x \mapsto 0\} \notin \mathcal{H}$ and $\alpha_1 := \alpha \cup \{x \mapsto 1\} \notin \mathcal{H}$ for some variable $x \notin \mathrm{Dom}(\alpha)$. Since the domains of α_0 and α_1 contain at most w variables, the only way for which they both do not lie in $\mathcal{H}$ is that they each falsify some clause $C = C' \vee x$ and $D = D' \vee \neg x$ of $\mathcal{F}_{w-1}$, respectively. Moreover, α must falsify both C' and D', implying that $C' \vee D'$ is also falsified by α. However, this latter clause can be derived by resolving C and D, meaning that $C' \vee D' \in \mathcal{F}_{w-1}$, contradicting the fact that $\alpha \in \mathcal{H}$.

(ii) $\Rightarrow$ (iii) Let $\mathcal{H}$ be a w-extensible family of partial assignments for φ. We give a simple strategy that allows the Duplicator to win the $(w-1)$-game. On each round, the Duplicator uses a partial assignment $\alpha \in \mathcal{H}$ being constructed during the game. By the properties of $\mathcal{H}$, this guarantees that α does not falsify φ. Starting from the empty assignment, when the Spoiler queries a variable $x \notin \mathrm{Dom}(\alpha)$, the Duplicator only needs to extend α to one of the two extensions over x that is guaranteed to be in $\mathcal{H}$.

(iii) $\Rightarrow$ (i) We prove the contrapositive statement. A Spoiler strategy can be easily constructed by walking up the refutation and querying all the variables in the current clause and forgetting all the other variables. Since each clause has at most $w-1$ literals, the Spoiler wins the $(w-1)$-game once a leaf is reached. □

Chapter 3

Automatability

3.1 The quest for finding proofs efficiently

We discussed how under the $\mathsf{NP} \neq \mathsf{coNP}$ hypothesis there cannot be a polynomially bounded propositional proof system, meaning that for each proof system there will always be a formula that requires proofs of large size. Therefore, it is impossible to construct proof search algorithms that are efficient in the formula size. However, we saw in the introduction how topics such as ATP are too important to give up, even approximately.

In the attempt of finding a good-enough solution, researchers tried to relax the problem, asking for a proof search algorithm that returns proofs with a size close-enough to the size of the shortest proof [Iwa97; AR08]. In modern times, this concept is referred to as *automatability*, first defined by [BPR97].

Definition 3.1 (Automatability)**.** Let Q be a proof system. Given a function $t : \mathbb{N} \to \mathbb{N}$, we say that Q is automatable in time t if there is an algorithm $\mathcal{A}$ that for all input formulas φ outputs a proof $\pi : Q \vdash \varphi$ in time $t(|\varphi| + \mathrm{size}_Q(\varphi))$ when φ is a tautology and $\bot$ otherwise.

We are particularly interested in polynomial time automatability: when this holds, the proof returned by the automating algorithm is of size at most $(|\varphi| + \mathrm{size}_Q(\varphi))^c$ for some $c > 0$ due to the upper bound on the running time. For this reason, we often simply say that Q is automatable when t is a polynomial function. We start by stating some simple results about automatability.

Proposition 3.2. Let Q be any proof system. Then:

- If $\mathsf{NP} \subseteq \mathsf{P}$ then Q is automatable in polynomial time;
- If $\mathsf{NP} \subseteq \mathsf{QP}$ then Q is automatable in quasi-polynomial time;
- If $\mathsf{NP} \subseteq \mathsf{SUBEXP}$ then Q is automatable in sub-exponential time.

Proof. We prove the statement for the $\mathsf{NP} \subseteq \mathsf{P}$ case; the others follow in a similar way. Consider the language $\mathsf{PSP}_Q = \{(\varphi, 1^t) \mid \text{ there is a } Q\text{-proof for } \varphi \text{ of size at most } t\}$, corresponding to a formalization of the Proof Size Problem. It is easy to see that

PSP_Q is in NP, therefore also in P due to the assumption. We construct the following algorithm $\mathcal{A}$:

1. On input φ, repeat the following for $i = 1, 2, 3, \ldots$:
   2 Check if $(\varphi, 1^{2^i}) \in \text{PSP}_Q$;
   3 If false, go to the next iteration;
   4 If true, search bit-by-bit for a proof of φ of size at most 2^i and return it.

The procedure goes on for at most $k = O(\log(\text{size}_Q(\varphi)))$ iterations, each costing polynomial time to check membership in PSP_Q. Once a valid size is found, the bit-by-bit proof search requires $O(2^k \cdot (|\varphi| + 2^k)^d)$, concluding that $\mathcal{A}$ automates Q in polynomial time. □

In this chapter we will focus mostly on how all common proof systems yield negative results for automatability, implying that a system with positive results would likely be more complex. In particular, the above proposition implies that non-automatability results are unlikely to be unconditional since they would provide a proof that NP differs from P, QP or SUBEXP, which are all considered to be very hard to prove through modern computational complexity results.

Nonetheless, some very weak positive results are known. For instance, the Resolution size-width tradeoff proven by Ben-Sasson and Wigderson [BW01] can be used to get an algorithm that automates Resolution in time $n^{O(\sqrt{n \log s})}$ via bounded-width search. However, this time bound is still exponential and is only slightly better than the trivial one obtainable through a brute force algorithm. A similar approach can be used to obtain an algorithm that automates Polynomial Calculus in time $n^{O(d)}$, where d is the minimum proof degree [CEI96; GL10a]. A stronger positive result can be achieved in tree-like Resolution. In particular, this proof system has a quasi-polynomial time automating algorithm. We defer the discussion on this result to Section 3.2.

Another concept of our interest will be *weak automatability* [AB04; Pud03], where the proof is allowed to be from any other proof system P, but the size bound is still that of the original system.

Definition 3.3 (Weak automatability)**.** Let Q be a proof system. Given a function $t : \mathbb{N} \to \mathbb{N}$, we say that Q is weakly automatable in time t if there is a proof system P and an algorithm $\mathcal{A}$ that for all input formulas φ outputs a proof $\pi : P \vdash \varphi$ in time $t(|\varphi| + \text{size}_Q(\varphi))$

Proposition 3.4. Let Q be a proof system. If there is another proof system P that p-simulates Q and is automatable in time t then Q is weakly automatable in time t^c for some $c > 0$.

Proof. Let $\mathcal{A}$ be the algorithm that automates P in time t and let $\mathcal{B}$ be the algorithm that allows P to simulate Q in time $O(n^c)$, for some $c > 0$. Then, for each formula φ the output of $\mathcal{A}(\varphi)$ is a P-proof of size $t(|\varphi| + \text{size}_P(\varphi))$. Since P p-simulates Q, we know that for each Q-proof π the output $\mathcal{B}(\pi)$ is a P-proof whose size is $O(|\pi|^c)$. Thus, it must hold that $t(|\varphi| + \text{size}_P(\varphi)) \leq t((|\varphi| + \text{size}_Q(\varphi))^c)$, concluding that $\mathcal{A}$ weakly automates Q in time t^c. □

The above proposition tells us that when a proof system is not weakly automatable there is no proof system above it that is automatable, giving us a powerful tool for proving non-automatability results. When restricted to polynomial time, the converse of the above implication also holds [AB04; Pud03]. That is, a proof system is weakly automatable in polynomial time if and only if there is another proof system that p-simulates it and is automatable in polynomial time.

This implies that proving the (conditional) non weakly automatability of Resolution is the holy grail of automatability theory: since almost any reasonable proof system can p-simulate Resolution due to it being weak enough, this would prove that no such proof system is automatable. We stress that the same does not hold for non automatability: Resolution may not be automatable, but a stronger system may be.

3.2 Quasi-automatability of Tree-like Resolution

As briefly mentioned before, tree-like Resolution has a non-trivial quasi-polynomial time algorithm that automates it [BP96]. The algorithm is based on two simple properties of tree-like refutations in Resolution, formalized by the next two propositions.

Proposition 3.5. Let φ be any CNF formula. If φ has a tree-like Resolution refutation of size at most s then $\varphi_{\restriction \ell=1}$ has a tree-like Resolution refutation of size at most $s/2$ for some literal ℓ.

Proof. Let π be a proof of φ of size at most s. The empty clause $\bot$ is obtained through a resolution step between two clauses equal to x_i and $\neg x_i$, respectively. Let π_{x_i} and $\pi_{\neg x_i}$ be the two subtrees of π having x_i and $\neg x_i$ as roots. Notice that $|\pi| = |\pi_{x_i}| + |\pi_{\neg x_i}| + 1$. Without loss of generality, assume that π_{x_i} is the smallest of the two subtrees and set $\pi' := \pi_{x_i}$. Then, it holds that $|\pi'| < s/2$. By restricting π' over $x_i = 1$, we get a refutation $\pi'_{\restriction x_i=1}$ for $\varphi_{\restriction x_i=1}$ of size less than $s/2$. □

Proposition 3.6. Let φ be any CNF formula. Given a literal ℓ of φ, if $\varphi_{\restriction \ell=0}$ has a tree-like Resolution refutation of size s then ℓ can be derived in tree-like Resolution starting from φ with at most s steps.

Proof. Let π' be a tree-like refutation of $\varphi_{\restriction \ell=0}$ of size s. Without loss of generality, we assume that π' never uses the weakening step. We will construct derivation π of ℓ from φ by mapping each clause of π' to a clause of π.

Each restricted axiom $C_{\restriction \ell=0}$ of $\varphi_{\restriction \ell=0}$ in π' gets substituted with its corresponding axiom, which is either equal to $C_{\restriction \ell=0}$ itself or equal to $C_{\restriction \ell=0} \cup \ell$. Consider the last two clauses $A' := x_i$ and $B' := \neg x_i$ that get resolved into $\bot$ in π'. The structure of π copies exactly that of π' up to these last two clauses, meaning that the same resolutions steps are applied along the two proofs. In particular, let A and B be the two clauses in π obtained by applying the process up to A' and B'. As a byproduct of the procedure, A (resp. B) may either be equal to A' or $A' \vee \ell$ (resp. B' or $B' \vee \ell$). In the first case, we apply a weakening step to get $\widehat{A} := A' \vee \ell$ (resp. $\widehat{B} := B' \vee \ell$).

In the second case, instead, we simply set $\widehat{A} := A$ (resp. $\widehat{B} := B$). Finally, we apply a resolution step between $\widehat{A}$ and $\widehat{B}$ over the variable x_i, which is guaranteed to yield a root node containing only ℓ. □

The idea behind the algorithm is to use these two propositions to apply a smart brute force approach where wrong paths are discarded. Given the input pair formula φ, we start by searching for a literal ℓ for which there is a tree-like Resolution refutation for $\varphi_{\restriction \ell=1}$ of size $s/2$ — which is guaranteed to exist by Proposition 3.5, assuming φ has a refutation of size s. For each literal, this is tested by recursively running the algorithm. Once one such literal is found, we also test if $\varphi \cup \{\neg\ell\}$ has a tree-like refutation of size s. If this second step also succeeds, the algorithm accepts. If one of the two steps fails for all literals, the algorithm rejects.

Algorithm 1 An algorithm $\mathcal{A}$ that finds Res* refutations

Require: A formula φ and a size parameter s
Ensure: A Res* refutation of φ

```
n ← |Vars(φ)|
if n = 1 then
    if φ = x_i ∧ ¬x_i then
        return π := {{x_i}, {¬x_i}, Res(x_i, ¬x_i)}.
    else
        return ∅
for ℓ ∈ {x_1, ..., x_n, ¬x_1, ..., ¬x_n} do
    π'_1 ← A(φ_{↾ℓ=1}, s/2)
    if π'_1 ≠ ∅ then
        π'_2 ← A(φ_{↾ℓ=0}, s)
        if π'_2 ≠ ∅ then
            π_1 ← convert(π'_1, ¬ℓ)      ▷ Converts π'_1 into a derivation of ¬ℓ from φ
            π_2 ← convert(π'_2, ℓ)
            return π := (π_1, π_2, Res(ℓ, ¬ℓ))
return ∅
```

Theorem 3.7. Algorithm 1 automates Res* in quasi-polynomial time.

Proof. We start by proving the correctness of the algorithm. We proceed by induction on n. When $n = 1$, the only way to get a refutation of φ is to resolve x_i with $\neg x_i$, which means that $\varphi = x_i \wedge \neg x_i$ must hold.Suppose now that $n > 1$. The contrapositive of Proposition 3.5 says that if the proof search $\mathcal{A}(\varphi_{\restriction \ell=1})$ fails for all literals then there cannot be a tree-like refutation for φ of size s, hence the algorithm correctly rejects. If at least one such literal ℓ is found, we search the proof $\mathcal{A}(\varphi_{\restriction \ell=0})$. If this last step succeeds, by joining the two derivations π_1 and π_2 of, respectively, $\neg\ell$ and ℓ, to then apply a final resolutions step, we obtain a proof of φ.

We now argue the time complexity of the algorithm. Let $T(s, n)$ be the running time of $\mathcal{A}$ for an input formula φ over n variables that has a tree-like refutation of size at most s. Fix $s^* := \text{size}_{\text{Res}^*}(\varphi)$. Since the refutation size of each formula is bounded by s^* itself, the value of $T(s^*, n)$ gives us the running time of the algorithm.

By construction of $\mathcal{A}$, it holds that $T(s,n) = 2nT(s/2, n-1) + T(s, n-1)$ and $T(s,1) = \Theta(1)$.

It can be proven by induction that $T(2^t, n) \leq (n+1)^{2t}$ holds for all $t \geq 1$. From this we get that $T(s,n) \leq T(2^{\lceil \log s \rceil}, n) \leq (n+1)^{\lceil \log s \rceil}$. Therefore, for each formula φ the runtime is bounded by $n^{O(\log s^*)}$, which is quasi-polynomial with respect to $n + s^*$. □

We observe that the proof constructed by Algorithm 1 generally does not return a proof of minimal size. In fact, the function $T(s^*, n)$ estimates only the running time of the algorithm but says nothing about the size of the proof. Nonetheless, the proof size is obviously still bounded by the running time, meaning that it is guaranteed that the output is a proof of size at most $n^{O(\log s^*)}$.

However, this algorithm is the best we can hope for. Alekhnovich and Razborov [AR08] showed that automating tree-like resolution in polynomial time is $W[P]$-hard. Recently, Rezende [Rez21] extended this further by proving that under the *Randomized Exponential Time Hypothesis* (rETH) [IP99] there is no algorithm capable of automating tree-like Resolution in time $n^{o(\log s)}$.

Conjecture 3.8 (Randomized Exponential Time Hypothesis)**.** 3-SAT cannot be solved by a randomized algorithm in runtime $2^{o(n)}$.

The framework used in this intractability result is similar to the one used in Section 3.4 to prove that Resolution is not automatable under $\mathsf{P} \neq \mathsf{NP}$.

Theorem 3.9. Tree-like Resolution cannot be automated in time $n^{o(\log s)}$, unless the rETH is false.

Proof. Omitted. We refer the reader to [Rez21]. □

3.3 Non-automatability of strong proof systems

The first results regarding non-automatability of proof systems came, surprisingly, for strong proof systems [BPR00; BDG+04; KP98]. These results were proved through a known connection between automatability and another property known as *feasible interpolation* [Cra57; Kra97]. Let $\varphi(x,y)$ and $\psi(x,z)$ be two CNF formulas, where x are shared variables and y, z are private ones, such that setting all variables in x makes at least one of the two formulas unsatisfiable. A proof system with feasible interpolation is capable of discerning which of these two formulas is falsified by an assignment x that defines only the shared variables.

Definition 3.10 (Feasible interpolation)**.** Let Q be a refutational proof system. We say that Q has feasible interpolation in time t if there is an algorithm $\mathcal{I}$ that given an unsatisfiable formula $\varphi(x,y) \wedge \psi(x,z)$ and a Q-refutation π for the formula, returns in time t, with respect to $|\pi|$, a Boolean circuit $\mathcal{C}_{\mathcal{I}}(\alpha)$ such that:

- if $\mathcal{C}_{\mathcal{I}}(\alpha) = 0$ then $\varphi(\alpha, y)$ is unsatisfiable

- if $\mathcal{C}_{\mathcal{I}}(\alpha) = 1$ then $\psi(\alpha, z)$ is unsatisfiable

We observe that feasible interpolation is down-ward closed: if $Q \leq_p P$ and P has feasible interpolation then we can just convert the input Q-refutation into a P-refutation and apply the same algorithm. For systems *closed under restrictions*, i.e. systems for which if π is a Q-proof of φ then $\pi_{\restriction_\alpha}$ is a Q-refutation of π, feasible interpolation is implied by weak automatability (hence also by automatability).

Theorem 3.11. Let Q be a proof system closed under restrictions. If Q is weakly automatable in time t then it has feasible interpolation in time t.

Proof. We prove the theorem for the polynomial time case; a similar argument follows for a generic runtime. By the above observation, it suffices to prove this for a system that is automatable. Let $\mathcal{A}$ be an algorithm that automates Q in time $p = \text{poly(n)}$. We define the following algorithm $\mathcal{B}$. Given a Q-refutation π of size at most $O(N^c)$, where $N = |\varphi(x, y) \wedge \psi(x, z)|$, $\mathcal{B}$ constructs an algorithm $\mathcal{C}$ that, on input α, runs $\mathcal{A}$ on $\varphi(\alpha, y)$ for at most $p(N^c)$ steps. If the computation returns a refutation, $\mathcal{C}$ outputs 0, otherwise it outputs 1.

By well known computational complexity arguments (see [AB09] for a detailed proof), the algorithm $\mathcal{C}$ can be turned into a Boolean circuit of size $O(t \log t)$ [AB09; PF79], which is polynomial in this case. When $\mathcal{C}(\alpha)$ returns 0, the formula $\varphi(\alpha, y)$ is unsatisfiable by construction of $\mathcal{C}$. If the computation of $\mathcal{A}$ exceeds the time bound, instead, we know that $\psi(\alpha, z)$ is unsatisfiable. To show this, suppose by contradiction that $\psi(\alpha, \beta)$ is satisfiable for some assignment β on the z variables. Then, $\pi_{\restriction_{\alpha\cup\beta}}$ is a Q-refutation of $(\varphi(\alpha, y) \wedge \psi(\alpha, z))_{\restriction_\beta} \equiv \varphi(\alpha, y)$. Therefore, since π has polynomial size, the execution of $\mathcal{A}$ on $\varphi(\alpha, y)$ should've found a refutation, raising a contradiction. □

This connection was first found by Bonet, Pitassi, and Raz [BPR00]. Years prior, Krajíček and Pudlák [KP98] already proved that Extended Frege does not admit polynomial time feasible interpolation unless the RSA encryption scheme [RSA77] can be broken. This was extended down to TC^0-Frege systems in [BPR97], a sub-system between Frege and AC^0-Frege, and later on down to AC^0-Frege in [BDG+04]. All these results share the same idea: the *Diffie-Hellman* key exchange protocol [DH76] can be encoded as a depth-d propositional formula, with $d > 0$. Through feasible interpolation, these systems are capable of breaking the DH scheme through circuits of sub-exponential size, which is conjectured to be false. This makes Frege and all proof systems above it impossible to automate (conditionally).

Theorem 3.12. No proof system p-simulating AC^0-Frege is weakly automatable in polynomial time, unless the DH procedure can be broken by circuits of size 2^{n^ε}, for some $\varepsilon > 0$.

Proof. Omitted. We refer the reader to [BDG+04]. □

Recent results expand these results to *quantum automatability*, i.e. automatability through quantum algorithms. In particular, Arteche, Carenini, and Gray [ACG24]

proved that quantum automating TC^0-Frege is as hard as solving the Learning-With-Errors (LWE) problem through quantum algorithms. The hardness of the LWE problem is a building block of modern post-quantum cryptography [Reg09].

For weaker proof systems, the argument based on the impossibility of feasible interpolation fails. In fact, most weak proof systems actually possess feasible interpolation (Resolution [Kra97], Nullstellensatz and Polynomial Calculus [Hak20], Cutting Planes [Pud97]), making this approach impossible. Nonetheless, we will see in Section 3.4 a more complex argument that proves the non-automatability of weak proof systems under the $\mathsf{P} \neq \mathsf{NP}$ assumption.

Another connection between the two properties is given by the formula

$$\mathrm{Reflex}_s^Q(\varphi) := (\mathrm{Sat}(\varphi, \alpha) \to \neg \mathrm{Ref}_s^Q(\varphi, \pi)) \tag{3.1}$$

where $\mathrm{Sat}(\varphi, \alpha)$ encodes the statement "α satisfies φ" and $\mathrm{Ref}_s^Q(\varphi, \pi)$ encodes the statement "π is a Q-refutation of φ of size at most s". This formula is referred to as the *Reflection Principle* [Pud03; AB04] of a refutational system Q and it is always a tautology since if φ is satisfiable then it cannot have a refutation. Therefore, the formula $\neg\mathrm{Reflex}_s^Q(\varphi)$ is always unsatisfiable and, assuming it has short proofs, feasible interpolation may be applied to it in order to find short proofs of φ. This gives us a partial inverse of the previous theorem.

Theorem 3.13. Let Q be a proof system. If there is a proof system P that has feasible interpolation in polynomial time and has polynomial size proofs of Reflex_s^Q then Q is weakly automatable in polynomial time.

Proof. Again, we prove the theorem for the polynomial time case. We define a new refutational proof system R as follows. An R-refutation for a formula φ is a triple $(1^s, \pi_P, \mathcal{C})$ such that π_P is a size t P-refutation of the formula $\mathrm{Reflex}_s^Q(\varphi)$ and $\mathcal{C}_\mathcal{I} = \mathcal{I}(\mathrm{Reflex}_s^Q(\varphi), \pi_P)$ is an interpolating circuit for $\mathrm{Reflex}_s^Q(\varphi)$ such that $\mathcal{C}_\mathcal{I}(\varphi) = 0$, where $\mathcal{I}$ is the algorithm certifying feasible interpolation for P. The automatability of R follows from its very own definition: the whole triple can be computed in polynomial time.

Even though it seems artificial, R is a Cook-Reckhow system and, moreover, it p-simulates Q. Consider the following verifier for R. Given the input $(\varphi, (1^s, \pi, \mathrm{C}))$, we start by checking if π is a valid P-refutation of polynomial size for $\mathrm{Reflex}_s^Q(\varphi)$. Then, we compute $\mathcal{C}_\mathcal{I} = \mathcal{I}(\mathrm{Reflex}_s^Q(\varphi), \pi)$ and check if $\mathcal{C} = \mathcal{C}_\mathcal{I}$. If all checks have survived, the verifier accepts, otherwise it rejects.

it is easy to see that if the verifier accepts for some witness $(1^s, \pi, \mathcal{C})$ then $\mathcal{C}(\varphi) = 0$ implies that $\mathrm{Sat}(\varphi, \alpha)$ is unsatisfiable for each α, which is possible if and only if φ is unsatisfiable. Vice versa, if the verifier rejects all witnesses, even for well-defined ones, we know that every interpolating circuit returns 1, meaning that $\mathrm{Ref}_s^Q(\varphi)$ is unsatisfiable for each s. However, this is possible if and only if φ does not admit Q-refutations. This concludes that R is both sound and complete.

A similar argument can be used to prove the p-simulation of Q by R. Given a Q-refutation π_Q of φ, we know that $\mathrm{Sat}(\varphi, \alpha)$ is unsatisfiable for each α, we know that $\mathcal{C}_\mathcal{I}(\varphi) = 0$, therefore the triple $(1^{|\pi_Q|}, \pi_P, \mathcal{C}_\mathcal{I})$ is a valid R-refutation.

Moreover, since π_P and $\mathcal{C}_\mathcal{I}$ can both be computed in polynomial time, we conclude that $Q \leq_p R$. □

We notice that the above theorem matches the known results on proof systems. Currently, there are no proof systems known to have both properties and it is conjectured that one such proof system may not exist [Pud03].

The most prominent evidence for this conjecture is the Res(k) proof system. It is known [AB04] that this proof system has short refutations of Resolution's Reflection Principle $\text{Reflex}_s^{\text{Res}}(\varphi)$, meaning that Resolution would be weakly automatable if Res(k) had feasible interpolation in polynomial time. However, in the same paper it is also proven that both of these properties are actually equivalent to proving that Res(k) itself is weakly automatable [AB04].

Theorem 3.14. For all constants $k \geq 2$, the following three statements are equivalent:

- Res is weakly automatable in polynomial time
- Res(k) is weakly automatable in polynomial time
- Res(k) has feasible interpolation in polynomial time

3.4 Non-automatability of weak proof systems

3.4.1 The Ref formula

To prove that weak proof systems are not automatable under the $\mathsf{P} \neq \mathsf{NP}$ hypothesis, we use a meta-complexity approach. As the name implies, *meta-complexity* is the area of computational complexity theory that focuses on studying how complex it is to prove that something is complex [Ins23]. To achieve our goal, we will focus on how hard it is for a proof system to prove that something is hard to prove in Resolution. This approach is a refinement of previous approaches [AR08; GL10a; MPW19] based on different meta-complexity problems.

Given a CNF formula φ and a parameter $s \in \mathbb{N}$, we define a formula $\text{Ref}_s(\varphi)$ representing the statement "φ has a Resolution refutation with s clauses" [Pud03; AM20; RGN+21; AAR+25]. Our final goal will be to prove that these refutation statements can act as a discriminant between satisfiability and unsatisfiability. Our goal will be proving the following theorem.

Theorem 3.15. Let φ be a formula with n variables and m clauses. Then:

(i) If $\varphi \in \text{SAT}$ then $\text{Ref}_s(\varphi)$ has polynomial size Resolution refutations;
(ii) If $\varphi \notin \text{SAT}$ then $\text{Ref}_s(\varphi)$ requires exponential size Resolution refutations.

This shows that, basically, Resolution is not capable of proving its own lower bounds (recall that a refutation of $\text{Ref}_s(\varphi)$ is a proof of $\neg\text{Ref}_s(\varphi)$, the formula stating that "φ requires refutations of size bigger than s"). We split the proof of this theorem in three parts, proven respectively in the next three sub-sections (see Lemma 3.19, Lemma 3.30, Lemma 3.28). This result can be generalized to

Nullstellensatz, Polynomial Calculus and Cutting Planes. More information on this generalization is deferred to Section 3.4.6.

Together with the fact that $\mathrm{Ref}_s(\varphi)$ can be constructed in polynomial time, the above result can be used as a dichotomy tool for discerning satisfiable formulas from unsatisfiable ones. Therefore, if Resolution were to be automatable, we would be able to solve SAT in polynomial time, which implies $\mathsf{P} = \mathsf{NP}$. Moreover, we observe that this result is *optimal* by Proposition 3.2 (recall that $\mathrm{P} \subseteq \mathsf{QP} \subseteq \mathsf{SUBEXP}$).

Lemma 3.16. Automating Resolution is NP-hard.

Proof. Let $\mathcal{A}$ be an algorithm that automates Resolution in time t. By Theorem 3.15(i) we know that for every $s \geq n^2$ and all satisfiable φ on n variables it holds that $\mathrm{size}_{\mathrm{Res}}(\mathrm{Ref}_s(\varphi)) \leq O(N^c)$ for some $c > 0$, where N is the number of variables in $\mathrm{Ref}_s(\varphi)$.

We define a new algorithm $\mathcal{B}$ that, given a CNF formula φ in input, constructs $\mathrm{Ref}_s(\varphi)$ to then run $\mathcal{A}(\mathrm{Ref}_s(\varphi))$. If the number of steps in the execution of $\mathcal{A}$ exceeds $t(N + N^c)$, we stop the computation and immediately reject φ. Otherwise, if the computation does not exceed the bound, $\mathcal{A}$ returns either a refutation π for $\mathrm{Ref}_s(\varphi)$ or $\bot$. In the first case, $\mathcal{B}$ accepts, otherwise it rejects.

When φ is satisfiable, Theorem 3.15(i) ensures that $\mathrm{Ref}_s(\varphi)$ has a refutation of size at most N^c, thus the bound $t(N + N^c)$ is not violated and $\mathcal{B}$ accepts. When φ is unsatisfiable, instead, item (ii) of the same theorem guarantees that the bound is always violated, making $\mathcal{B}$ immediately reject. Since $\mathrm{Ref}_s(\varphi)$ is constructible in polynomial time with respect to n, $\mathcal{A}$ solves SAT in time $O(t(n^{cd}))$, for some $d > 0$. □

Corollary 3.17. Let $\mathcal{A}$ be an algorithm automating Resolution in time t. Then:

- If t is any polynomial function then $\mathsf{NP} \subseteq \mathsf{P}$;
- If t is any quasi-polynomial function then $\mathsf{NP} \subseteq \mathsf{QP}$;
- If t is any sub-exponential function then $\mathsf{NP} \subseteq \mathsf{SUBEXP}$.

The simplicity of Resolution allows us to describe this meta-complexity refutation statement through a natural encoding. The idea is to formalize $\mathrm{Ref}_s(\varphi)$ as a blueprint for a generic Resolution refutation for φ with at most s clauses. Each clause is represented by a *block*, a set of variables that describe the literals that appear in the clause, how it was obtained (resolved or weakened from another clause) and two (or one) pointer variables that keep track of the parent blocks it was derived from. The axioms of the formula are formal descriptions of coherence rules between these variables. For instance, if the variable saying that a block B was derived through a resolution step is set to true then all the other variables that talk about the derivation itself (parent pointers, resolving literal, ...) must also be set.

The initial standard encoding of this idea was the one used in Pudlák [Pud03], referred to in the literature as the *unary encoding*. This encoding uses unary pointers, meaning that for every block $B \in [s]$ there are up to $2(s-1)$ variables that point at the blocks from which B may be derived. Pudlák proved the easiness of $\mathrm{Ref}_s(\varphi)$ when φ is satisfiable through this encoding, but the proof can be adapted for other encodings.

In order to prove their result, Atserias and Müller [AM20] adapted this encoding by introducing *enable variables*. When the enable variable is true, the block is considered as part of the underlying refutation described by the blueprint (as if we are forcing the refutation to use this block). When this is not the case, instead, the block gets completely ignored by the refutation, automatically satisfying all the axioms that mention it. This encoding is known as the *relativized unary encoding*. This encoding was first introduced to prove the hardness of $\mathrm{Ref}_s(\varphi)$ when φ is unsatisfiable due to failure of achieving such result with the non-relativized encoding. Nonetheless, Garlík [Gar19] proved that the hardness of this formula does not come from the introduction of these variables by proving that the size lower bound also holds for the non-relativized encoding.

Finally, the last common encoding is the *relativized binary encoding*, first defined by Rezende, Göös, Nordström, et al. [RGN+21]. This encoding describes the parent pointers of each block in binary, meaning that for each block $B \in [s]$ there are only $O(\log s)$ variables describing its pointers. This encoding drastically reduces the width of the formula, making it possible to prove the size lower bound simply by using a size-width tradeoff.

In this work we will use the relativized unary encoding as it is both the most natural one for the problem and the easiest to work with (see [AAR+25] for a more in depth comparison behind the encodings). Given n variables $x_1, \ldots, x_n$, let $\mathrm{Lit}_n = \{x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$. We define the variables of the formula $\mathrm{Ref}_s(\Phi)$, parametrized over a generic formula Φ with n variables and m clauses, as follows:

a-lit_ℓ^A	:	literal $\ell \in \mathrm{Lit}_n$ is present in the clause $A \in [m]$ of Φ;
enable^B	:	block $B \in [s]$ is enabled;
derived^B	:	block $B \in [s]$ was obtained by a Resolution step;
weak_A^B	:	block $B \in [s]$ is obtained by weakening from clause $A \in [m]$ of Φ;
lit_ℓ^B	:	literal $\ell \in \mathrm{Lit}_n$ is present in the block $B \in [s]$;
$\text{res}_{x_i}^B$	:	block $B \in [s]$ was obtained by resolving over the variable x_i;
$\text{lpoint}_{B'}^B$	:	block $B \in [s]$ was resolved on the left from block $B' \in [s], B' < B$;
$\text{rpoint}_{B'}^B$	:	block $B \in [s]$ was resolved on the right from block $B' \in [s], B' < B$.

When working with a specific formula φ, we restrict $\mathrm{Ref}_s(\Phi)$ to the formula $\mathrm{Ref}_s(\varphi) := \mathrm{Ref}_s(\Phi)_{\restriction \alpha_\varphi}$ through a partial assignment α_φ that sets the values of all the variables a-lit_ℓ^A in a way that describes the clauses of φ, i.e. $\alpha_\varphi(\text{a-lit}_\ell^A)$ is set to 1 if ℓ is part of the A-th clause of φ, otherwise we set it to 0. Moreover, we also observe that the weakening variables weak_A^B are defined only for axiom clauses. Of course, this can be extended to non-axiom clauses but, for the purpose of making proofs easier and shorter, we will assume without loss of generality that the weakening step can be used only on axioms. The clauses of $\mathrm{Ref}_s(\varphi)$ are then defined as follows:

$$\left(\text{enable}^B \wedge \text{res}^B_{x_i} \wedge \text{lpoint}^B_{B'} \wedge \text{lit}^{B'}_\ell\right) \to \text{lit}^B_\ell \quad \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{x_i\}, \quad \text{(REF-1)}$$

$$\left(\text{enable}^B \wedge \text{res}^B_{x_i} \wedge \text{rpoint}^B_{B'} \wedge \text{lit}^{B'}_\ell\right) \to \text{lit}^B_\ell \quad \text{for } B, B' \in [s], B' < B, i \in [n], \ell \in \text{Lit}_n \setminus \{\neg x_i\}, \quad \text{(REF-2)}$$

$$\left(\text{enable}^B \wedge \text{weak}^B_A \wedge \text{a-lit}^A_\ell\right) \to \text{lit}^B_\ell \quad \text{for } B \in [s], A \in [m], \ell \in \text{Lit}_n, \quad \text{(REF-3)}$$

$$\left(\text{enable}^B \wedge \text{derived}^B\right) \to \bigvee_{i \in [n]} \text{res}^B_{x_i} \quad \text{for } B \in [s], \quad \text{(REF-4)}$$

$$\left(\text{enable}^B \wedge \text{derived}^B\right) \to \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{lpoint}^B_{B'} \quad \text{for } B \in [s], \quad \text{(REF-5)}$$

$$\left(\text{enable}^B \wedge \text{derived}^B\right) \to \bigvee_{\substack{B' \in [s] \\ B' < B}} \text{rpoint}^B_{B'} \quad \text{for } B \in [s], \quad \text{(REF-6)}$$

$$\left(\text{enable}^B \wedge \neg\text{derived}^B\right) \to \bigvee_{A \in [m]} \text{weak}^B_A \quad \text{for } B \in [s], \quad \text{(REF-7)}$$

$$\left(\text{enable}^B \wedge \text{lpoint}^B_{B'}\right) \to \text{enable}^{B'} \quad \text{for } B, B' \in [s], B' < B \quad \text{(REF-8)}$$

$$\left(\text{enable}^B \wedge \text{rpoint}^B_{B'}\right) \to \text{enable}^{B'} \quad \text{for } B, B' \in [s], B' < B \quad \text{(REF-9)}$$

$$\neg\text{lit}^s_\ell \quad \text{for } \ell \in \text{Lit}_n, \quad \text{(REF-10)}$$

$$\text{enable}^s. \quad \text{(REF-11)}$$

The axiom clauses of $\text{Ref}_s(\Phi)$ have been written as implications for the sake of readability. This encoding of $\text{Ref}_s(\Phi)$ is defined over $N = \Theta(s^2 + sm + sn + nm)$ variables and $M = \Theta(s^2n^2 + smn)$ clauses, meaning that $|\text{Ref}_s(\Phi)| = \text{poly}(s, n, m)$. The maximum width of the clauses is $O(s + m + n)$.

Proposition 3.18. φ has a Resolution refutation of length at most s if and only if $\text{Ref}_s(\varphi)$ is satisfiable.

Proof. Let π be a refutation of φ of length $s' \leq s$. We define a satisfying assignment for $\text{Ref}_s(\varphi)$ by enabling exactly s' blocks to then set all their variables by following the description of π.

Vice versa, let α be a satisfying assignment for $\text{Ref}_s(\varphi)$. Since the clauses of $\text{Ref}_s(\varphi)$ act as a blueprint for a resolution refutation, we can read the contents of the enabled blocks in α and traverse the encoded refutation starting from the root block s down to the axioms blocks. The steps taken by the encoded refutation correspond to the steps taken by a real refutation π' of φ. Since $\text{Ref}_s(\varphi)$ has at most s enabled blocks, we get a refutation of length at most s for φ. □

The above proposition not only formalizes the soundness of the formula $\text{Ref}_s(\varphi)$, but it also shows that, when it is a tautology, the minimum size of a Q-proof π of $\neg\text{Ref}_s(\varphi)$ determines the ability of a proof system to reason efficiently or not about Resolution lower bounds. In particular, when $\text{Ref}_s(\varphi)$ is unsatisfiable one of two cases may hold: either φ is unsatisfiable but it requires refutations of length at least $s + 1$ or φ is satisfiable, thus it has no refutations at all. This second case may also be seen as if φ requires "refutations of infinite length".

3.4.2 An upper bound on Ref for satisfiable formulas

We prove that when φ is satisfiable the formula $\text{Ref}_s(\varphi)$ admits a short Resolution refutation, corresponding to Theorem 3.15(i). This result was first proven by Pudlák [Pud03] for the unary encoding. Atserias and Müller [AM20] showed that such

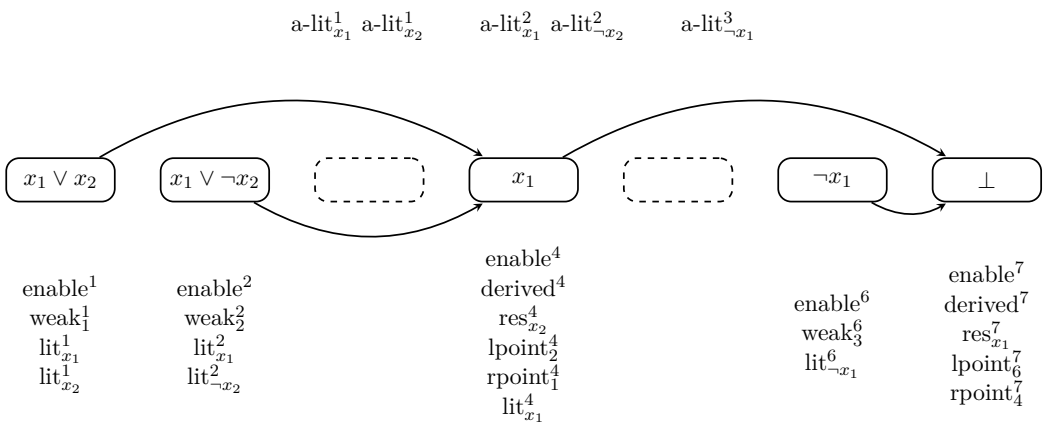


Figure 7. Mapping between a refutation of $\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge \neg x_1$ and a satisfying assignment for $\mathrm{Ref}_7(\varphi)$. Displayed variables are set to 1, the remaining ones are set to 0.

upper bound is preserved in the relativized unary encoding, while Rezende, Göös, Nordström, et al. [RGN+21] showed that it holds for the relativized binary encoding. Here, we use an adaptation of the proof for the relativized unary encoding shown by Arteche, Atserias, Rezende, et al. [AAR+25].

Lemma 3.19. Let φ be a satisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ the formula $\mathrm{Ref}_s(\varphi)$ has a Resolution refutation of length $O(s(m + sn^2))$.

Proof. Let α be a satisfying assignment for φ. We will construct a refutation for $\mathrm{Ref}_s(\varphi)$ using α as a guide. For all $B \in [s]$, we will derive (in increasing order of B) the following clause $\mathrm{True}(B, \alpha)$ asserting that "if block B is enabled then α satisfies it":

$$\mathrm{True}(B, \alpha) := \neg \mathrm{enable}^B \vee \bigvee_{\substack{\ell \in \mathrm{Lit}_n: \\ \alpha(\ell)=1}} \mathrm{lit}^B_\ell \tag{3.2}$$

After deriving $\mathrm{True}(s, \alpha)$, the proof concludes by resolving it sequentially against all clauses REF-10 and REF-11 defined for s.

We start by deriving the clause $Z_1(B) := \mathrm{derived}^B \vee \mathrm{True}(B, \alpha)$ for each $B \in [s]$. Since $\alpha(\varphi) = 1$, for each clause $C_j \in \varphi$ there is at least one literal $\ell^\alpha_j \in C_j$ such that $\alpha(\ell^\alpha_j) = 1$. Now, consider the following auxiliary clause $W(B, j)$:

$$W(B, j) := \neg \mathrm{enable}^B \vee \neg \mathrm{weak}^B_j \vee \bigvee_{\substack{\ell \in \mathrm{Lit}_n: \\ \alpha(\ell)=1}} \mathrm{lit}^B_\ell \tag{3.3}$$

Notice that $W(B, j)$ is a weakening of the axiom clause REF-3(B, j, ℓ^α_j) since the variable a-lit$^j_{\ell^\alpha_j}$ gets deleted by the restriction once φ has been fixed inside $\mathrm{Ref}_s(\Phi)$. This accounts for a total of m weakening steps.

By resolving axiom REF-7(B) sequentially, for all $j \in [m]$, with each $W(B, j)$ over the variable weak^B_j, we obtain the clause $Z_1(B)$. This accounts for a total of m

resolution steps. Since this procedure has to be applied for each block $B \in [s]$, we get a total of $O(sm)$ steps to obtain $Z_1(1), \ldots, Z_1(s)$.

Next, we will derive $\text{True}(B, \alpha)$ for each $B \in [s]$ through an inductive procedure. When $B = 1$, the axiom REF-5(1) degenerates to $\neg\text{enable}^1 \vee \neg\text{derived}^1$ since no block comes before it. Therefore, by resolving REF-5(1) with $Z_1(1)$ over the variable derived^1 we obtain $\text{True}(1, \alpha)$

Fix $B \in [s]$ with $B > 1$ and assume that for each $B' \in [B-1]$ we have already derived $\text{True}(B', \alpha)$. Fix a block $B' \in [B-1]$ and an index $i \in [n]$. Suppose that $\alpha(x_i) = 0$ (we omit the dual case since it can be constructed similarly by following the right-hand pointers). Observe that (REF-1) propagates every literal except x_i and when $\alpha(x_i) = 0$ the literal x_i is absent from $\text{True}(B', \alpha)$. For each $k \in [n]$, set $\ell_k = x_k$ if $\alpha(x_k) = 1$ and $\ell_k = \neg x_k$ if $\alpha(x_k) = 0$.

By resolving $\text{True}(B', \alpha)$ sequentially, for all $k \in [n]$, with each axiom REF-$1(B, B', i, \ell_k)$ over the variable $\text{lit}^{B'}_{\ell_k}$ we get the auxiliary clause $U_1(B, B', i)$ defined as:

$$U_1(B, B', i) := \neg\text{enable}^B \vee \neg\text{enable}^{B'} \vee \neg\text{res}^B_{x_i} \vee \neg\text{lpoint}^B_{B'} \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}^B_\ell \tag{3.4}$$

This requires $O(n)$ steps. By resolving each $U_1(B, B', i)$ with REF-$8(B, B')$ over the variable $\text{enable}^{B'}$, we obtain the clauses $U_2(B, B', i)$ of the form

$$U_2(B, B', i) := \neg\text{enable}^B \vee \neg\text{res}^B_{x_i} \vee \neg\text{lpoint}^B_{B'} \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}^B_\ell \tag{3.5}$$

This requires another $O(n)$ steps. By repeating above process across all $i \in [n]$, we derive the clauses $\{U_2(B, B', i)\}_{i \in [n]}$ for a fixed B' with $O(n^2)$ steps. Now, we cut REF-$4(B)$ sequentially, for all $i \in [n]$, with $U_2(B, B', i)$ over the variable $\text{res}^B_{x_i}$ to obtain the clauses $U_3(B, B')$ of the form

$$U_3(B, B') := \neg\text{enable}^B \vee \neg\text{derived}^B \vee \neg\text{lpoint}^B_{B'} \vee \bigvee_{\substack{\ell \in \text{Lit}_n: \\ \alpha(\ell)=1}} \text{lit}^B_\ell \tag{3.6}$$

Repeating this over all $B' \in [B-1]$, we obtain all the clauses $\{U_3(B, B')\}_{B' \in [B-1]}$ with $O(sn^2)$ steps. Finally, by cutting REF-$5(B)$ sequentially, for all $B' \in [B-1]$, with $U_3(B, B')$ over the variable $\text{lpoint}^{B'}$ we get the clause $\neg\text{derived}^B \vee \text{True}(B, \alpha)$, which can then be resolved with the corresponding clause $\text{derived}^B \vee \text{True}(B, \alpha)$ to get $\text{True}(B, \alpha)$. Similar steps can be applied with right pointers when $\alpha(x_i) = 1$. By applying this whole inductive procedure across all block $B \in [s]$, we get a total of $O(s(m + sn^2))$ steps. □

3.4.3 Decision tree reductions

The main tools that we will use to prove Theorem 3.15(ii) are reductions between formulas that relate the variables of the first formula φ to the variables of the second

formula ψ via shallow decision trees which guarantee that the axioms of φ imply those of ψ [RGN+21; GT26].

Definition 3.20 (Decision tree reduction)**.** Let φ and ψ be, respectively, CNF formulas over the variables $x_1, \ldots, x_n$ and $y_1, \ldots, y_t$. A depth-d decision tree reduction between φ and ψ, written as $\varphi \leq_d^{dt} \psi$ is a function $f : \{0,1\}^n \to \{0,1\}^t$ such that:

- Each output bit f_i is computed by an independent depth-d decision tree $\mathcal{T}_i : \{0,1\}^n \to \{0,1\}$
- For each axiom C of ψ, viewed as a function $C : \{0,1\}^t \to \{0,1\}$, the composition $C \circ f$ can be computed by a depth-$(d \cdot |C|)$ decision tree $\mathcal{T}_C$
- For each axiom C of ψ, every clause of the CNF associated with $\mathcal{T}_C$ is a weakening of an axiom of φ

The key idea behind these reduction is to use the CNF representation property of decision trees to get a formula that is equal to ψ but where each variable y_i is replaced by the corresponding decision tree. This idea naturally translates to a relationship between the widths of the two formulas [RGN+21; AD08]. To formally prove this, we can use the Spoiler-Duplicator game defined in Section 2.4.

Proposition 3.21. If $\varphi \leq_d^{dt} \psi$ then $\text{width}_{\text{Res}}(\varphi) \leq d \cdot \text{width}_{\text{Res}}(\psi)$.

Proof. Let f be the function witnessing $\varphi \leq_d^{dt} \psi$. Given a width-w Spoiler strategy $\mathcal{G}$ for ψ, we define a width-dw Spoiler strategy $\mathcal{F}$ for φ by simulating $\mathcal{G}$ round-by-round through f.

We will use the following invariant during each round of the game: if the game state for $\mathcal{G}$, i.e. the partial assignment to y, records a value $y_i = b$ for some $b \in \{0,1\}$, then the current game state ρ for $\mathcal{F}$, i.e. the partial assignment to x, satisfies $f_i(\rho) = b$ while having at most d values of x_j recorded in ρ.

On each round, if $\mathcal{G}$ queries a variable y_i then $\mathcal{F}$ computes $f_i(x) = b$ through queries and simulates $\mathcal{G}$ by responding $y_i = b$ — in other words, $\mathcal{F}$ plays as the Duplicator against $\mathcal{G}$. When $\mathcal{G}$ forgets a variable y_i, instead, $\mathcal{F}$ forgets all the variables x_j that are not required to know the values of $f_j(x)$ for all those $j \in [t]$ such that the value of y_j is not forgotten by $\mathcal{G}$.

This simulation ensures that width of the strategy of $\mathcal{F}$ is at most dw. Moreover, when the game ends for $\mathcal{G}$ it will also end for $\mathcal{F}$: if $\mathcal{G}$'s game state falsifies an axiom of G, it will also falsify an axiom of $\mathcal{F}$ through the definition of f. □

The above proposition is the main tool to obtain a width lower bound on $\text{Ref}_s(\varphi)$: it suffices to find a proper initial propositional formula from which we can obtain a large width lower bound and an efficient decision tree reduction. It turns out that this final ingredient is the *Retraction Weak Pigeonhole Principle* rPHP_m^{2m} [Jeř07; PT19], a variant of the standard Pigeonhole Principle formula. In particular, we will show that the width of any Resolution refutation of rPHP_m^{2m} is at least $\Omega(m)$ and that it nicely reduces through relatively shallow decision trees to $\text{Ref}_s(\varphi)$ for unsatisfiable formulas φ, formalized through the below theorem.

Theorem 3.22. $\mathrm{rPHP}_m^{2m} \leq^{dt}_{O(n \log m)} \mathrm{Ref}_{m(n-\log m+2)-1}(\varphi)$ when $\varphi \notin \mathrm{SAT}$.

The proof of the lower bound is deferred to Section 3.4.4, while the proof of the reduction is deferred to Section 3.4.5. For now, we show how to use this theorem to obtain a width lower bound on refutational statements. Let m be such that $s = m(n - \log m + 2) - 1$. By the above results we can conclude that:

$$\mathrm{width}_{\mathrm{Res}}(\mathrm{Ref}_s(\varphi)) \geq \Omega\left(\frac{\mathrm{width}_{\mathrm{Res}}(\mathrm{rPHP}_m^{2m})}{n \log m}\right) \geq \Omega\left(\frac{s}{n^2 \log(s/n)}\right) \tag{3.7}$$

However, due to the presence of s in the exponent, this lower bound is not enough to get an exponential size lower bound on Ref. In fact, since $\mathrm{Ref}_s(\varphi)$ has axioms of width $O(s + m + n)$ and has $N = \Theta(s^2 + sm + sn + nm)$ variables, through the general size-width tradeoff by Ben-Sasson and Wigderson defined in Section 2.3.1 we obtain a trivial constant lower bound:

$$\mathrm{size}_{\mathrm{Res}}(\mathrm{Ref}_s(\varphi)) \geq \exp\left(\Omega\left(\frac{(s/(n^2 \log(s/n)) - (s + m + n))^2}{N}\right)\right) = \Omega(1) \tag{3.8}$$

To circumvent this issue, we define a new width measure for Resolution, called *block-width* [AM20; RGN+21; AAR+25]. This measure is defined over any formula φ whose variables $x_1, \ldots, x_n$ can be partitioned into distinct blocks, similar to what is done in $\mathrm{Ref}_s(\Phi)$. When a variable x is part of a block B, we say that x *mentions* B. By extension, a clause mentions a block B if one of its variables mentions B.

Definition 3.23. Let φ be a CNF formula whose variables can be partitioned into blocks. The Resolution block-width of a clause C over the variables of φ, written as $\mathrm{bwidth}(C)$, is defined as the number of different blocks mentioned by the variables of the literals in C, without counting the mentions of the root block s.

By natural extension, the block-width $\mathrm{bwidth}_{\mathrm{Res}}(\pi)$ of a Resolution refutation π for φ is the maximum block-width over all the clauses in π. By extension, the block-width $\mathrm{bwidth}_{\mathrm{Res}}(\varphi)$ of a formula φ is the minimum block-width across all its refutations. Block-width allows us to make a more fine-grained analysis of the variables contained inside the clauses of the refutations of $\mathrm{Ref}_s(\varphi)$. In particular, we can define the so-called *block-aware reductions*, a more fine-grained type of decision tree reductions where all the decision trees that work with the variables of a block are replaced by a single decision tree that works on the whole block.

Definition 3.24 (Block-aware decision tree reduction)**.** Let f be a depth-d decision tree reduction witnessing $\varphi \leq^{dt}_d \psi$. Suppose that the variables $y_1, \ldots y_t$ of ψ are partitioned into blocks. We say that the reduction f is block-aware if for each block $B \subseteq [t]$ there is a depth-d decision tree that computes all the values $f_B(x) := (f_i(x))_{i \in B}$ simultaneously.

Spoiler-Duplicator games can equally well characterize block-width, defined for a game state as the number of blocks that the state records values from.

Proposition 3.25. If $\varphi \leq^{dt}_d \psi$ via a block-aware reduction then $\mathrm{width}_{\mathrm{Res}}(\varphi) \leq d \cdot \mathrm{bwidth}_{\mathrm{Res}}(\psi)$.

Proof. The proof is similar to that of Proposition 3.21, assuming an additional invariant: for each block B for which $\mathcal{G}$ records the value of some variable y_i, the simulation $\mathcal{F}$ knows $f_B(x)$ by recording at most d values of the x_j variables. From this new constraint it follows that if $\mathcal{G}$ has block-width w then $\mathcal{F}$ has width at most dw. □

In Section 3.4.6 we will show that in the case of refutation statements we can achieve better tradeoffs between block-width and size, obtaining our desired exponential lower bound.

3.4.4 The retraction pigeonhole principle

The *Retraction Weak Pigeonhole Principle* formula, denoted with rPHP_m^{2m}, is a variant of the standard formula PHP_m^{2m} that provides an efficient way to invert the mapping between pigeons and holes described by an assignment. This is achieved through two functions $f : [2m] \to [m]$ and $g : [m] \to [2m]$, where the addressing of the mappings is given in binary:

- *Pigeon mapping*: for every pigeon $i \in [2m]$ there are $\lceil \log m \rceil$ variables $f_{i,k}$, $k \in [\log m]$, that describe the binary encoding of the hole $f(i)$ that will be occupied by the i-th pigeon.
- *Hole mapping*: for every hole $j \in [m]$ there are $\log 2m$ variables $g_{j,h}$, $h \in [\log 2m]$, that describe the binary encoding of the pigeon $g(j)$ that will occupy the j-th hole.

The axioms of rPHP_m^{2m} state for every $i \in [2m]$, $j \in [m]$ if $f(i) = j$ then $g(j) = i$ (equivalently, g is a left-inverse function of f). To encode this idea, for each pair $(i, j) \in [2m] \times [m]$ we define the formula $\neg[f(i) = j]$ as:

$$\neg[f(i) = j] := \bigvee_{k \in [\log m]} \ell(i, j, k) \tag{3.9}$$

where $\ell(i, j, k) = f_{i,k}$ if $j_k = 0$, otherwise $\ell(i, j, k) = \neg f_{i,k}$ — recall that j_k is the k-th bit of j's binary encoding. Then, the condition $f(i) = j \to g(j) = i$ is described by:

$$\bigwedge_{h \in [\log 2m]} (\neg[f(i) = j] \vee \gamma(j, i, h)) \tag{3.10}$$

where $\gamma(j, i, h) = g_{j,h}$ if $i_h = 1$, otherwise $\gamma(j, i, h) = \neg g_{j,h}$. Notice that the γ literals basically encode the statement $[g(j) = i]$, rewritten as a CNF formula. Ranging across all pairs, the formula rPHP_m^{2m} has $O(m^2 \log m)$ clauses, each containing $O(\log m)$ literals. We observe that such axioms still imply the pigeonhole principle: injectivity of f follows from the retraction axioms, while totality is automatic in the binary encoding. A Resolution width lower bound can be proven through the Spoiler-Duplicator game by adapting the classic strategy used for the standard pigeonhole principle [PT19; AD08].

Lemma 3.26. $\text{width}_{\text{Res}}(\text{rPHP}_m^{2m}) \geq \Omega(m)$.

Proof. We give a strategy for the Duplicator. Let $w = \lfloor m/2 \rfloor$. The Duplicator maintains a family $\mathcal{H}$ of partial assignments α with $|\text{dom}(\alpha)| \leq w$. Each $\alpha \in \mathcal{H}$ is associated with a partial injective mapping $q_\alpha : P_\alpha \to H_\alpha$, where $P_\alpha \subseteq [2m]$ and $H_\alpha \subseteq [m]$ with $|P_\alpha| = |H_\alpha| \leq |\text{dom}(\alpha)| \leq w$, such that α is consistent with q_α.

Let α be the Duplicator's assignment of the current game state, where $|\text{dom}(\alpha)| < w$. Suppose that the Spoiler queries a variable $f_{i,k}$ (analogous argument for variables $g_{j,h}$, where pigeons and holes are inverted). The idea is to associate each of the $\log m$ hole bits to a different pigeon. In particular, if $i \notin P_\alpha$ then $|H_\alpha| \leq |\text{dom}(\alpha)| < w$ guarantees that at least one hole $j' \in [m] \setminus H_\alpha$ is unassigned. Hence, the Duplicator answers with j'_k to then set $\alpha' = \alpha \cup \{f_{i,k} \mapsto j'_k\}$ and $q_{\alpha'} = q_\alpha \cup \{i \mapsto j'\}$ to keep track of progress.

When the Spoiler forgets a variable $f_{i,k}$ (or $g_{j,h}$), the Duplicator removes the mappings in α and q_α associated with pigeon i (or hole j). This strategy guarantees that the Duplicator preserves local consistency without exhausting available pigeons or holes as long as at most w variables are assigned. □

3.4.5 Reduction from rPHP to Ref for unsatisfiable formulas

We give a generalization of the argument shown in [AM20; Gar19; RGN+21], while also sharpening the exact values used in the proof. In particular, we generalize the reduction $\text{rPHP}_n^{2n} \leq^{dt}_{O(n \log n)} \text{Ref}_{n^2}(\varphi)$ to any value $s = m(n - \log m + 2) - 1$ by proving that $\text{rPHP}_m^{2m} \leq^{dt}_{O(n \log m)} \text{Ref}_{m(n-\log m+2)-1}(\varphi)$.

Let T_n be the following full tree-like Resolution refutation of the formula φ, defined on the variables $x_1, \ldots, x_n$. We assume that the weakening rule is applied only on the leaves and that at each layer $i \in [n]$ of the tree the resolution rule is always applied on the variable x_i. Therefore, T_n has 2^n leaves corresponding to all the possible width-n clauses, each being a weakening of some axiom of φ. For any assignment α to rPHP_m^{2m}, the reduction is going to produce an assignment β to $\text{Ref}_s(\varphi)$ that represents a purported refutation of size s of φ isomorphic to some subtree $T_\alpha \subseteq T_n$ of the full tree T_n.

The set of nodes of T_α that have smaller degree in T_α than in T_n is referred to as the *boundary* of T_α. Clearly, T_α is not a valid refutation of φ since the boundary nodes have at least one missing child, but the non-boundary nodes will be indistinguishable from the corresponding neighborhoods of T_n. For this reason, the only axiom violations of $\text{Ref}_s(\varphi)$ will be a result of the boundary nodes.

Theorem 3.22 (restated)**.** $\text{rPHP}_m^{2m} \leq^{dt}_{O(n \log m)} \text{Ref}_{m(n-\log m+2)-1}(\varphi)$ when φ is an unsatisfiable CNF formula. Moreover, the reduction is block-aware.

Proof. Let $s = m(n - \log m + 2) - 1$. Given a total assignment α to rPHP_m^{2m}, consider the subtree $T_\alpha \subseteq T_n$ defined as above. We will use α to identify each block B of $\text{Ref}_s(\varphi)$ with some node in T_n. Once the position of the node is known, the values of the variables in block B will be completely defined. To achieve this, we define the

following partial mapping $h : [2m] \to [m] \cup \{*\}$:

$$h(i) = \begin{cases} f(i) & \text{if } g(f(i)) = i \\ * & \text{otherwise} \end{cases} \tag{3.11}$$

Observe that even if α is a total assignment to rPHP_m^{2m}, the function h is still necessarily undefined for some pigeons. The below claim explains the reason why we are using the retraction pigeonhole principle instead of the standard one.

Claim 3.27. *h and its inverse can be computed by a decision tree of depth $O(\log m)$*

Proof of the claim. Fix $i \in [2m]$. By querying the variables $\{f_{i,k}\}_{k\in[\log m]}$, we get the full decision tree $\mathcal{T}_f$ computing $f(i)$. Then, on each leaf j of $\mathcal{T}_f$ we attach the full decision tree $\mathcal{T}_g$ computing $g(j)$ by querying the variables $\{g_{j,h}\}_{h\in[\log 2m]}$. Let $\mathcal{T}_h$ be the decision tree obtained after attaching each $\mathcal{T}_g$. Every leaf of $\mathcal{T}_h$ such that $g(j) = i$ outputs j, while all the others output $*$. This decision tree computes h with depth $\log m + \log 2m$. For the inverse of h, a similar approach can be used by inverting the roles of $\mathcal{T}_f$ and $\mathcal{T}_g$. □

Consider now the graph G_α with $n+1$ layers $\ell \in \{0, \ldots, n\}$, each containing $\min\{2^\ell, m\}$ nodes. Notice that this implies that the first $\lfloor \log m \rfloor$ layers make up a complete binary tree structure with $(m-1)$ nodes, while all the remaining $n - \log m + 1$ layers contain m nodes, for a total of s nodes. This hardcoded part of the graph will be used to force the isomorphism previously discussed. We highly suggest reading the rest of this proof while looking at Figure 8.

The idea is to map each block of $\mathrm{Ref}_s(\varphi)$ to a node of G_α. In particular, the last block B_s is placed at the root of G_α, the node on layer 0, and the rest of the blocks are mapped to the graph in decreasing order from left to right across all layers.

Next, we consider the block pointers given by the total assignment α. For each of these pointers, we will add an edge to G_α corresponding to it. For the first $\lfloor \log m \rfloor$ layers, we hardcode the edges in G_α in a way that they represents a full binary tree. Notice that this part of the pointer assignment does not depend on the variables α.

Now, for each of the remaining layers $\ell \in [n] \setminus [\log m]$, we think of the $2m$ pointers going from the m blocks of layer ℓ to those of layer $\ell + 1$ as the $2m$ pigeons going to the m holes, as defined by h. More precisely, we define the left pointer of the i-th block on layer ℓ as the $h(2i-1)$-th block on layer $\ell + 1$. Similarly, the right pointer is the $h(2i)$-th block on layer $\ell + 1$. When $h(i)$ is undefined, which can happen only when an axiom of rPHP_m^{2m} associated with i is violated, we define the corresponding pointer as *null* and we point it directly to the root block B_s, which automatically results in an axiom violation for $\mathrm{Ref}_s(\varphi)$.

We observe that the resulting topology (ignoring null pointers) is a forest of binary trees: it is constructed by stitching together a binary tree at the top with a layered sequence of copies of the same partial matchings where we have identified pairs of pigeons. Now that the topology of G_α has been defined, we will show that one of its subtrees is isomorphic to a subtree $T_\alpha \subseteq T_n$.

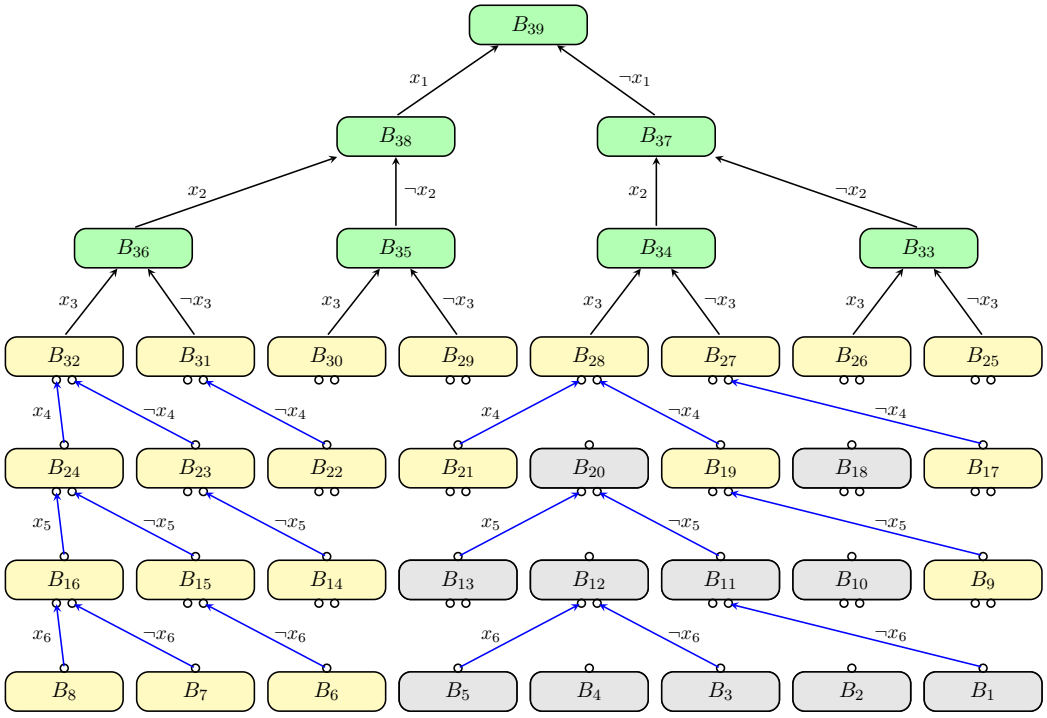


Figure 8. The graph G_α of the reduction from rPHP_m^{2m} to $\mathrm{Ref}_{m(n-\log m+2)-1}(\varphi)$ with $n = 6$ and $m = 8$. The copies of the partial matching defined by $h : [2m] \to [m]$ are represented by **blue** arrows. Hardcoded nodes are represented in **green**. Derived nodes that survive the walk-up phase are represented in yellow, while disabled blocks are represented in gray. The subtree isomorphic to T_α is given by green and yellow nodes.

Consider the following procedure to construct a decision tree that computes the literal set for a block B. Starting from B, we walk up G_α to its unique parent in the binary forest by computing the inverse of h using $O(\log m)$ queries. Then, we repeat this process walking upwards for at most n steps until we reach a block B^* without a parent. We have two cases:

- $B^* = B_s$. Consider the reverse path p from B^* to B, viewed as a sequence of left and right turns. The path p identifies a node v of T_α obtained by following the same sequence of turns described by p. Once v is found, we copy all the local structure of v into B. In particular, the literal set of B is made equal to that of v, where a left turn at layer ℓ is associated with x_ℓ and a right turn is associated with $\neg x_\ell$. If v is derived in T_α by resolving on x_k, we make B a derived block and set its resolved variable to x_k. Similarly, if v is a leaf of T_α that is a weakening of some axiom $C_A \in \varphi$, we make B an axiom block and set its weakening index to A.
- $B^* \neq B_s$. In this case, the block B is set as disabled since it is not part of the subtree isomorphic to T_α. All of the other variables of B are set to 0 since they are not needed.

Through this procedure, all variables of a single block are computed from the same walk-up path, so one depth-$O(n \log m)$ decision tree computes the whole block.

To conclude the proof of the theorem it remains to show that every axiom in $\mathrm{Ref}_s(\varphi)$ (composed with the reduction) is implied by some axiom of rPHP_m^{2m}. We argue the contrapositive statement. Since the reduction produces a purported refutation isomorphic to a subtree $T_\alpha \subseteq T$ together with some disabled blocks which do not violate axioms of $\mathrm{Ref}_{m(n+1)-1}(\varphi)$, the only possible axiom violations in $\mathrm{Ref}_{m(n+1)-1}(\varphi)$ are caused by a block on a non-hardcoded layer $\ell \in [n] \setminus [\log m]$ containing a null pointer. Any null pointer is caused by the decision tree querying a pigeon i with $h(i) = *$, which means that the decision tree has witnessed an axiom violation of the form $f(i) = j \to g(j) = i$ for rPHP_m^{2m}. □

3.4.6 A lower bound on Ref for unsatisfiable formulas

To conclude the proof of Theorem 3.15(ii), we show that an improved tradeoff between block-width and size holds for the formula $\mathrm{Ref}_s(\varphi)$. This tradeoff was first used by Atserias and Müller [AM20], using a similar measure called *index-width*. Here, we restate it using the concept of block-width, similar to what is done in [RGN+21].

The tradeoff is based on a *random restriction* argument [BP96; BW01], another tool typically used to prove Resolution lower bounds. The idea is to randomly compute a restriction that has non-zero probability of having the desired property, which implies that there must be at least one restriction that possesses it.

Lemma 3.28. Let φ be an unsatisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ with $s \geq n^2$, it holds that $\mathrm{bwidth}_{\mathrm{Res}}(\mathrm{Ref}_{s/2}(\varphi)) \leq O(\log \mathrm{size}_{\mathrm{Res}}(\mathrm{Ref}_s(\varphi)))$

Proof. We start by observing that $\mathrm{Ref}_{s-1}(\varphi)$ is a subformula of $\mathrm{Ref}_s(\varphi)$ (after renaming variables). Therefore, we can assume that s is odd in order to simplify our argument. We define a partial assignment ρ obtained through the following randomized procedure:

1. Initialize ρ as the empty mapping.
2. Pair the first $s-1$ blocks into $(s-1)/2$ pairs
3. For each pair (B_i, B_{i+1}), flip a balanced coin c_i. If $c_i = 0$ then set $\rho \leftarrow \rho \cup \{\mathrm{enable}^{B_i} \mapsto 0\}$. Otherwise, set $\rho \leftarrow \rho \cup \{\mathrm{enable}^{B_{i+1}} \mapsto 0\}$.
4. After disabling one block of each pair, we set all the remaining blocks as enabled. Moreover, for each disabled block we randomly assign all its remaining variables.

Let π be a Resolution refutation of $\mathrm{Ref}_s(\varphi)$. After the above procedure is completed, ρ decides for each pair which of the two blocks is disabled, with probability $1/2$. Therefore, the restricted refutation $\pi_{\restriction\rho}$ is refutation of $\mathrm{Ref}_s(\varphi)_{\restriction\rho} = \mathrm{Ref}_{s/2}(\varphi)$ (up to renaming variables). Moreover, we observe that leaving out the last block B_s in the above procedure guarantees that enable^{B_s} is never set to 0 in the restriction, otherwise $\mathrm{Ref}_s(\varphi)_{\restriction\rho}$ would contain the empty clause $\bot$, hence a refutation of size 1.

Claim 3.29. $\mathrm{bwidth}_{\mathrm{Res}}(\pi_{\restriction\rho}) \leq O(\log |\pi|)$ with probability $p > 0$.

Proof. For each block B_i, with $i \in [s-1]$, consider any literal ℓ mentioning B_i. After applying the above procedure, if ℓ is a literal over enable^{B_i} then $\Pr_\rho[\rho(\ell) = 1] = 1/2$

trivially follows by the procedure. If ℓ is any other variable mentioning B_i, instead, we have that:

$$\Pr_\rho[\rho(\ell) = 1] \geq \Pr_\rho[\rho(\ell) = 1, B_i \text{ disabled}] = 1/4$$

Therefore, it always holds that $\Pr_\rho[\rho(\ell) = 1] \geq 1/4$. Consider now any clause C in π of block-width w (value to be determined later). Since C mentions at least $w/2$ different pairs and since each pair is independent from the others, the probability that the whole clause did not get killed in the restriction by setting any of its literals to 1 is bounded by $\Pr_\rho[C_{\restriction_\rho} \neq 1] \leq (3/4)^{w/2}$. Applying the union bound across all clauses of π, we get that:

$$\Pr_\rho[\exists C \in \pi \text{ with } \text{bwidth}(C_{\restriction_\rho}) \geq w] \leq |\pi| \cdot \left(\frac{3}{4}\right)^{w/2} \tag{3.12}$$

In order for the claim to hold, the above bound must be smaller than $1 - p$. Indeed, this is true when:

$$w \geq \frac{2(\log|\pi| - \log(1-p))}{\log(4/3)} \geq \frac{2\log|\pi|}{\log(4/3)} \tag{3.13}$$

□

Since the above claim holds with probability greater than 0, there must be at least one partial assignment ρ^* for which it holds. The lemma follows by choosing π as a refutation of $\text{Ref}_s(\varphi)$ with the smallest size possible. □

Lemma 3.30. Let φ be an unsatisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ with $s \geq n^2$, the formula $\text{Ref}_s(\varphi)$ requires Resolution refutations of block-width $\Omega\left(\frac{s}{n^2 \log(s/n)}\right)$.

Proof. For each integer $m \geq 0$, define $s_m := m(n - \log m + 2) - 1$. Let m^* be the maximum integer satisfying $s \geq s_{m^*}$. By Theorem 3.22 we know that $\text{rPHP}^{2m^*}_{m^*} \leq^{dt}_{O(n \log m^*)} \text{Ref}_{s_{m^*}}(\varphi)$ via a block-aware reduction. By Lemma 3.26 and Proposition 3.25 we get that:

$$\text{bwidth}_{\text{Res}}(\text{Ref}_{s_{m^*}}(\varphi)) \geq \Omega\left(\frac{\text{width}_{\text{Res}}(\text{rPHP}^{2m}_m)}{n \log m}\right) \geq \Omega\left(\frac{m^*}{n \log m^*}\right) \tag{3.14}$$

Since any proof of $\text{Ref}_s(\varphi)$ can be restricted to a proof of $\text{Ref}_{s_{m^*}}(\varphi)$ by disabling $s - s_{m^*}$ blocks, we know that $\text{bwidth}_{\text{Res}}(\text{Ref}_s(\varphi)) \geq \text{bwidth}_{\text{Res}}(\text{Ref}_{s_{m^*}}(\varphi))$. Moreover, by maximality of m^*, we know that $s < (m^*+1)(n - \log m + 2) - 1 < (m^*+1)(n+2)$, implying that $m^* > s/(n+2) - 1$. This concludes that any Resolution proof of $\text{Ref}_s(\varphi)$ must have block-width $\Omega\left(\frac{s}{n^2 \log(s/n)}\right)$. □

By combining this with Lemma 3.28, we obtain our exponential lower bound of $\text{size}_{\text{Res}}(\text{Ref}_s(\varphi)) \geq 2^{\Omega(s/(n^2 \log(s/n)))}$. For completeness, we restate Theorem 3.15 with the precise size bounds given by the lemmas of the last sections. By choosing a nice value of s, say n^3, this bound becomes easy to work with for a more precise proof of Lemma 3.16.

Theorem 3.15 (restated)**.** Let φ be a formula with n variables and m clauses.

(i) If $\varphi \in \text{SAT}$ then $\text{Ref}_s(\varphi)$ has a Resolution refutation of size $O(s(m + sn^2))$;

(ii) If $\varphi \notin \text{SAT}$ then every Resolution refutation of $\text{Ref}_s(\varphi)$ has size $2^{\Omega(s/(n^2 \log(s/n)))}$.

All the results proven can be extended to other weak proof systems by similar arguments. In particular, various results [RGN+21; Gar24; GKM+20; IR22; Pap24] extended them by considering variants of the refutation formula, either by imposing more structure, such as forcing the encoded refutation to be tree-like, or by lifting its complexity by composing it with gadget formulas. To this day, NP-hardness of automatability has been extended with this meta-complexity approach up to the highest weak proof systems commonly studied: Cutting Planes [GKM+20], Polynomial Calculus [RGN+21] and AC^0-Frege [Pap24]. However, some weak proof systems (Parity Resolution [IS20], Sherali-Adams [SA90], Sum-of-Squares [Las01], ...) still resist this argument, but only due to the lack of proper ways of proving lower bounds. In particular, the original version of [RGN+21] claimed the non-automatability of Sherali-Adams, but this statement was later retracted in a revised version.

Recently, Arteche, Atserias, Rezende, et al. [AAR+25] proved that refutation statements are actually *universal* for automatability. In particular, they proved that any reasonably strong proof system (at least above EF) is (weakly) automatable if and only if it is (weakly) automatable on refutation statements. This shows that the non weakly automatability results of systems from EF to above can, in some way, be simplified by restricting them to $\text{Ref}_s(\varphi)$.

Chapter 4

Analyzability

4.1 Do proofs of lower bounds leak information?

Pudlák's upper bound (Theorem 3.15(i)) shows that satisfying assignments can be used to construct short proofs of Resolution lower bounds. Atserias and Müller showed through their lower bound (Theorem 3.15(ii)) that these two concepts are also connected in the opposite direction: if one such proof has sub-exponential size then the formula must be satisfiable. However, their lower bound argument is non-constructive, meaning that it does not extract a satisfying assignment from sub-exponential proofs. Therefore, it seems natural to ask if this is also possible.

Arteche, Atserias, Rezende, et al. [AAR+25] investigated this possibility by introducing a new concept somewhat dual to automatability, referred to as *analyzability*. Informally, the *Proof Analysis Problem* (PAP) for a proof system Q asks to establish if a formula φ is satisfiable or not by looking at a Q-proof of the Resolution lower bound $\neg\mathrm{Ref}_s(\varphi)$ (always recall that the refutation statement is defined for Resolution when no proof system is specified).

Definition 4.1. Let Q be a proof system. The Proof Analysis Problem over Q for Resolution lower bounds of size at least $\sigma(n)$ is defined as the language

$$\mathrm{PAP}_Q[\sigma(n)] = \{(\varphi, \pi, 1^s) \mid \varphi \in \mathrm{SAT}, \pi : Q \vdash \neg\mathrm{Ref}_s^{\mathrm{Res}}(\varphi), s \geq \sigma(|\varphi|)\}$$

The search version of the problem $\mathrm{FPAP}_Q[\sigma(n)]$ is defined analogously, asking to return a satisfying assignment (if one exists).

We observe that the restriction imposed by $\sigma(n)$ on the size of the lower bounds is necessary. For most reasonable proof systems, lower bounds that are too small admit some degenerate instances that make the problem trivially NP-hard. For instance, each $\varphi \in \mathrm{SAT}$ can be mapped to the triple $(\varphi, \eta, 1)$ of $\mathrm{PAP}_{\mathrm{Res}}[1]$ where π is a trivial Resolution refutation of $\mathrm{Res}_1(\varphi)$ (notice that φ has a refutation of size 1 if and only if $\bot \in \varphi$). To avoid these issues, we will work with values $\sigma(n) \geq n^2$.

Definition 4.2 (Analyzability)**.** Let Q be a proof system. Given two functions $\sigma, t : \mathbb{N} \to \mathbb{N}$, we say that Q is analyzable in time t for lower bounds of size σ if $\mathrm{PAP}_Q[\sigma(n)]$ is decidable in time t.

When both σ and t are polynomial functions, we simply say that the proof system is analyzable. We notice that, by definition, analyzability acts as a bridge from automatability to solving SAT. This follows from an argument very similar to what is used to show the NP-hardness of weak proof systems (Lemma 3.16).

Proposition 4.3. Let Q be a proof system. If Res $\leq_p Q$ then Q cannot be both automatable and analyzable, unless P = NP.

Proof. Suppose that Q has an automating algorithm $\mathcal{A}$ and an analyzing algorithm $\mathcal{B}$, both with polynomial runtime. Since Res $\leq_p Q$, by Theorem 3.15(i) we know that when φ is satisfiable there is a Q-proof of size $O(n^c)$ for some $c > 0$.

We define a new algorithm $\mathcal{C}$ that solves SAT using this time bound. Given an input formula φ, $\mathcal{C}$ constructs $\mathrm{Ref}_s(\varphi)$ and executes $\mathcal{A}$ for at most N^c steps, where $N = |\mathrm{Ref}_s(\varphi)|$ and $s = n^d$, for $d > 0$. If this time bound is exceeded, $\mathcal{C}$ immediately rejects. Otherwise, $\mathcal{C}$ outputs $\mathcal{B}(\varphi, \pi, 1^s)$, where π is the proof returned by $\mathcal{A}$. □

We observe that the above argument differs from the one of Lemma 3.16 only by one thing: instead of using the lower bound in the case of unsatisfiable formulas, we directly use PAP to compute SAT once the proof is computed. Of course, the Resolution lower bound can be used to prove that Resolution is indeed analyzable, obtaining a simplified argument for its NP-hardness.

Proposition 4.4. $\mathrm{PAP}_{\mathrm{Res}}[n^3 \log n] \in \mathsf{P}$

Proof. Consider any refutation π of $\mathrm{Ref}_s(\varphi)$, with $s \geq n^3 \log n$ and $n = |\varphi|$. By Theorem 3.15(ii), there is a constant $\varepsilon > 0$ such that if $|\pi| < 2^{\varepsilon n}$ then φ is satisfiable.

We use this to construct the following algorithm $\mathcal{A}$. On input $(\varphi, \pi, 1^s)$, $\mathcal{A}$ checks that π is a valid refutation and computes its length. If it stands below $2^{\varepsilon n}$, $\mathcal{A}$ immediately accepts. Otherwise, $\mathcal{A}$ bruteforces SAT on φ using the standard algorithm with runtime $O(2^n)$. Observe that the input size of $\mathcal{A}$ is $M = |\varphi| + |\pi| + s$, thus $\mathcal{A}$ runs in linear time. □

Observation 4.5. In the original result [AAR+25], the above result is stated as $\mathrm{PAP}_{\mathrm{Res}}[n^2] \in \mathsf{P}$. This improved result is obtained by using the original Atserias-Müller lower bound, which is slightly better than the one obtained through the reduction in Section 3.4.5. For our purposes, this is no issue.

we will refer to the above proof strategy as the "length test". This test can be applied to any proof system with a *Ref-hardness* result similar to that of Resolution.

Definition 4.6 (Ref-hardness)**.** A proof system Q is (c, ε)-Ref-hard, with $c, \varepsilon > 0$, when for every unsatisfiable formula φ on n variables and for every $s \geq n^c$ it holds that $|\pi| \geq 2^{\varepsilon n}$ for all $\pi : Q \vdash \neg\mathrm{Ref}_s(\varphi)$.

In particular, if Q is (c, ε)-Ref-hard then $\mathrm{PAP}_Q[n^c] \in \mathsf{P}$. By Theorem 3.15(ii), Resolution is $(4, \varepsilon)$-Ref-hard. In Section 4.3 we will prove Polynomial Calculus with Resolution is also $(4, \varepsilon)$-Ref-hard, concluding that it is also analyzable.

In Section 3.4.6 we mentioned how the lower bounds behind non-automatability of other weak systems can be obtained through variants of $\mathrm{Ref}_s(\varphi)$. However, due to the fact that analyzability is strictly defined over this exact formula, those similar arguments break down completely in this context: one would have to prove that the lower bounds for the variants also holds for the standard formula $\mathrm{Ref}_s(\varphi)$. Furthermore, it turns out Ref-hardness is not only sufficient but also necessary for most proof system [AAR+25].

Theorem 4.7. Let Q be a proof system that p-simulates Resolution, is constructively closed under modus ponens and contrapositions and proves that Resolution is closed under restrictions. Under some standard complexity assumption, the following are equivalent:

- Q is analyzable;
- Q is (c, ε)-Ref-hard for some $c, \varepsilon > 0$.

Proof. Omitted. The reader is referred to [AAR+25]. □

Another important property to observe is that analyzability is downward closed under p-simulations, while automatability is not. This comes by trivially mapping a PAP triple in the first proof system to the PAP triple in the second proof system, substituting the old proof with the one obtained through the simulation. Therefore, proving that a proof system Q above Resolution is analyzable automatically implies that no proof system between Q and Resolution is automatable, unless $\mathsf{P} = \mathsf{NP}$.

Proposition 4.8. Let P, Q be two proof systems such that $Q \leq_p P$. Then, if P is (search-)analyzable Q also is.

we will use this to conclude that every proof system below PCR is analyzable. Notice that this does not imply a simplified proof of the NP-hardness of automating NS and PC since they are incomparable with Resolution.

In the very same article, Arteche, Atserias, Rezende, et al. [AAR+25] point out that analyzability fails in the context of strong proof systems. In particular, they prove that Extended Frege is NP-hard to analyze, even in the decisional version, through the fact that EF can prove [CP90] Haken's Resolution lower bound for the pigeonhole principle [Hak85]. In particular, they extend this by showing that for every φ the system EF always has a proof π_φ of $\neg\mathrm{Ref_s}(\varphi \vee \mathrm{PHP}_n)$ constructible in polynomial time n^c. The NP-hardness follows by mapping $\varphi \mapsto (\varphi, \pi_\varphi, 1^{n^c})$, solving SAT in polynomial time. In a more recent article [ARK+26], they extended this result by proving that AC^0-Frege is not analyzable in quasi-polynomial time. These non-analyzability results shows that the technique of proving non-automatability results through upper and lower bounds on $\mathrm{Ref}_s(\varphi)$ fails for Frege Systems and above.

4.2 Search-analyzability of Resolution

In this section we discuss the main result of [AAR+25], which showed that satisfying assignments can be algorithmically extracted from short Resolution proofs of

refutation statements, settling the question regarding a constructive proof for the contrapositive of the Atserias-Müller lower bound.

The extraction algorithm is subdivided in two phases: a *block-width reduction* phase and a *proof-analysis* phase. In the first phase, a deterministic version of the block-width restriction of Lemma 3.28 is applied to to the given $\mathrm{Ref}_s(\varphi)$ proof π, obtaining a smaller proof with a known block-width lower bound for $\mathrm{Ref}_{s'}(\varphi)$ with $s' < s$. This deterministic version gives a slightly worse lower bound, but it suffices for the extraction argument. Then, the second phase analyzes the restricted proof by traversing it from the root to the axioms, arriving at one of three conclusions: a satisfying assignment is found, a clause with large block-width is found or an error in the proof is found.

We start by giving a more precise definition of the type restrictions that we are going to use for $\mathrm{Ref}_s(\varphi)$.

Definition 4.9 (d-disabling restriction)**.** Let ρ be an assignment to the N variables of $\mathrm{Ref}_s(\varphi)$. We say that ρ is a d-disabling restriction when:

(i) exactly d blocks of $\mathrm{Ref}_s(\varphi)$ are disabled by ρ and all remaining ones are enabled;
(ii) every variable belonging to a disabled block has a value assigned in ρ;
(iii) no other variables are assigned.

it is easy to see that the argument in Lemma 3.28 yields a $\lfloor s/2 \rfloor$-disabling restriction. Notice that this never falsifies the formula since all axioms involving a disabled block are satisfied independently of the values assumed by its variables.

Corollary 4.10. Let φ be an formula. For each probability $0 < p < 1$, there is a randomized algorithm $\mathcal{R}$ that given 1^N as input, where N is the number of variables in $\mathrm{Ref}_s(\varphi)$, returns a $\lfloor s/2 \rfloor$-disabling restriction ρ such that for each Resolution-refutation π of $\mathrm{Ref}_s(\varphi)$ the following holds:

- ρ does not falsify $\mathrm{Ref}_s(\varphi)$;
- with probability at least p the block width of $\pi_{\restriction_\rho}$ is at most $O(\log |\pi| - \log(1-p))$
- $\mathcal{R}(1^N)$ runs in $O(N)$.

To make this restriction deterministic, we use a greedy assignment policy based on killing the remaining clauses with the highest block-width. Given a set W of clauses and a literal ℓ mentioning block B_i, the *frequency* of ℓ in X, let $\nu(X, \ell)$ be the number of times ℓ appears in clauses of X divided by $|X|$. By extension, let $\nu(X, B_i)$ be the number of times block B_i is mentioned in clauses of X, divided by $|X|$. Consider the following algorithm, inspired by result in [CP90; BP96; BW01].

Algorithm 2 A deterministic algorithm $\mathcal{A}$ that reduces block-width

Require: A triple $(\varphi, \pi, 1^s)$ where π is a Res-refutation of $\mathrm{Ref}_s(\varphi)$
Ensure: A d-disabling restriction
$\rho \leftarrow \varnothing$
Active $\leftarrow [s-1]$
Disabled $\leftarrow \varnothing$

$W \leftarrow \{C \in \pi \mid \text{bwidth}(C) \geq w\}$ ▷ *w is a parameter to be specified later*
while $W \neq \varnothing$ and $\exists i \in$ Active with B_i mentioned in W **do**
  Find $i \in$ Active that maximizes $\nu(W, B_i)$
  $W_{B_i} \leftarrow \{C \in W \mid C \text{ mentions } B_i\}$
  if $\nu(W_{B_i}, \text{enable}^{B_i}) \geq 1/3$ **then**
    $\rho \leftarrow \rho \cup \{\text{enable}^{B_i} \mapsto 1\}$
    Active $\leftarrow$ Active $\setminus \{i\}$
    $W \leftarrow W_{\restriction_\rho}$
  else
    $\rho \leftarrow \rho \cup \{\text{enable}^{B_i} \mapsto 0\}$
    Disabled $\leftarrow$ Disabled $\cup \{i\}$
    Active $\leftarrow$ Active $\setminus \{i\}$
    $W \leftarrow W_{\restriction_\rho}$
    for every variable x of block B_i **do**
      if $\nu(W, x) \geq \nu(W, \neg x)$ **then**
        $\rho \leftarrow \rho \cup \{x \mapsto 1\}$
      else
        $\rho \leftarrow \rho \cup \{x \mapsto 0\}$
Update ρ by enabling all blocks that were not mentioned in W
Return ρ

The above algorithm is dense in notation, but its workings are quite intuitive. The set W corresponds to the set of clauses with initial block width exceeding a to-be-fixed parameter. On each iteration, we search for the active block that maximizes its frequency in W. If the enable variable of B_i appears in at least $1/3$ of the clauses in W that mention it, we enable block B_i and update W accordingly. Otherwise, we disable block B_i to then set as true all of its variables that appear more positively than negatively in W, while setting all its other variables to false.

Lemma 4.11. Let π be a Resolution refutation for $\text{Ref}_s(\varphi)$. For an adequate choice of w, Algorithm 2 $\mathcal{A}$ returns a d-disabling restriction ρ in $O(s\,|\pi|)$ such that:

- ρ does not falsify $\text{Ref}_s(\varphi)$;
- $d \leq \frac{c}{2}\sqrt{s \log |\pi|}$
- the block-width of $\pi_{\restriction_\rho}$ is at most $c\sqrt{s \log |\pi|}$

Proof. By construction, the procedure terminates once W is either empty or when all clauses in W mention blocks that are no longer active. Once it terminates, exactly $|D|$ blocks have been disabled, while all the others have been set as enabled. The algorithm runs for at most s iterations, each computable in at most $O(|\pi|)$ steps.

We want to choose a value w so that after $\ell \leq s$ iterations, the set W is guaranteed to be empty. Once W gets emptied by the procedure, all clauses left in $\pi_{\restriction_{\rho_\ell}}$ will have block-width at most w. Since each iteration enables or disables exactly one block, on each iteration j there are at least $s - j - 1$ active blocks, at most $j - 1$ disabled blocks and the block width might have decreased up to $w - (j - 1)$. Through an averaging argument, we know that the most frequent block is mentioned in at least a $(w - (j - 1))/(s - (j - 1))$ fraction of the clauses left in W.

Moreover, if block B_i is chosen in iteration j, the procedure does not guarantee that all the clauses mentioning B_i are killed, but we are guaranteed to kill at least $1/3$ of them. Indeed, if block B_i gets enabled this is possible only if at least $1/3$ of all the clauses in W mention it. Otherwise, if it gets disabled, we are guaranteed to restrict at least $1/2$ of the remaining at least $2/3$ fraction of the clauses in W mentioning B_i, which amounts to at least $1/3$ of clauses. Therefore, after ℓ iterations we have that:

$$\begin{aligned}\left|W_{\restriction_{\rho_\ell}}\right| &\leq |W| \left(1 - \frac{w}{3s}\right)\left(1 - \frac{w-1}{3(s-1)}\right)\cdots\left(1 - \frac{w-(\ell-1)}{3(s-(\ell-1))}\right) \\ &\leq |W| \left(1 - \frac{w-\ell}{3s}\right)^{\ell} \\ &< |W|\, e^{-\ell \cdot \frac{w-\ell}{3s}}\end{aligned} \tag{4.1}$$

where the last inequality comes from the fact that $1-x < e^{-x}$ for all $x \neq 0$. To ensure that W is eventually emptied, it suffices to choose a value of ℓ such that $\left|W_{\restriction_{\rho_\ell}}\right| < 1$, which is true for $\ell = \lfloor w/2 \rfloor$ and $w > \sqrt{12 \ln |\pi|} > \sqrt{12s \ln |W|}$. By choosing a constant $c > 0$ such that $w = c\sqrt{s \log |\pi|} > \sqrt{12 \ln |\pi|}$, all constraints are satisfied. This concludes that $\mathrm{bwidth}(\pi_{\restriction_{\rho_\ell}}) \leq c\sqrt{s \log |\pi|}$ and $|D| \leq \ell = \frac{c}{2}\sqrt{s \log |\pi|}$. □

We now move to the proof-analysis phase. We start by observing that this phase can be used after both the randomized and deterministic algorithms, obtaining improved results for the randomized one. The idea behind the algorithm is to traverse the restricted refutation in a fashion inspired by the Spoiler-Duplicator game defining block-width (see Section 2.4 and Section 3.4.3). In particular, we assume that $s-d$ blocks in the restricted formula $\mathrm{Ref}_s(\varphi)_{\restriction_\rho}$ are arranged in n layers, each containing $\lfloor (s-d)/n \rfloor$ blocks, plus one additional top layer containing the root node s, labeled as layer 0. Similarly to what is done in the reduction from rPHP to $\mathrm{Ref}_s(\varphi)$ (see Section 3.4.5), we assume that each block on layer $0 \leq i < n$ is used to resolve on variable x_{i+1}. Blocks on layer n are assumed to describe a clause that is a weakening of some axiom of φ.

Throughout the traversal, we will build a partial assignment α on the variables of $\mathrm{Ref}_s(\varphi)$, referred to as the *reservation*. This assignment keeps track of the information needed to continue traversal, mimicking what the Duplicator has in mind while playing against the Spoiler. In particular, since $\mathrm{Ref}_s(\varphi)$ encodes a potential refutation, the reservation α will encode information about how blocks are connected. Just like the proof of Theorem 3.22, we suggest that the reader looks at Figure 9 while reading what comes next.

In the following algorithm, we will denote the variables of $\mathrm{Ref}_s(\varphi)$ with $y_1, \ldots, y_N$ and the variables of φ with $x_1, \ldots, x_n$. Similarly, clauses of $\mathrm{Ref}_s(\varphi)$ and π are denoted with capital font C, while clauses of φ are denoted with script font $\mathscr{C}$. Blocks are labeled with fraktur font $\mathfrak{B}$. We say that a block $\mathfrak{B}$ is *reserved* in α if the variables defining the clause described by $\mathfrak{B}$ are assigned in α.

Algorithm 3 The extraction algorithm for Resolution

Require: A quadruple $(\varphi, \pi, 1^s, \rho)$ where π is a Res-refutation of $\mathrm{Ref}_s(\varphi)$ and ρ is a d-disabling restriction

Ensure: A satisfying assignment, an illegal derivation or a large clause

- $\pi \leftarrow \pi_{\restriction_\rho}$
- $\alpha \leftarrow \rho$
- Set D as the set of blocks disabled by ρ
- Set C as the root of π
- **while** a leaf of π is not reached **do**
  - **if** C is obtained from an illegal derivation step **then**
    - Output C with message "illegal step"
  - **else if** C is the weakening of a clause C' **then**
    - $C \leftarrow C'$
  - **else if** C is derived by resolving $A \vee y$ and $B \vee \neg y$ **then**
    - **if** y belongs to a block $\mathfrak{B}$ on layer $0 \leq \ell < n$ **then**
      - **if** $\mathfrak{B}$ is not reserved in α **then**
        - Search two unreserved blocks $\mathfrak{B}_1, \mathfrak{B}_2$ on layer $\ell + 1$
        - If the above search fails, output C with message "large clause"
        - Update α so that $\mathfrak{B}$ describes the clause $\bigvee_{j \in [\ell]} x_j$
        - Update α so that $\mathfrak{B}_1$ describes the clause $x_{\ell+1} \vee \bigvee_{j \in [\ell]} x_j$
        - Update α so that $\mathfrak{B}_2$ describes the clause $\neg x_{\ell+1} \vee \bigvee_{j \in [\ell]} x_j$
        - Update α so that $\mathfrak{B}$ is derived by resolving $\mathfrak{B}_1$ and $\mathfrak{B}_2$
      - **else if** $\mathfrak{B}$ is reserved in alpha but its children are not **then**
        - Let $\mathscr{C}$ be the clause described by $\mathfrak{B}$
        - Search two unreserved blocks $\mathfrak{B}_1, \mathfrak{B}_2$ on layer $\ell + 1$
        - If the above search fails, output C with message "large clause"
        - Update α so that $\mathfrak{B}_1$ describes the clause $x_{\ell+1} \vee \mathscr{C}$
        - Update α so that $\mathfrak{B}_2$ describes the clause $\neg x_{\ell+1} \vee \mathscr{C}$
        - Update α so that $\mathfrak{B}$ is derived by resolving $\mathfrak{B}_1$ and $\mathfrak{B}_2$
    - **else if** y belongs to a block $\mathfrak{B}$ on layer n **then**
      - **if** $\mathfrak{B}$ is not reserved in α **then**
        - Search an axiom $\mathscr{A}$ of φ such that $\bigvee_{j \in [n]} x_j$ is a weakening of it
        - If the above search fails, output $\neg \bigvee_{j \in [n]} x_j$ with message "satisfies"
        - Update α so that $\mathfrak{B}$ describes the clause $\bigvee_{j \in [n]} x_j$
        - Update α so that $\mathfrak{B}$ is derived by weakening $\mathscr{A}$
      - **else if** $\mathfrak{B}$ is reserved in alpha but its child is not **then**
        - Let $\mathscr{C}$ be the clause described by $\mathfrak{B}$
        - Search an axiom $\mathscr{A}$ of φ such that $\mathscr{C}$ is a weakening of it
        - If the above search fails, output $\neg\mathscr{C}$ with message "satisfies"
        - Update α so that $\mathfrak{B}$ is derived by weakening $\mathscr{A}$
    - Set $C \leftarrow A \vee y$ if $\alpha(y) = 0$ or $C \leftarrow B \vee \neg y$ if $\alpha(y) = 1$.
  - Clean-up α by forgetting all the variables of blocks that are not disabled, that are not mentioned in C or that are not a child of a block mentioned in C

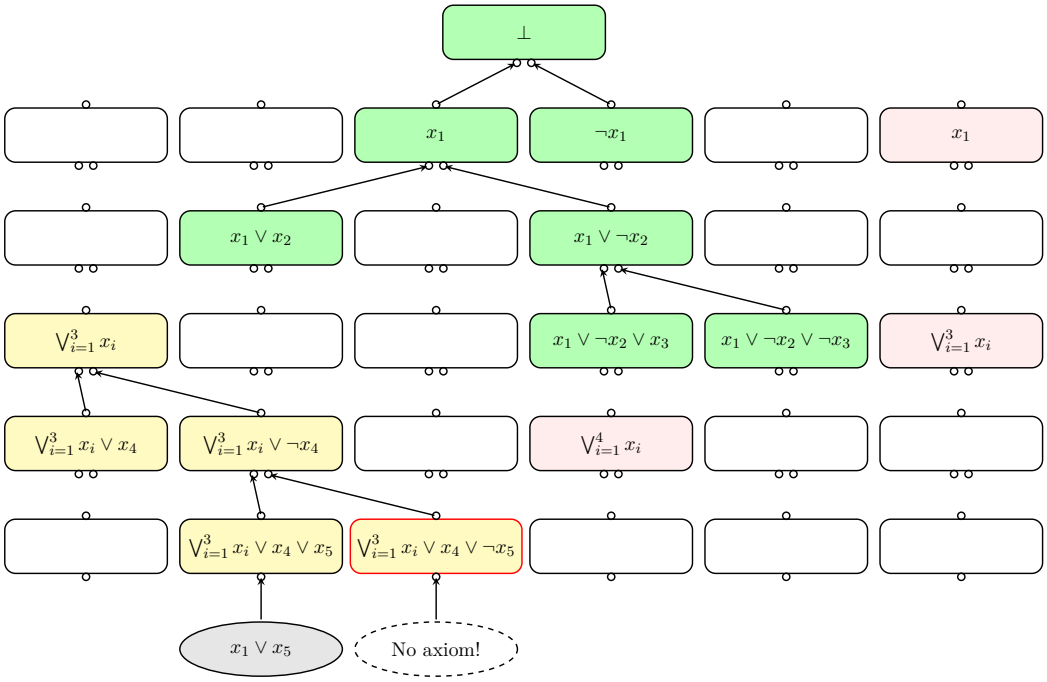


Figure 9. Snapshot of the reservations made during an execution of Algorithm 3. The **green** tree is obtained by the answers given by the Duplicator by consistently following the canonical exponential-size tree-like refutation, starting from the root. The **yellow** tree is obtained in the same way, but starting from an new block that was never mentioned. **Pink** nodes are queried out-of-context only once. **Gray** nodes are axioms of φ obtained by answering valid weakening steps. The leaf marked in **red** is not the weakening of any axiom, meaning that its negation is a satisfying assignment.

Lemma 4.12. Let π be a Resolution refutation for $\mathrm{Ref}_s(\varphi)$ and let ρ be a d-disabling restriction. Let also N be the number of variables of $\mathrm{Ref}_s(\varphi)$. Algorithm 3 $\mathcal{A}$ returns in $O(N\,|\pi|)$ exactly one of the following possible outputs:

- an incorrect derivation step in π;
- a large C in $\pi_{\restriction_\rho}$ such that $\mathrm{bwidth}(C) \geq \frac{1}{3}\lfloor (s-d-n)/n \rfloor$;
- a total satisfying assignment for φ.

Proof. The algorithm traverses a path in $\pi_{\restriction_\rho}$, where each step assigns the variables of at most 3 blocks. This amounts to a total of $O(N\,|\pi|)$ steps. To justify the correctness of the algorithm, we state the following invariant and prove that it holds throughout the traversal.

Claim 4.13. At the beginning of each iteration of $\mathcal{A}$ it holds that:

(i) the reservation α falsifies C;
(ii) a non-disabled block $\mathfrak{B}$ is reserved in α only if it is either mentioned by C or one of its parents is mentioned by C. In particular, $\mathrm{bwidth}(\alpha) - d \leq 3 \cdot \mathrm{bwidth}(C)$;
(iii) if α contains any information about a non-disabled block $\mathfrak{B}$ from a layer ℓ then α also exactly determines that $\mathfrak{B}$ contains exactly ℓ literals over $x_1, \ldots, x_\ell$;
(iv) the reservation α does not falsify any axiom of $\mathrm{Ref}_s(\varphi)$

Proof of the claim. We proceed by structural induction. The conditions are all trivially true for the initial iteration, i.e. when $C = \bot$ and $\alpha = \rho$. Consider now any other iteration of the algorithm. If C is an illegal step, the algorithm immediately terminates so nothing has to be verified. When C is obtained through a weakening or resolution step, if one between (i), (iii) and (iv) were to be false, it would also be false for the clause through which C was explored, raising a contradiction.

This leaves us with reasoning only about point (ii). Notice that $3\text{bwidth}(C)$ is the maximum number of blocks that the reservation has to remember when working with clause C: it mentions $\text{bwidth}(C)$ blocks and each of these blocks may mention at most 2 parents. Thanks to the clean-up phase, once the traversal moves from C to the next clause D of the proof, all blocks that are parents of blocks in C (equivalently, blocks that are parents of parents of blocks in D) get forgotten by the Duplicator. In particular, at most $3\text{bwidth}(D)$ are preserved in α, plus the d disabled blocks. □

We observe that the above claim implies that a leaf of π is never reached. In fact, if C were to be a leaf, conditions (i) and (iv) would raise a contradiction. Therefore, the algorithm must always halt before this happens through one of the two possible types of returns (assuming π is a correct refutation).

In the first case, the algorithm tried to reserve a block $\mathfrak{B}$ on a layer $0 \leq \ell < n$ but the search failed, outputting C. This can happen only if α has already reserved at least $\lfloor (s-d)/n \rfloor - 1$ blocks of that layer. Together with the d blocks disabled by α, we get that $\text{bwidth}(\alpha) \geq \lfloor (s-d)/n \rfloor - 1 + d$. By condition (ii) of the invariant, we conclude that $\text{bwidth}(C) \geq \frac{1}{3} \lfloor (s-d-n)/n \rfloor$.

In the second case, instead, the algorithm failed to find an axiom $\mathscr{A}$ of φ that was a weakening of the clause $\mathscr{C}$ described in α for a block $\mathfrak{B}$ on the last layer n. Since $\mathscr{C}$ contains n literals by condition (iii) of the invariant and since it is not a weakening of any axiom of φ, it must hold that $\neg\mathscr{C}$ is a total assignment for φ that satisfies it. This concludes the proof of the lemma. □

We are now ready to put the two phases together. Since the bound of the randomized restriction is slightly better, it can be used to work with $\text{FPAP}[n^2]$, while the deterministic restriction suffices only for $\text{FPAP}[n^3]$.

Theorem 4.14. $\text{FPAP}[n^3] \in \mathsf{FP}$ and $\text{FPAP}[n^2] \in \mathsf{FZPP}$.

Proof. We prove only the deterministic case; the randomized one follows the exact same proof using the improved bound. Let $\mathcal{A}$ and $\mathcal{B}$ be, respectively, the deterministic restriction algorithm Algorithm 2 and the extraction algorithm Algorithm 3. We define a new procedure $\mathcal{C}$ as follows. On input $(\varphi, \pi, 1^s)$ with $s \geq n^3$, we scan π in search for illegal derivation steps. If one is found, we immediately reject. Otherwise, we run $\mathcal{A}(\varphi, \pi, 1^s)$ to obtain the d-disabling restriction ρ with $d \leq \frac{c}{2}\sqrt{s \log |\pi|}$, for some $c > 0$. Then, we run $\mathcal{B}(\varphi, \pi, 1^s, \rho)$ to obtain either a satisfying assignment β for φ or a clause C with $\text{bwidth}(C) \geq \frac{1}{3} \lfloor (s-d-n)/n \rfloor$. In the first case, we simply output β. In the second case, instead, we know that:

$$c\sqrt{s \log |\pi|} \geq \text{bwidth}(\pi_\rho) \geq \text{bwidth}(C) \geq \frac{1}{3} \lfloor (s-d-n)/n \rfloor \tag{4.2}$$

since $d \leq \frac{c}{2}\sqrt{s \log |\pi|}$ and $s \geq n^3$, from the above inequality we obtain that $|\pi| \geq 2^{\varepsilon s/n^2} = 2^{\varepsilon n}$ for some $\varepsilon > 0$. Therefore, since the input is so large, $\mathcal{C}$ proceeds to find a satisfying assignment through bruteforce. In both cases, the runtime is $\text{poly}(n, |\pi|, s)$. □

We observe that the size bound obtained in the above proof is $|\pi| \geq 2^{\Omega(s/n^2)}$. This is achieved by bypassing the reduction to rPHP, just as in the original argument by Atserias and Müller [AM20]. As a consequence, we also get that Resolution is $(3, \varepsilon)$-analyzable, improving the bound of Proposition 4.4.

4.3 Extending analyzability to algebraic systems

This section contains the first real new contribution of this work, other than the sharpening and the reproving of bounds in Chapter 3. In particular, we shall prove that analyzability can be extended all the way up to Polynomial Calculus with Resolution. To achieve this, we prove that the lower bound argument for Resolutions transfers to PCR by adapting decision tree reductions to *algebraic reductions* [RGN+21].

Definition 4.15 (Algebraic reduction)**.** Let φ and ψ be, respectively, CNF formulas over the variables $x_1, \ldots, x_n$ and $y_1, \ldots, y_t$. A degree-d algebraic reduction between φ and ψ, written as $\varphi \leq_d^{alg} \psi$ is a function $r : \{0,1\}^n \to \{0,1\}^t$ such that:

- Each output bit $r_i : \{0,1\}^n \to \{0,1\}$ is computed by an independent degree-d polynomial.
- For each polynomial g of ψ, viewed as a function $g : \{0,1\}^t \to \{0,1\}$, the multilinearization of the composition $g \circ r$ has a Nullstellensatz derivation of degree $d \cdot \deg(g)$ from the polynomials of φ

To better understand why this translation comes in handy, we observe that any function computable by a depth-d decision tree can also be computed through a degree-d polynomial [Juk12; RGN+21]. Let $\mathcal{T}$ be a decision tree. For every leaf $\ell \in \text{Leaves}(\mathcal{T})$ of the decision tree, let D_ℓ be the term made of all the literals corresponding to the query outcomes on the path from the root to the leaf. Then, $\neg D_\ell$ is a clause. Let q_ℓ be the polynomial obtained through the standard mapping between clauses and polynomials. For each assignment α, it holds that $\alpha(D_\ell) = 1$ if and only if $\alpha(q_\ell) = 0$ under the constraints $\{x_i^2 - x_i = 0\}_{i \in [n]}$ imposed by NS, PC and PCR proofs. Moreover, q_ℓ is a polynomial of degree at most d.

By subtracting all the polynomials of all leaves of $\mathcal{T}$ outputting true from 1, we obtain a polynomial $q_\mathcal{T}$ that evaluates to 1 if and only if $\mathcal{T}$ evaluates to 0, preserving the flipped nature of truth values in the algebraic context.

$$q_\mathcal{T} := \sum_{\substack{\ell \in \text{Leaves}(\mathcal{T}): \\ \text{out}(\ell)=1}} (q_\ell) \tag{4.3}$$

As in the Resolution case, we will be interested in working with *block-aware* measures. To do this, we define *block-degree* as the degree measure analogous to block-width (recall that width gets translated to degree in algebraic systems).

Definition 4.16 (Block-aware algebraic reduction)**.** Let r be a degree-d algebraic reduction witnessing $\varphi \leq_d^{alg} \psi$. Suppose that the variables $y_1, \dots y_s$ of ψ are partitioned into blocks. We say that the reduction $r : \{0,1\}^n \to \{0,1\}^t$ is block-aware when:

- For each block $B \subseteq [t]$ and each $T \subseteq B$ the polynomial $r_T := \text{mult}\,(\prod_{i \in T})\, r_i$ has degree at most d
- For each polynomial g of ψ, viewed as a function $g : \{0,1\}^t \to \{0,1\}$, the multilinearization of the composition $g \circ r$ has a Nullstellensatz derivation of degree $d \cdot \text{bdeg}(g)$ from the polynomials of φ

Proposition 4.17. If $\varphi \leq_d^{dt} \psi$ then $\varphi \leq_d^{alg} \psi$. The same also holds for block-aware reductions.

Proof. We directly address block-aware reductions. Let $f : \{0,1\}^n \to \{0,1\}^t$ be the block-aware reducing function from φ to ψ computed by depth-d decision trees. For each block $B \subseteq [t]$ of ψ we know that there is a depth-d decision tree $\mathcal{T}_B$ computing $f_B(x) = (f_i(x))_{i \in [B]}$ simultaneously. By the above discussion, there is a polynomial q_B that computes f_B.

Now consider any axiom $C \in \psi$. Recall that the decision tree reduction guarantees that every clause in $C \circ f$ is a weakening of an axiom of φ. Thus, we may write $C \circ r = \bigwedge_{i \in I}(A_i \vee B_i)$, with $A_i \in \varphi$, where at most one clause $(A_i \vee B_i)$ evaluates to false. Therefore, the multilinearization of $\sum_{i \in I} p_{A_i} \cdot p_{B_i}$ computes $p_C \circ f$, where p_{A_i}, p_{B_i} are the degree-$(d \cdot \text{bdeg}(p_C))$ polynomials representations of A_i, B_i and $\text{bdeg}(p_C) = \text{bwidth}(C)$. □

Furthermore, the adaptation from decision tree reductions to algebraic reductions is also *lossless* on lower bounds, meaning that there is no additional factor gained due to a switch to the algebraic context.

Proposition 4.18. If $\varphi \leq_d^{alg} \psi$ via a block-aware reduction then $\deg_{\text{Q}}(\varphi) \leq d \cdot \text{bdeg}_{\text{Q}}(\psi)$, where $Q \in \{\text{NS}, \text{PC}, \text{PCR}\}$.

Proof. We first prove the result for Nullstellensatz. Let r be the function computing the reduction and let $\psi = \{g_1, \dots, g_k\}$ be the axiom polynomials of ψ. We observe that for each NS-refutation $\pi = \{p_1, \dots, p_k\}$ for ψ, it holds that:

$$\sum_{j \in [k]} (p_j \circ r)(g_j \circ r) = \left(\sum_{j \in [k]} p_j g_j \right) \circ r = 1 \circ r = 1 \tag{4.4}$$

since π is a refutation and 1 contains no variables to compose with. Moreover, by definition of algebraic reduction, each polynomial $g_j \circ r$ can be derived from the

axioms of φ with degree $d \cdot \mathrm{bdeg}(g_i)$. Therefore, there is an equivalent NS-refutation for φ of block-degree at most $d \cdot \mathrm{bdeg}(\pi)$.

Next, we prove the result for PC and PCR. We observe that the given argument does not rely on the presence of twin variables, hence it works for both proof systems. Let π be a PC-refutation for ψ. We proceed by structural induction on π to obtain a PC-refutation π' of φ. In particular, whenever p is derived in π, we will derive $p \circ r$.

Again, for each axiom g_i of ψ we know that $g_i \circ r$ can be derived in Nullstellensatz with degree at most $d \cdot \mathrm{bdeg}(\pi)$, hence also in PC and PCR by simulation (the simulation can be made so that it preserves degree). For each polynomial $p + q$ derived by summing p and q, by summing $p \circ r$ and $q \circ r$ we obtain $(p + q) \circ r$. For each $y_i p$ derived through multiplication, instead, we can derive $r_i(p \circ r) = y_i p \circ r$ from $p \circ r$ through at most d multiplications and a polynomial number of sums (notice that degree is preserved, hence the number of sums does not matter). Since the two last operations blow up the block-degree of the original proof only by a factor of d (due to the composition), we conclude that $\deg(\pi') \leq d \cdot \mathrm{bdeg}(\pi)$. □

After proving that the tools translate in the algebraic context in a natural way, the argument proceeds similarly as that of Section 3.4.6. In particular, one can show that $\deg(\mathrm{rPHP}^{2m}_m) = \Omega(m/\log m)$ [RGN+21; Raz98; PT19] to obtain a PCR block-degree lower bound of $\Omega\left(\frac{s}{n^2 \log^2(s/n)}\right)$ for $\mathrm{Ref}_s(\varphi)$ when φ is unsatisfiable. Notice that the bound is similar to the one obtained for block-width, up to a logarithmic factor. To get the final exponential size lower bound, we adapt to PCR the argument shown in Lemma 3.28, inspired by [IPS99; GL10b].

Lemma 4.19. Let φ be an unsatisfiable CNF formula with n variables and m clauses. Then, for all $s \in \mathbb{N}$ with $s \geq n^2$, it holds that $\mathrm{bdeg}_{\mathrm{PCR}}(\mathrm{Ref}_{s/2}(\varphi)) \leq O(\log \mathrm{size}_{\mathrm{PCR}}(\mathrm{Ref}_s(\varphi)))$

Proof. We proceed similarly to what is done for Resolution. For convenience, we assume that s is odd. We define a partial assignment ρ obtained through the same randomized procedure:

1. Initialize ρ as the empty mapping;
2. Pair all the first $s - 1$ blocks into $(s - 1)/2$ pairs;
3. For each pair (B_i, B_{i+1}), flip a balanced coin c_i. If $c_i = 0$ then set $\rho \leftarrow \rho \cup \{\mathrm{enable}^{B_i} \mapsto 1, \overline{\mathrm{enable}^{B_i}} \mapsto 0\}$. Otherwise, set $\rho \leftarrow \rho \cup \{\mathrm{enable}^{B_{i+1}} \mapsto 1, \overline{\mathrm{enable}^{B_{i+1}}} \mapsto 0\}$;
4. After disabling one block of each pair, we set all the remaining blocks as enabled. Moreover, for each disabled block we randomly assign all its remaining variables.

The procedure has been adapted in order to manage the twin variable axioms: for each variable that gets disabled we have to enforce that its twin variable gets set to the opposite value. This is necessary in order to avoid contradicting the axioms $\{1 - \mathrm{enable}^{B_i} - \overline{\mathrm{enable}^{B_i}}\}_{i \in [s]}$. Other than this change, the proof follows the same reasoning as the original one.

Let π be a PCR refutation of $\mathrm{Ref}_s(\varphi)$. For each block B_i, with $i \in [s - 1]$, consider any literal ℓ mentioning B_i. We argue that the same reasoning applied to

the case of Resolution transfers naturally to PCR by swapping zeros and ones (i.e. preserving the truth value swap of the algebraic context). In particular, we have that $\Pr_{\rho}[\rho(\ell) = 0] \geq 1/4$ always holds.

Consider now any monomial m in π of block-degree w (value to be determined later). The probability that the whole monomial does not get killed in the restriction by setting any of its literals to 0 is bounded by $\Pr_{\rho}[m_{\restriction_\rho} \neq 0] \leq (3/4)^{w/2}$. Therefore, we have that:

$$\Pr_{\rho}[\exists m \in \pi \text{ with } \mathrm{bdeg}(m_{\restriction_\rho}) \geq w] \leq |\pi| \cdot \left(\frac{3}{4}\right)^{w/2} \tag{4.5}$$

By setting $w \geq (2(\log|\pi| - \log(1-p)))(\log(4/3))$ with $0 < p < 1$, the above probability is lower than 1, thus there must be at least one partial assignment ρ^* for which no monomial of block-degree greater than w survives. The lemma follows by choosing π as a refutation of $\mathrm{Ref}_s(\varphi)$ with the smallest size possible. □

Corollary 4.20. Let φ be an unsatisfiable formula with n variables. Then, every PCR refutation of $\mathrm{Ref}_s(\varphi)$ has size $2^{\Omega(s/(n^2 \log^2(s/n)))}$

Theorem 4.21. If $Q \leq_p \mathrm{PCR}$ then $\mathrm{PAP}_Q[n^3 \log^2 n] \in \mathsf{P}$.

Proof. Follows for PCR from Corollary 4.20 by using the length test (Proposition 4.4). For all $Q \leq_p$ PCR it follows from downward closure of analyzability. □

Since Res $\leq_p$ PCR, the above result gives a simplified proof of the non-automatability of Polynomial Calculus with Resolution under the $\mathsf{P} \neq \mathsf{NP}$ assumption. However, we remark that the same does not hold for Nullstellensatz and standard Polynomial Calculus since they do not p-simulate Resolution. This could be fixed by proving that Pudlák's upper bound also holds in NS and PC. In [RGN+21], it is argued that this may be doable in PC using a clever argument, but it seems out of reach for NS.

Corollary 4.22. Automating PCR is NP-hard.

Even though extending analyzability to PCR uses ideas similar to the ones for Resolution, extending search-analyzability to PCR is not so straightforward. In particular, we observe that Algorithm 3 walks down a single path of falsified clauses, guided by a Duplicator strategy that characterizes width. In PCR, there are three issues that must be overcome: (1) lines are polynomials, thus there is no notion of a "falsified line" along one path; (2) degree lower bounds are proved through Razborov's [Raz98] pseudo-reduction operators (R-operators), which act on the whole ideal rather than along a path; (3) a polynomial of degree d may mention more than d blocks at the same time through different monomials all of degree d.

Therefore, an extraction algorithm for PCR needs a constructive version of the R-operator argument to mimic what is achieved through the Spoiler-Duplicator game for Resolution. The randomized and deterministic restriction phases, instead, can be naturally translated to the PCR context. Another way to prove the search-analyzability of PCR is by a completely different extraction argument that analyzes

the proof from a global perspective instead of a local one, but this requires finding a particularly complex R-operator.

4.4 Is analyzability linked to feasible interpolation?

Due to the tight relations between (weak) automatability and feasible interpolation (Theorem 3.11, Theorem 3.13, Theorem 3.14) and between automatability and analyzability (Proposition 4.3), investigating whether analyzability and feasible interpolation are connected to each other follows as a natural question. Moreover, both analyzability and feasible interpolation are *extraction properties*: the former extracts information from proofs of $\mathrm{Ref}_s(\varphi)$, while the latter can extract information from unsatisfiable split formulas $A(x,y) \wedge B(x,z)$. In this section, we investigate both directions.

First of all, we start by observing that analyzability and feasible interpolation indeed seem to be related to each other. In particular, all proof systems that are known to be analyzable, i.e. those below PCR (Theorem 4.21), also have feasible interpolation [Kra97; Hak20]. However, not all proof systems that possess feasible interpolation are known to be analyzable (Cutting Planes [Pud97], Sum-of-Squares [Hak20], ...). Nonetheless, all proof systems that are known to not be analyzable (AC^0-Frege, Frege and Extended Frege [AAR+25; ARK+26]) do not possess feasible interpolation [Kra97; BPR97; BPR00]. For Res(k), neither analyzability nor feasible interpolation has been proven. This analysis raises two main questions: (i) does feasible interpolation imply analyzability? (ii) does the converse hold?

Proving the first implication is trivially hard: by Theorem 3.11 and Proposition 4.3, if the implication were to hold we would immediately obtain that Resolution is not weakly automatable, unless $\mathsf{P} = \mathsf{NP}$. Therefore, if it were to be true, proving that feasible interpolation unconditionally implies analyzability is at least as hard as proving that weakly automating Resolution is impossible under the $\mathsf{P} \neq \mathsf{NP}$ assumption. Of course, this does not rule out the possibility of the implication being unconditionally true, but it shows that is far from being provable with modern proof complexity results.

Proposition 4.23. Under the $\mathsf{P} \neq \mathsf{NP}$ assumption, if every proof system $Q \geq$ Res closed under restriction that has feasible interpolation is also analyzable then Resolution is not weakly automatable.

The answer to the second implication is also negative, but it requires more work. We start by defining a concept closely related to feasible interpolation and weak automatability: *disjoint* NP*-pairs*.

Definition 4.24 (Disjoint NP-pair)**.** The pair (U, V) is said to be a disjoint NP-pair when $U, V \in \mathsf{NP}$ and $U \cap V = \varnothing$. The pair (U, V) is said to be separable in time t if there is an algorithm $\mathcal{S}$ with runtime t such that $U \subseteq L(\mathcal{S})$ and $V \subseteq \overline{L(\mathcal{S})}$, where $L(\mathcal{S})$ is the set of inputs accepted by $\mathcal{S}$.

When t is a polynomial function, we say that the pair is simply separable. The concept of polynomial time reduction is extended to disjoint NP-pairs in a natural

	Res	NS	PC	PCR	CP	Res(k)	AC^0-F	F	EF
Analyzable	✓	✓	✓	✓	?	?	×	×	×
Feas. Int.	✓	✓	✓	✓	✓	?	×	×	×

Table 4.1. Analyzability and feasible interpolation in common proof systems. All negative results are conditional.

way. We say that (U, V) reduces to (U', V'), written as $(U, V) \leq_p (U', V')$, when there is a many-to-one reduction f such that $f(U) \subseteq U'$ and $f(V) \subseteq V'$.

Pudlák [Pud03] originally proved his upper bound on refutation statements for satisfiable formulas while working with the canonical pair of Resolution. The *canonical pair* of a proof system Q [Raz94] is the disjoint NP-pair $\mathrm{Can}(Q) := (\mathrm{SAT}^*, \mathrm{REF}^Q)$, where for any $(\varphi, 1^t)$, with φ being a CNF formula, it holds that $(\varphi, 1^t) \in \mathrm{SAT}^*$ iff φ is satisfiable, while $(\varphi, 1^t) \in \mathrm{REF}^Q$ iff $\mathrm{Ref}^Q_t(\varphi)$ is satisfiable. In particular, Pudlák showed that Can(Res) is *symmetrical*, meaning that $(\mathrm{SAT}^*, \mathrm{REF}^{\mathrm{Res}}) \leq_p (\mathrm{REF}^{\mathrm{Res}}, \mathrm{SAT}^*)$. Furthermore, he also proved that separating the canonical pair of proof system is equivalent to weakly automating it.

Proposition 4.25. Let Q be a proof system. Then, $\mathrm{Can}(Q)$ is separable if and only if Q is weakly automatable.

Proof. Let $\mathcal{S}$ be an algorithm separating $\mathrm{Can}(Q)$. We define a new proof system P as follows: 1^t is a P-refutation of φ when $\mathcal{S}(\varphi, 1^t)$ rejects. Notice that P is a valid Cook-Reckhow proof system: if φ is satisfiable then P has no refutation, while if φ is unsatisfiable then $1^{\mathrm{Size}_Q(\varphi)}$ is a valid P-refutation. Moreover, P can be easily automated by testing $\mathcal{S}(\varphi, 1^{2^t})$ with $t = 1, 2, 4, \ldots$ until $t = \lceil \log \mathrm{size}_Q(\varphi) \rceil$ is reached. The p-simulation of Q from P follows by definition.

Vice versa, suppose that Q is weakly automatable and let $\mathcal{A}$ be an algorithm automating $P \geq_p Q$. Then, we can build a separator $\mathcal{B}$ that on input $(\varphi, 1^t)$ runs $\mathcal{A}(\varphi)$ for at most t steps, accepting if and only if the computation returned a refutation before the time bound. □

We shall use the above equivalence to prove that if analyzability were to imply feasible interpolation then every proof system would be weakly automatable. In particular, we prove something much stronger: under those assumptions, every NP-pair becomes separable. In particular, starting from an arbitrary disjoint NP-pair, we build an analyzable system that fails to interpolate unless the pair is separable. The idea is to give the system a single superpower that only works in a private corner of the variable space, making sure that refutation statements never enter that corner.

Let (U, V) be a disjoint NP-pair. By the Cook-Levin theorem [Coo71; Lev73; AB09], we know that there are two polynomial-time constructible CNFs $A_n(x, y)$ and $B_n(x, z)$, with $|x| = n$, such that for every $a \in \{0,1\}^n$ it holds that $a \in U$ iff $A_n(a, \cdot)$ is satisfiable and $a \in V$ iff $B_n(a, \cdot)$ is satisfiable. Since $U \cap V = \emptyset$, the formula $F_n := A_n \wedge B_n$ is a split unsatisfiable CNF formula and this also holds

for every restriction $F_{n\restriction_\rho}$. of *reserved* variables $r_{n,1}, \ldots, r_{n,k}$ that no refutational formula uses and write every F_n over reserved variables only.

We define a proof system $Q_{U,V}$ extending Resolution as follows. This construction follows the classical idea of tailoring a proof system to a disjoint NP-pair [Raz94; Bey06; GS84]. The new ingredient is a *reserved alphabet*, which lets the system keep the Ref-hardness of Resolution while making the pair (U, V) trivially refutable.

A $(Q_{U,V})$-refutation is a sequence of clauses $\pi = (C_1, \ldots, C_t)$ where each clause is either an axiom or is obtained through an application of the resolution rule, the weakening rule or a new rule referred to as the *F-rule*. Whenever the disjunction $C \vee D$ has been derived for every clause C of some restriction of F_n under some partial assignment ρ, the F-rule allows the system to derive D.

$$\frac{(C \vee D \quad \forall C \in F_{n\restriction_\rho})}{D} \qquad (F\text{-rule})$$

Each use of the F-rule is represented by a triple (n, ρ, D), where n is written in binary, ρ is written as a string over $\{0, 1, *\}^{|\mathrm{Vars}(F_n)|}$. We argue that this rule is sound: since $F_{n\restriction_\rho}$ is unsatisfiable, $\bigwedge_C (C \vee D) \equiv (F_{n\restriction_\rho}) \vee D$ implies D. Checking an application of the F-rule means computing $F_{n\restriction_\rho}$ and looking up its clauses among earlier lines. Since ρ is encoded with $|\mathrm{Vars}(F_n)| \geq n$ symbols, computing F_n (time poly(n)) is polynomial in the length of the step, so $Q_{U,V}$ is a valid Cook-Reckhow system. Moreover, because ρ has a fixed-length encoding, restricting each application does not change the size of its representation.

Lemma 4.26. For every disjoint NP-pair (U, V), the proof system $Q_{U,V}$:

(i) p-simulates Resolution;
(ii) is closed under restriction;
(iii) is $(3, \varepsilon)$-Ref-hard for some $\varepsilon > 0$.

Proof. p-simulation follows by definition of $Q_{U,V}$ since it has the Resolution rule and the weakening rule. Consider an application of the F-rule represented by (n, ρ, D) and a restriction τ. Let τ' be the sub-assignment of τ on the variables of F_n that ρ leaves free. For every $C \in F_{n\restriction_\rho}$, the variables of C are among those free variables, thus $C_{\restriction_\tau} = C_{\restriction_{\tau'}}$. Hence, the restricted premises $(C \vee D)_{\restriction_\tau}$ are exactly the clauses $C' \vee D_{\restriction_\tau}$ for $C' \in F_{n\restriction_{(\rho \cup \tau')}}$, together with some premises that became true.

Notice that this is a legal application of the F-rule because the premises indexed by clauses satisfied by τ' are exactly those dropped from $F_{n\restriction_{(\rho\cup\tau')}}$. In particular, notice that τ may falsify a whole clause C of $F_{n\restriction_{(\rho\cup\tau')}}$, meaning that $\bot \in F_{n\restriction_{(\rho\cup\tau')}}$, thus the corresponding premise is $D_{\restriction_\tau}$ itself and the step is a repetition. Moreover, this step is represented by $(n, \rho \cup \tau', D_{\restriction_\tau})$, which has the same fixed length thus the size does not grow. If τ satisfies D, the conclusion is true and the step disappears. This concludes that the system is closed under restrictions.

We now prove the Ref-hardness of $Q_{U,V}$. Let π be a $Q_{U,V}$-refutation of $\mathrm{Ref}_s(\varphi)$ and let β set every reserved variable occurring in π to 0. Consider any application of the F-rule in π. All its premises $C \vee D$ are lines of π, hence every variable of $F_{n\restriction_\rho}$ occurs in π and is assigned by β. Since $F_{n\restriction_\rho}$ is unsatisfiable, β must falsify at least

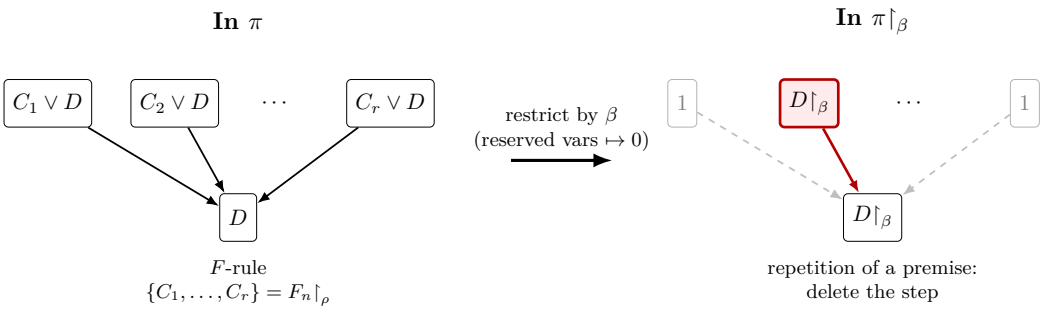


Figure 10. The idea behind Lemma 4.26. $F_n\restriction_\rho$ is unsatisfiable and all its variables are reserved, so β falsifies some clause. That premise becomes $D\restriction_\beta$, while the other premises become true. Axioms of $\mathrm{Ref}_s(\varphi)$ use only ordinary variables, so $\mathrm{Ref}_s(\varphi)\restriction_\beta = \mathrm{Ref}_s(\varphi)$.

one clause C_0. We notice that this is where the reserved variables are required: to collapse an application of the F-rule we must assign the variables of $F_n\restriction_\rho$. If some of the variables of $\mathrm{Ref}_s(\varphi)$ were to be part of the reserved set, β would also restrict $\mathrm{Ref}_s(\varphi)$ itself.

Now, the premise $C_0 \vee D$ restricts to $D_{\restriction_\beta}$, which is exactly the restricted conclusion, so the step becomes a repetition. Moreover, the axioms of $\mathrm{Ref}_s(\varphi)$ contain no reserved variables, thus they are untouched. Therefore $\pi_{\restriction_\beta}$ is a standard Resolution refutation of $\mathrm{Ref}_s(\varphi)$ that never gains literals and where each collapsed F-step can be deleted, because its conclusion $D_{\restriction_\beta}$ already occurs as the premise $C_0 \vee D_{\restriction_\beta} = D_{\restriction_\beta}$. Therefore, we get that $\mathrm{size}_{\mathrm{Res}}(\mathrm{Ref}_s(\varphi)) \leq |\pi|$. By $(3, \varepsilon)$-Ref-hardness of Resolution, we conclude that $Q_{U,V}$ must also be $(3, \varepsilon)$-Ref-hard. □

Theorem 4.27. There is at least one proof system that p-simulates Resolution, is closed under restrictions and is analyzable but does not possess feasible interpolation, unless every disjoint NP-pair is separable.

Proof. Assume that the premise is false and fix a disjoint NP-pair (U, V). Then, by the previous lemma, $Q_{U,V}$ must have an interpolating algorithm $\mathcal{I}$. Consider the restriction $\rho = \emptyset$ and the clause $D = \bot$. The F-rule can be used to derive $\bot$ in polynomial time directly from the clauses of $A_n \wedge B_n$, obtaining a refutation π_n of size $O(|F_n|)$, constructible in polynomial time. By definition, $\mathcal{I}(\pi_n, a)$ must output 1 when $a \in U$ (because then $A_n(a, \cdot)$ is satisfiable and 0 is forbidden) and 0 when $a \in V$. This concludes that $\mathcal{I}$ successfully separates (U, V). □

To further justify the negativeness of the result, we discuss how unlikely the premise of Theorem 4.27 is outside of the automatability context. First of all, we observe that the separability of every disjoint NP-pair trivially implies that $\mathsf{P} = \mathsf{NP} \cap \mathsf{coNP}$: for any $L \in \mathsf{NP} \cap \mathsf{coNP}$ a separator for $(L, \overline{L})$ solves L in polynomial time. More interestingly, the general separability property implies that $\mathsf{P} = \mathsf{UP}$, where UP is the class of NP languages that have exactly one witness per input. Notice that the existence of *one-way permutations* (conjectured to be true) implies that $\mathsf{P} \neq \mathsf{UP}$ [GS84]. We use an argument inspired by Grollmann and Selman

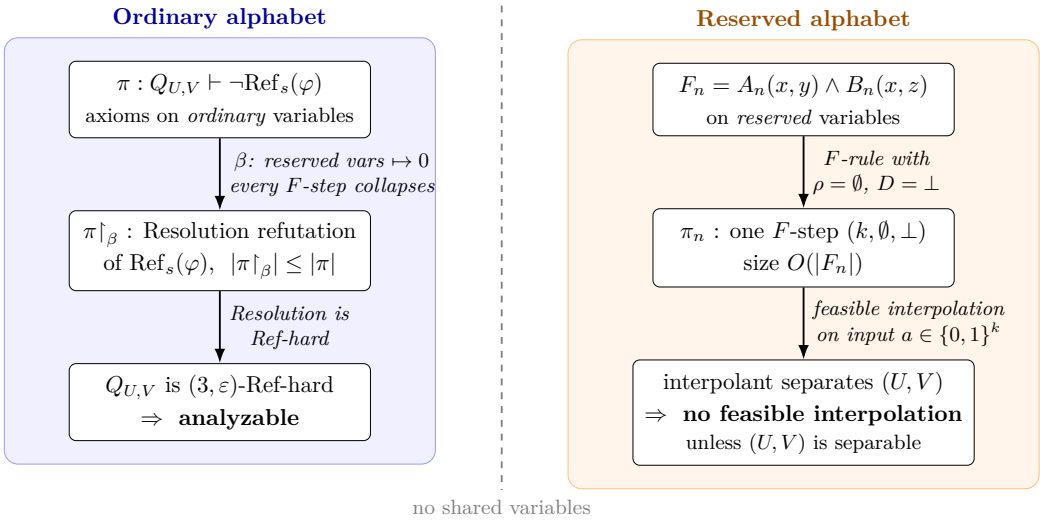


Figure 11. Summary of Lemma 4.26 and Theorem 4.27.

[GS84] and Glasser, Selman, Sengupta, et al. [GSS+04]. Given $L \in \mathsf{UP}$, where $x \in L$ has a unique witness w_x, consider the disjoint NP-pair $(L_{\le y}, L_{>y})$, where $L_{\le y} := \{(x,y) : x \in L,\ w_x \le y\}$ and $L_{>y} := \{(x,y) : x \in L,\ w_x > y\}$. A separator $\mathcal{S}$ for this pair can binary search over y to find a candidate witness, which is then verified.

This concludes that analyzability does not unconditionally imply feasible interpolation for systems above Resolution, unless (i) every disjoint NP-pair is separable, (ii) every proof system is weakly automatable, (iii) $\mathsf{P} = \mathsf{NP} \cap \mathsf{coNP}$, (iv) $\mathsf{P} = \mathsf{UP}$, (v) one-way permutations do not exist.

Conclusions

We proved that Polynomial Calculus with Resolution is analyzable, and hence so are Nullstellensatz and Polynomial Calculus, by lifting the block-width argument to block-degree. As a corollary, we obtained a new proof that automating PCR is NP-hard. We then compared analyzability with feasible interpolation. Proving that interpolation implies analyzability would show that Resolution is not weakly automatable unless P = NP, while for every disjoint NP-pair there is an analyzable system, simulating Resolution and closed under restrictions, that has feasible interpolation only if the pair is polynomial-time separable. Any connection between the two notions must therefore use specific properties of natural proof systems. Several questions that further research may explore remain open.

Search-analyzability of PCR. The most immediate question left open is whether a satisfying assignment of φ can be extracted in polynomial time from a short PCR proof of $\neg\mathrm{Ref}_s(\varphi)$.

Pudlák's upper bound in algebraic systems. Our proof that automating PCR is NP-hard uses Pudlák's upper bound for Resolution, which PCR inherits because it simulates Resolution. Nullstellensatz and Polynomial Calculus do not simulate Resolution efficiently, so the same argument does not give the NP-hardness of automating them, even though both are analyzable by our result. It is open whether $\mathrm{Ref}_s(\varphi)$, or a suitable algebraic variant of it, has short Polynomial Calculus or Nullstellensatz refutations when φ is satisfiable. A positive answer, combined with our analyzability results, would give a simplified proof that automating these systems is NP-hard.

Analyzability of Cutting Planes, Res(k) and bounded-depth Frege. Every analyzability result needs a lower bound for $\mathrm{Ref}_s(\varphi)$ in the analyzed system. For Res(k), the block-width technique may extend through switching lemmas for k-DNFs. For Cutting Planes, the usual techniques are lifting from communication complexity and monotone interpolation, and it is unclear whether either adapts to $\mathrm{Ref}_s(\varphi)$. For bounded-depth Frege, switching-lemma lower bounds for the pigeonhole principle exist, but the retraction version would need a separate analysis.

A natural separation of analyzability and interpolation. We showed that analyzability and feasible interpolation are different in general, but the separating system is artificial. Is there a natural analyzable proof system without feasible interpolation, or a natural system with feasible interpolation that is not analyzable? Cutting Planes is the natural candidate for the second question, since it has feasible interpolation through monotone real circuits and its analyzability is open. A proof

that Cutting Planes is not analyzable under a standard assumption would give the first natural separation. A proof that it is analyzable would extend the known coincidence of the two properties to a system that is incomparable with PCR.

Does interpolation imply analyzability on restricted systems?. In the last chapter we ruled out a general proof that feasible interpolation implies analyzability, unless the weak automatability of Resolution is settled. It does not rule out such a proof for restricted classes of systems. A natural candidate is the class of systems that extend Resolution, are closed under restrictions and in which lines are evaluated by a bounded number of Boolean functions, such as Res(k). A result of this kind would explain why every analyzable system known before this thesis also has feasible interpolation.

Weak automatability of Resolution. The relationship between analyzability and interpolation is tied to the weak automatability of Resolution, one of the central open problems of the area. Weak automatability asks for an algorithm that finds a proof in some possibly stronger system, so the meta-complexity method does not apply directly: the stronger system may refute $\mathrm{Ref}_s(\varphi)$ efficiently even when φ is unsatisfiable. Any progress on analyzability for systems above Resolution, such as Res(2), therefore bears directly on this problem.

Acknowledgements

English version

After two theses, I have learned the hard way that writing exhaustive acknowledgements is like searching for a proof: it exists somewhere, but nobody promises it can be found efficiently. In these pages I have defined, assumed, stated lemmas and, probably more than once, contradicted myself. Yet this is the only section in which I can state something without a single quantifier: I did not get here alone.

To my advisors, Nicola and Massimo. Thank you for your patience, your rigor, and the rare ability to say "this step is wrong" or even "I can't make sense of any of this". You taught me that a good question is worth more than a quick answer, and that a good answer is worth checking twice. You are not only mentors but also a source of aspiration. Whatever is correct in these pages I owe largely to you. Whatever is not is, as tradition demands, left as an exercise for the reader.

To the thesis reviewers and the graduation committee. Thank you for giving your time to a text whose central message is that finding short proofs is hard, and for then asking you to read several that are extremely long. I hope the effort feels worthwhile, and I promise that the next proof will be shorter, although I cannot prove it without breaking some assumption.

To my friends and department colleagues, Sara, Alessio, Armando, Ivan and Ionut, for the whiteboards full of lemmas that did not survive, for the coffee that did, and for the conversations in which "it is obviously true" turned into "it is obviously false" within ten minutes. A thesis is a solitary object, but the years of study behind it were not.

To the people who, unfortunately, have to deal with me outside academia. Thank you for never asking what a proof system is and, if only for that, for preserving my sanity. For the evenings and the discussions in which the only open problem was where to go out. And yes, I can finally answer the question "so, when do you graduate?" with a date. Thank you to Sara, Gaia and Clara who, unfortunately for them, have been with me forever, and to Lorenzo, Micol, Giulia and Valentina for putting up with me every day, provided that I actually get in touch, of course.

To my family. Thank you for your support, for your sacrifices, and for believing in me long before there was anything to believe in. You gave me the freedom to choose and the certainty of always having a place to come back to. I hope this milestone, like the future ones, feels like yours too, because it is: every line of this thesis has a piece of your work behind it, and no result of mine will ever be as

important as this one.

To Camilla. Our relationship developed just like the search for a result: the outcome was a foregone conclusion, but it was necessary to detour and explore other paths, some of them difficult, in order to obtain it. You listened to explanations you were under no obligation to follow, celebrated results you could not decipher, and consoled failures you could not verify. I always tell you this and I claim it here: you are the only person I consider above myself. You are out of this world; indeed, I will not deny that the green dress you wore the first time we met suggested you were an alien. Luckily, my obsession is the study of complex, intractable and incomprehensible things.

To myself, for not abandoning the project during the weeks when the main result seemed impossible to reach, and for finally understanding that a deadline is a theorem you only believe once you have proved it.

Finally, to everyone I forgot to name: the omission is a limit of my memory, but not of my gratitude. To ensure correctness I had to sacrifice completeness, but I cannot even prove my own correctness. Damned Gödel and his two stupid theorems.

Simone Bianco, Rome, October 2026

Versione italiana

Dopo ben due tesi ho imparato, a mie spese, che scrivere dei ringraziamenti esaustivi è come cercare una dimostrazione: esiste da qualche parte, ma nessuno promette che si possa trovare in modo efficiente. In queste pagine ho definito, assunto, enunciato lemmi e, probabilmente più di una volta, contraddetto me stesso. Tuttavia, questa è l'unica sezione in cui posso affermare qualcosa senza un solo quantificatore: non sono arrivato fin qui da solo.

Ai miei relatori, Nicola e Massimo. Grazie per la pazienza, il rigore e la rara capacità di dire "questo passaggio è sbagliato" o persino "non si capisce niente". Mi avete insegnato che una buona domanda vale più di una risposta rapida, e che una buona risposta vale la pena di essere controllata due volte. Non siete solo mentori ma anche fonte di aspirazione. Tutto ciò che è corretto in queste pagine lo devo in larga parte a voi. Tutto ciò che non lo è, come vuole la tradizione, è lasciato per esercizio al lettore.

Ai revisori della tesi e alla commissione di laurea. Grazie per aver dedicato il vostro tempo a un testo il cui messaggio centrale è che trovare dimostrazioni brevi è difficile, e per avervi poi chiesto di leggerne svariate che sono lunghissime. Spero che lo sforzo vi sembri valso la pena e prometto che la prossima dimostrazione sarà più breve, anche se non sono in grado di dimostrarlo a meno di rompere qualche assunzione.

Ai miei amici e colleghi di dipartimento, Sara, Alessio, Armando, Ivan e Ionut, per le lavagne piene di lemmi che non sono sopravvissuti, per il caffè che invece ce l'ha fatta e per le conversazioni in cui la frase "è ovviamente vero" diventava "è ovviamente falso" nel giro di dieci minuti. Una tesi è un oggetto solitario, ma gli di studio anni che ci sono dietro non lo sono stati.

Alle persone che, purtroppo per loro, hanno a che fare con me quotidianamente. Grazie per non aver mai chiesto cosa sia un sistema di dimostrazione e che, anche solo per questo, hanno preservato la mia sanità mentale. Per le serate e le discussioni in cui l'unico problema aperto era dove uscire. E sì, finalmente posso rispondere alla domanda "allora, quando ti laurei?" con una data. Grazie a Sara, Gaia e Clara che mi accompagnano da sempre e grazie a Lorenzo, Micol, Giulia e Valentina per sopportarmi ogni giorno, a patto che io abbia la decenza di farmi vivo ovviamente.

Alla mia famiglia. Grazie per il sostegno, per i sacrifici e per aver creduto in me molto prima che ci fosse qualcosa in cui credere. Mi avete lasciato la libertà di scegliere e la certezza di avere sempre un posto in cui tornare. Spero che questo traguardo, come quelli futuri, sia percepito da voi anche come vostro, perché lo è: ogni riga di questa tesi ha dietro un pezzo del vostro lavoro e nessun risultato mio sarà mai importante quanto questo.

A Camilla. Il nostro rapporto si è sviluppato proprio come la ricerca di un risultato: l'esito sembrava vero fin da subito, ma è stato necessario sviare ed esplorare altre strade, alcune difficili, per poterlo dimostrare. Hai ascoltato spiegazioni che non eri tenuta a seguire, festeggiato risultati che non potevi decifrare e consolato fallimenti che non potevi verificare. Te lo dico sempre e qui lo rivendico: sei l'unica persona che ritengo al di sopra di me. Sei al di fuori di questo mondo, difatti non nego che il vestito verde da te indossato al nostro primo incontro suggerisse il tuo essere aliena. Fortuna vuole che la mia ossessione sia lo studio di cose complesse, intrattabili ed incomprensibili.

A me stesso, per non aver abbandonato il progetto nelle settimane in cui il risultato principale sembrava impossibile da raggiungere e per aver finalmente capito che una scadenza è un teorema a cui si crede solo dopo averlo dimostrato.

Infine, a tutti quelli che ho dimenticato di nominare: l'omissione è un limite della mia memoria, ma non della mia gratitudine. Per assicurare la correttezza ho dovuto sacrificare la completezza, ma non posso neanche dimostrare la mia stessa correttezza. Dannato Gödel e i suoi due stupidi teoremi.

Simone Bianco, Roma, Ottobre 2026

Bibliography

[AAR+25] Noel Arteche, Albert Atserias, Susanna de de Rezende, et al. "The Proof Analysis Problem". In: *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS'25)*. IEEE Computer Society, 2025, pp. 2555–2576. DOI: 10.1109/FOCS63196.2025.00133.

[Aar17] Scott Aaronson. *P ?= NP*. Tech. rep. 2017. URL: scottaaronson.blog/?p=3095.

[AB04] Albert Atserias and Maria Luisa Bonet. "On the automatizability of resolution and related propositional proof systems". In: *Information and Computation* 189.2 (2004). DOI: https://doi.org/10.1016/j.ic.2003.10.004.

[AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. 1st. USA: Cambridge University Press, 2009. ISBN: 0521424267.

[ABM+01] Michael Alekhnovich, Sam Buss, Shlomo Moran, et al. "Minimum Propositional Proof Length Is NP-Hard to Linearly Approximate". In: *The Journal of Symbolic Logic* (2001). URL: http://www.jstor.org/stable/2694916.

[ABR+02] Michael Alekhnovich, Eli Ben-Sasson, Alexander A. Razborov, et al. "Space Complexity in Propositional Calculus". In: *SIAM Journal on Computing* (2002). DOI: 10.1137/S0097539700366735.

[ACG24] Noel Arteche, Gaia Carenini, and Matthew Gray. "Quantum Automating TC^0-Frege Is LWE-Hard". In: *Proceedings of the 39th Computational Complexity Conference*. CCC '24. Dagstuhl, DEU: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2024. DOI: 10.4230/LIPIcs.CCC.2024.15.

[AD08] Albert Atserias and Víctor Dalmau. "A combinatorial characterization of resolution width". In: *Journal of Computer and System Sciences* (2008). DOI: https://doi.org/10.1016/j.jcss.2007.06.025.

[Ajt94] M. Ajtai. "The complexity of the Pigeonhole Principle". In: *Combinatorica* 14 (1994). DOI: https://doi.org/10.1007/BF01302964.

[AM20] Albert Atserias and Moritz Müller. "Automating Resolution is NP-Hard". In: *J. ACM* (2020). DOI: 10.1145/3409472.

[AR08] Michael Alekhnovich and Alexander A. Razborov. "Resolution Is Not Automatizable Unless W[P] Is Tractable". In: *SIAM Journal on Computing* (2008). DOI: 10.1137/06066850X.

[ARK+26] Noel Arteche, Susanna de Rezende, Erfan Khaniki, et al. "The Proof Analysis Problem for Constant-Depth Frege". In: *2026 IEEE 67th Annual Symposium on Foundations of Computer Science (FOCS'26)*. To appear. IEEE Computer Society, 2026.

[Bd02] Harry Buhrman and Ronald de Wolf. "Complexity measures and decision tree complexity: a survey". In: *Theoretical Computer Science* (2002), pp. 21–43. DOI: https://doi.org/10.1016/S0304-3975(01)00144-X.

[BDG+04] Maria Luisa Bonet, Carlos Domingo, Ricard Gavaldà, et al. "Non-Automatizability of Bounded-Depth Frege Proofs". In: *Computational Complexity* 68.47 (2004). DOI: 10.1007/s00037-004-0183-5.

[BEG+98] Maria Luisa Bonet, Juan Luis Esteban, Nicola Galesi, et al. "On the Relative Complexity of Resolution Refinements and Cutting Planes Proof Systems". In: *SIAM J. Comput.* (1998). URL: https://api.semanticscholar.org/CorpusID:7512512.

[Bey06] Olaf Beyersdorff. "Disjoint NP-Pairs from Propositional Proof Systems". In: Springer Berlin Heidelberg, 2006, pp. 236–247. DOI: https://doi.org/10.1007/11750321_23.

[BG01] Maria Luisa Bonet and Nicola Galesi. "Optimality of size-width tradeoffs for resolution". In: *Computational complexity* (2001). DOI: https://doi.org/10.1007/s000370100000.

[BG94] Mihir Bellare and Shafi Goldwasser. "The Complexity of Decision Versus Search". In: *SIAM Journal on Computing* 23.1 (1994), pp. 97–119. DOI: 10.1137/S0097539792228289.

[BGI+01] Sam Buss, Dima Grigoriev, Russell Impagliazzo, et al. "Linear Gaps between Degrees for the Polynomial Calculus Modulo Distinct Primes". In: *Journal of Computer and System Sciences* (2001). DOI: https://doi.org/10.1006/jcss.2000.1726.

[BHM+21] Armin Biere, Marijn Heule, Hans van Maaren, et al., eds. *Handbook of Satisfiability*. Washington, DC: IOS Press, 2021.

[BIK+94] P. Beame, R. Impagliazzo, J. Krajicek, et al. "Lower bounds on Hilbert's Nullstellensatz and propositional proofs". In: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 1994. DOI: 10.1109/SFCS.1994.365714.

[BKS04] Paul Beame, Henry Kautz, and Ashish Sabharwal. "Towards understanding and harnessing the potential of clause learning". In: *J. Artif. Int. Res.* (2004).

[Bla38] Archie Blake. "Canonical Expressions in Boolean Algebra". PhD thesis. University of Chicago Press, 1938.

[BP96] P. Beame and T. Pitassi. "Simplified and improved resolution lower bounds". In: *Proceedings of 37th Conference on Foundations of Computer Science*. 1996. DOI: 10.1109/SFCS.1996.548486.

[BPR00] Maria Luisa Bonet, Toniann Pitassi, and Ran Raz. "On Interpolation and Automatization for Frege Systems". In: *SIAM Journal on Computing* (2000). DOI: 10.1137/S0097539798353230.

[BPR97] M.L. Bonet, T. Pitassi, and R. Raz. "No feasible interpolation for TC/sup 0/-Frege proofs". In: *Proceedings 38th Annual Symposium on Foundations of Computer Science*. 1997. DOI: 10.1109/SFCS.1997.646114.

[Bus87] Samuel R. Buss. "Polynomial Size Proofs of the Propositional Pigeonhole Principle". In: *The Journal of Symbolic Logic* (1987). URL: http://www.jstor.org/stable/2273826.

[BW01] Eli Ben-Sasson and Avi Wigderson. "Short proofs are narrow—resolution made simple". In: *J. ACM* (2001). DOI: 10.1145/375827.375835.

[CEI96] Matthew Clegg, Jeffery Edmonds, and Russell Impagliazzo. "Using the Gröbner basis algorithm to find proofs of unsatisfiability". In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing (STOC)*. 1996, pp. 174–183.

[Chv73] V. Chvátal. "Edmonds polytopes and a hierarchy of combinatorial problems". In: *Discrete Mathematics* (1973). DOI: https://doi.org/10.1016/0012-365X(73)90167-2.

[Cob65] Alan Cobham. "The Intrinsic Computational Difficulty of Functions". In: *Logic, methodology and philosophy of science*. Ed. by Yehoshua Bar-Hillel. North-Holland Pub. Co., 1965, pp. 24–30.

[Coo71] Stephen A. Cook. "The complexity of theorem-proving procedures". In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. STOC '71. Association for Computing Machinery, 1971. DOI: 10.1145/800157.805047.

[CP90] Stephen Cook and Toniann Pitassi. "A feasibly constructive lower bound for resolution proofs". In: *Information Processing Letters* 34.2 (1990). DOI: https://doi.org/10.1016/0020-0190(90)90141-J.

[CR74] Stephen Cook and Robert Reckhow. "On the lengths of proofs in the propositional calculus (Preliminary Version)". In: STOC '74. Association for Computing Machinery, 1974. DOI: 10.1145/800119.803893.

[CR79] Stephen A. Cook and Robert A. Reckhow. "The relative efficiency of propositional proof systems". In: *Journal of Symbolic Logic* (1979). DOI: 10.2307/2273702.

[Cra57] William Craig. "Three Uses of the Herbrand-Gentzen Theorem in Relating Model Theory and Proof Theory". In: *The Journal of Symbolic Logic* (1957). URL: http://www.jstor.org/stable/2963594.

[DH76] W. Diffie and M. Hellman. "New directions in cryptography". In: *IEEE Transactions on Information Theory* (1976). DOI: 10.1109/TIT.1976.1055638.

[DLL62] Martin Davis, George Logemann, and Donald Loveland. "A machine program for theorem-proving". In: (1962). DOI: 10.1145/368273.368557.

[DP60] Martin Davis and Hilary Putnam. "A Computing Procedure for Quantification Theory". In: *J. ACM* (1960). DOI: 10.1145/321033.321034.

[End01] H.B. Enderton. *A Mathematical Introduction to Logic*. Academic Press, 2001. ISBN: 9780080496467.

[Gar19] Michal Garlík. "Resolution Lower Bounds for Refutation Statements". In: Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl Leibniz-Zentrum für Informatik, 2019. DOI: 10.4230/LIPIcs.MFCS.2019.37.

[Gar24] Michal Garlík. "Failure of Feasible Disjunction Property for k-DNF Resolution and NP-Hardness of Automating It". In: *39th Computational Complexity Conference (CCC 2024)*. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany, 2024. DOI: 10.4230/LIPIcs.CCC.2024.33.

[Gil74] John T. Gill. "Computational complexity of probabilistic Turing machines". In: *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*. STOC '74. Association for Computing Machinery, 1974. DOI: 10.1145/800119.803889.

[GKM+20] Mika Göös, Sajin Koroth, Ian Mertz, et al. "Automating cutting planes is NP-hard". In: STOC 2020. New York, NY, USA: Association for Computing Machinery, 2020. DOI: 10.1145/3357713.3384248.

[GL10a] Nicola Galesi and Massimo Lauria. "On the Automatizability of Polynomial Calculus". In: *Theory of Computing Systems* 506.491 (2010).

[GL10b] Nicola Galesi and Massimo Lauria. "Optimality of size-degree tradeoffs for polynomial calculus". In: *ACM Trans. Comput. Logic* (2010). DOI: 10.1145/1838552.1838556.

[Gom58] Ralph E. Gomory. "Outline of an algorithm for integer solutions to linear programs". In: *Bulletin of the American Mathematical Society* (1958).

[Gri98] D. Grigoriev. "Tseitin's tautologies and lower bounds for Nullstellensatz proofs". In: *Proceedings 39th Annual Symposium on Foundations of Computer Science (Cat. No.98CB36280)*. 1998. DOI: 10.1109/SFCS.1998.743515.

[GS84] J. Grollmann and A.L. Selman. "Complexity Measures For Public-Key Cryptosystems". In: *25th Annual Symposium onFoundations of Computer Science, 1984*. 1984. DOI: 10.1109/SFCS.1984.715952.

[GSS+04] Christian Glasser, Alan L. Selman, Samik Sengupta, et al. "Disjoint NP-Pairs". In: *SIAM Journal on Computing* 33.6 (2004). DOI: 10.1137/S0097539703425848.

[GT26] Nicola Galesi and Jacobo Torán. *An Introduction to Proof Complexity*. Cambridge University Press, 2026.

[Hak20] Tuomas Hakoniemi. "Feasible Interpolation for Polynomial Calculus and Sums-Of-Squares". In: *47th International Colloquium on Automata, Languages, and Programming (ICALP 2020)*. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. DOI: 10.4230/LIPIcs.ICALP.2020.63.

[Hak85] Armin Haken. "The intractability of resolution". In: *Theoretical Computer Science* (1985). Third Conference on Foundations of Software Technology and Theoretical Computer Science. DOI: https://doi.org/10.1016/0304-3975(85)90144-6.

[Har89] Juris Hartmanis. "Gödel, von Neumann and the P =? NP Problem". In: *Current Trends in Theoretical Computer Science*. 1989. DOI: 10.1142/9789812794499_0033.

[Hås23] Johan Håstad. "On small-depth Frege proofs for PHP". In: *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*. 2023. DOI: 10.1109/FOCS57990.2023.00010.

[HS65] J. Hartmanis and R. E. Stearns. "On the Computational Complexity of Algorithms". In: *Transactions of the American Mathematical Society* (1965). URL: http://www.jstor.org/stable/1994208.

[Ins23] Simons Institute. *Conference on Meta-Complexity*. 2023. URL: https://simons.berkeley.edu/programs/Meta-Complexity2023.

[IP99] R. Impagliazzo and R. Paturi. "Complexity of k-SAT". In: *Proceedings. 14th Annual IEEE Conference on Computational Complexity*. 1999, pp. 237–240. DOI: 10.1109/CCC.1999.766282.

[IPS99] R. Impagliazzo, P. Pudlák, and J. Sgall. "Lower bounds for the polynomial calculus and the Gröbner basis algorithm". In: *Computational complexity* 8 (1999). DOI: https://doi.org/10.1007/s000370050024.

[IR22] Dmitry Itsykson and Artur Riazanov. "Automating OBDD proofs is NP-hard". In: *Electron. Colloquium Comput. Complex.* 2022. URL: https://api.semanticscholar.org/CorpusID:248303776.

[IS20] Dmitry Itsykson and Dmitry Sokolov. "Resolution over linear equations modulo two". In: *Annals of Pure and Applied Logic* 171.1 (2020). DOI: https://doi.org/10.1016/j.apal.2019.102722.

[Iwa97] Kazuo" Iwama. "Complexity of finding short resolution proofs"". In: *Mathematical Foundations of Computer Science*. Springer Berlin Heidelberg, 1997. DOI: https://doi.org/10.1007/BFb0029974.

[Jeř07] Emil Jeřábek. "On Independence of Variants of the Weak Pigeonhole Principle". In: *Journal of Logic and Computation* 17.3 (June 2007), pp. 587–604. DOI: 10.1093/logcom/exm017.

[Juk12] Stasys Jukna. *Boolean Function Complexity*. Springer Berlin, Heidelberg, 2012. ISBN: 978-3-642-24507-7. DOI: https://doi.org/10.1007/978-3-642-24508-4.

[Kar72] Richard M. Karp. "Reducibility among Combinatorial Problems". In: *Complexity of Computer Computations: Proceedings of a symposium on the Complexity of Computer Computations*. Springer US, 1972. DOI: 10.1007/978-1-4684-2001-2_9.

[KP98] Jan Krajíček and Pavel Pudlák. "Some Consequences of Cryptographical Conjectures forS12and EF". In: *Information and Computation* 140.1 (1998). DOI: https://doi.org/10.1006/inco.1997.2674.

[Kra01] Jan Krajíček. "On the weak pigeonhole principle". In: *Fundamenta Mathematicae* (2001).

[Kra19] Jan Krajíček. *Proof Complexity*. Cambridge University Press, 2019. ISBN: 9781108242066. DOI: https://doi.org/10.1017/9781108242066.

[Kra97] Jan Krajíček. "Interpolation Theorems, Lower Bounds for Proof Systems, and Independence Results for Bounded Arithmetic". In: *The Journal of Symbolic Logic* (1997). URL: http://www.jstor.org/stable/2275541.

[Las01] Jean B. Lasserre. "An Explicit Exact SDP Relaxation for Nonlinear 0-1 Programs". In: *Integer Programming and Combinatorial Optimization*. Springer Berlin Heidelberg, 2001.

[Lev73] Leonid A. Levin. "Universal Sequential Search Problems". In: *Problems of Information Transmission* 9.3 (1973). URL: https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=ppi&paperid=914&option_lang=eng#forwardlinks.

[Mac92] N. Macrae. *John Von Neumann: The Scientific Genius Who Pioneered the Modern Computer, Game Theory, Nuclear Deterrence and Much More*. Pantheon Books, 1992. ISBN: 9780679413080.

[MPW19] Ian Mertz, Toniann Pitassi, and Yuanhao Wei. "Short Proofs Are Hard to Find". In: *International Colloquium on Automata, Languages and Programming*. 2019. URL: https://api.semanticscholar.org/CorpusID:27905156.

[Pap03] Christos H. Papadimitriou. "Computational complexity". In: *Encyclopedia of Computer Science*. GBR: John Wiley and Sons Ltd., 2003. ISBN: 0470864125.

[Pap24] Theodoros Papamakarios. "Depth-d Frege Systems Are Not Automatable Unless P = NP". In: *39th Computational Complexity Conference (CCC 2024)*. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany, 2024. DOI: 10.4230/LIPIcs.CCC.2024.22.

[PBI93] Toniann Pitassi, Paul Beame, and Russell Impagliazzo. "Exponential lower bounds for the pigeonhole principle". In: *Computational complexity* 140 (1993). DOI: https://doi.org/10.1007/BF01200117.

[PD11] Knot Pipatsrisawat and Adnan Darwiche. "On the power of clause-learning SAT solvers as resolution engines". In: *Artificial Intelligence* (2011). DOI: https://doi.org/10.1016/j.artint.2010.10.002.

[PF79] Nicholas Pippenger and Michael J. Fischer. "Relations Among Complexity Measures". In: *J. ACM* (1979). DOI: 10.1145/322123.322138.

[PI00] Pavel Pudlák and Russell Impagliazzo. "A lower bound for DLL algorithms for k-SAT (preliminary version)". In: *ACM-SIAM Symposium on Discrete Algorithms*. 2000. URL: https://api.semanticscholar.org/CorpusID:12577194.

[Pou92] William Poundstone. *Prisoner's Dilemma: John Von Neumann, Game Theory and the Puzzle of the Bomb*. Doubleday, 1992. ISBN: 978-0385415804.

[PT19] Pavel Pudlák and Neil Thapen. "Random resolution refutations". In: *Computational complexity* (2019). DOI: https://doi.org/10.1007/s00037-019-00182-7.

[Pud00] Pavel Pudlák. "Proofs as Games". In: *The American Mathematical Monthly* 107 (2000). URL: http://www.jstor.org/stable/2589349.

[Pud03] Pavel Pudlák. "On reducibility and symmetry of disjoint NP pairs". In: *Theoretical Computer Science* (2003). DOI: https://doi.org/10.1016/S0304-3975(02)00411-5.

[Pud97] Pavel Pudlák. "Lower Bounds for Resolution and Cutting Plane Proofs and Monotone Computations". In: *The Journal of Symbolic Logic* (1997). URL: http://www.jstor.org/stable/2275583.

[Raz02] Alexander A. Razborov. "Proof Complexity of Pigeonhole Principles". In: *Developments in Language Theory*. Springer Berlin Heidelberg, 2002. DOI: https://doi.org/10.1007/3-540-46011-X_8.

[Raz94] Alexander A. Razborov. "On provably disjoint NP-pairs". In: *BRICS Report Series* 1.36 (1994). DOI: 10.7146/brics.v1i36.21607.

[Raz98] Alexander Razborov. "Lower bounds for the polynomial calculus". In: *Computational complexity* 7 (1998). DOI: https://doi.org/10.1007/s000370050013.

[Reg09] Oded Regev. "On lattices, learning with errors, random linear codes, and cryptography". In: *J. ACM* (2009). DOI: 10.1145/1568318.1568324.

[Rez21] Susanna de Rezende. "Automating Tree-Like Resolution in Time no(log n) Is ETH-Hard". In: *Procedia Computer Science* (2021). DOI: https://doi.org/10.1016/j.procs.2021.11.021.

[RGN+21] Susanna F. de Rezende, Mika Göös, Jakob Nordström, et al. "Automating algebraic proof systems is NP-hard". In: STOC 2021. Association for Computing Machinery, 2021. DOI: 10.1145/3406325.3451080.

[Rob65] J. A. Robinson. "A Machine-Oriented Logic Based on the Resolution Principle". In: *J. ACM* (1965). DOI: 10.1145/321250.321253.

[RSA77] Ronald L. Rivest, Adi Shamir, and Len Adleman. *A Method for Obtaining Digital Signatures and Public-key Cryptosystems*. 1977. URL: https://hdl.handle.net/1721.1/148910.

[SA90] Hanif D. Sherali and Warren P. Adams. "A Hierarchy of Relaxations between the Continuous and Convex Hull Representations for Zero-One Programming Problems". In: *SIAM Journal on Discrete Mathematics* 3.3 (1990). DOI: 10.1137/0403036.

[Sip92] Michael Sipser. "The history and status of the P versus NP question". In: *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*. STOC '92. Association for Computing Machinery, 1992. DOI: 10.1145/129712.129771.

[Sip96] Micheal Sipser. *Introduction to the Theory of Computation*. International Thomson Publishing, 1996.

[Stå96] Gunnar Stålmarck. "Short resolution proofs for a sequence of tricky formulas". In: *Acta Informatica* (1996). DOI: https://doi.org/10.1007/s002360050044.

[Tur37] A. M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society* s2-42.1 (Jan. 1937), pp. 230–265. ISSN: 0024-6115. DOI: 10.1112/plms/s2-42.1.230.

[Urq87] Alasdair Urquhart. "Hard examples for resolution". In: *J. ACM* (1987). DOI: 10.1145/7531.8928.